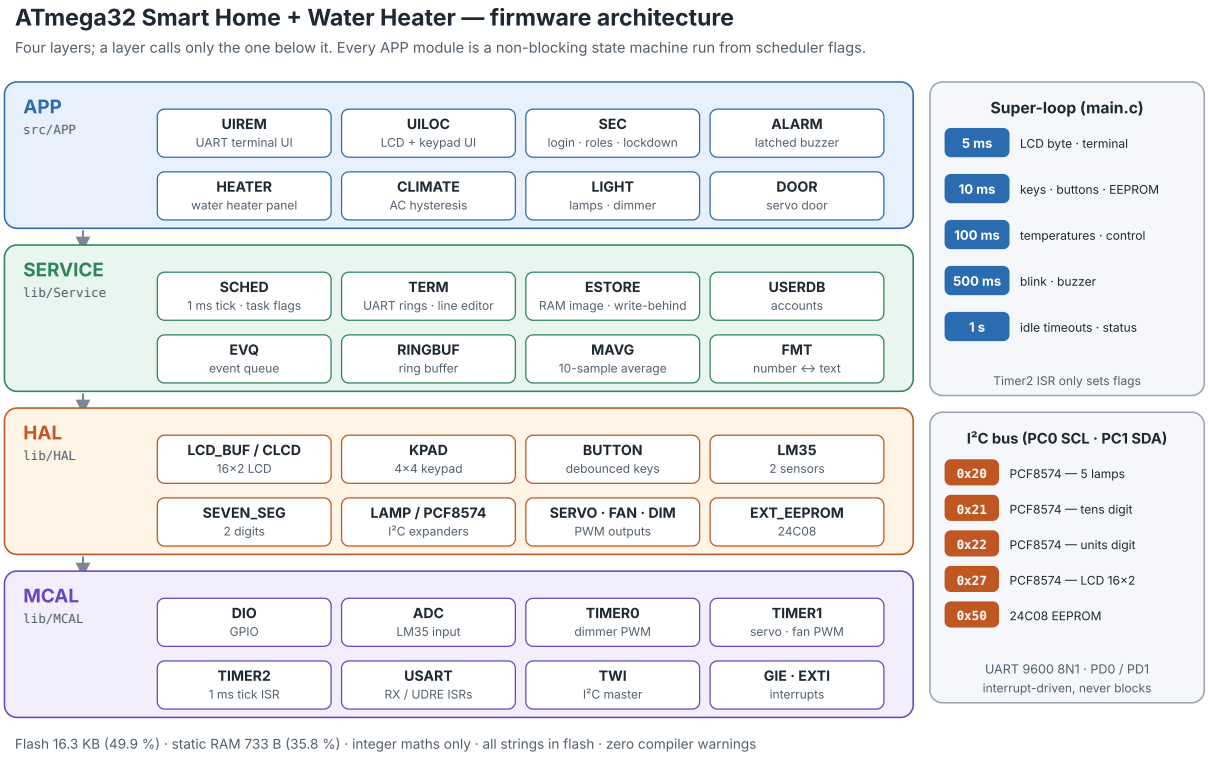
GRADUATION PROJECT · TECHNICAL DOCUMENTATION

ATmega32 Smart Home + Water Heater

Complete Technical Guide — from the register to the running system

Author	Abdelrahman Sherif Medhat
Course	Embedded Systems Diploma, AMIT Learning, 14 Sep 2025 – 8 Apr 2026
Instructor	Eng. Eslam Hefny (author of the course MCAL/HAL base drivers)
Target	ATmega32 @ 16 MHz · bare-metal C (avr-gcc, PlatformIO) · Proteus 8
Firmware	firmware/project_v1, release v1.0



Course drivers (MCAL/HAL base set) by Eng. Eslam Hefny, AMIT Learning. Project code, documentation and this guide by Abdelrahman Sherif Medhat.

FRONT MATTER

How to read this guide

This guide explains the firmware bottom-up: one hardware block, then the driver for it, then the service that uses the driver, then the application feature, and finally the timing of the whole system. Every number, register value, address and formula is taken from the code in `firmware/project_v1` or computed from it.

Who it is for

- **An embedded student** who knows C and the basics of the AVR but not this code base: read chapters 1–3 in order, then the subsystem you need in chapter 5.
- **The instructor**: chapter 1 (architecture and resource budgets), chapter 6 (the traced scenarios), chapter 7 (timing analysis) and chapter 8 (verification and traceability).
- **The author, months from now**: the "re-implementation checklist" at the end of every module and the appendices (register reference, pin table, EEPROM map, UART commands).

Structure of every module section

Each driver, service and subsystem follows the same template, so you always know where to look:

Purpose	what the module is for and which requirement IDs it serves
Hardware / registers	the chip or the registers, with the reason for every bit that is set
Design	the data the module keeps and the algorithm, including what may happen in an ISR
API	the public functions, exactly as declared in the header
Example	a worked example with real values
Pitfalls	what went wrong or could go wrong in this project
Re-implementation checklist	the steps to write the module again from scratch

Conventions

- Code excerpts are copied from the source file named above them, with the original line numbers. No excerpt is longer than 15 lines; blank lines are left out.
- Module names are written as in the code: `HEATER`, `ESTORE`. Functions follow the course style `MODULE_<type><Name>`, for example `LAMP_u8SetState`.
- Requirement IDs (`SEC-05`, `HTR-09` ...) are those of `docs/specification.md` §6; in the code they appear as `/* REQ-HTR-09 */`. Design decisions `D-1` ... `D-20` are in `docs/architecture.md` §9.
- Colours follow the architecture diagram: **blue = APP**, **green = SERVICE**, **orange = HAL**, **purple = MCAL**.
- Module tags: E course driver used as it is, F course driver fixed or extended, N new module written in this project.

Where the numbers come from

There is no hardware-in-the-loop. Every timing figure in this guide carries one of these labels:

Label	Meaning
computed	derived on paper from `F_CPU = 16 000 000` (from `platformio.ini`), the `_cfg.h` constants and the ATmega32 data sheet. The generator script reads the constants from the headers, so a changed constant changes the guide.
simavr	measured by running the unchanged `app.elf` in simavr, a cycle-accurate AVR simulator, with every external part emulated (I²C chips, keypad, buttons, LM35, terminal). Three bugs of the simavr 1.6 TWI/ADC models were fixed and the I²C bit time was tied to TWBR (§7.1). Method and limits: §7.1.
Proteus	measured or observed in Proteus 8 by the author, as recorded in the repository (`docs/`, test sequence PDFs, presentation captures).

Figures without a timing label are drawn from the source code.

Contents

System and architecture 6
1.1 Requirements 6
1.2 Layers and the rules between them 7
1.3 Pin map and the 32-pin budget 7
1.4 I²C bus and address map 9
1.5 Timers and peripherals 9
1.6 Memory budget 10
1.7 Build environments 10
1.8 Boot order 11

MCAL — the register layer 12
2.0 Common ground: registers, bits and interrupts 12
2.1 DIO — digital I/O MCALF 12
2.2 GIE — global interrupt enable MCALE 13
2.3 ADC — analog-to-digital converter MCALF 13
2.4 TIMER0 — 8-bit PWM for the dimmer MCALF 15
2.5 TIMER1 — 16-bit PWM for the servo and the fan MCALN 16
2.6 TIMER2 — the 1 ms system tick MCALN 18
2.7 USART — 9600 8N1, interrupt driven MCALF 19
2.8 TWI — the I²C master MCALF 21
2.9 EXTI — external interrupts MCALF 22

HAL — one driver per part 23
3.1 PCF8574 — I²C port expander HALF 23
3.2 CLCD — character LCD over I²C HALF 25
3.3 LCD_BUF — the non-blocking LCD HALN 26
3.4 KPAD — 4×4 keypad HALF 29
3.5 BUTTON — heater push buttons HALN 30
3.6 LM35 — temperature sensors HALN 31
3.7 SEVEN_SEG — two-digit display HALN 33
3.8 LAMP — five lamps on a PCF8574 HALN 34
3.9 RELAY, LED, BUZZER — plain outputs HALN 36
3.10 DIMMER, SERVO, FAN — PWM outputs HALN 37
3.11 EXT_EEPROM — the 24C08 HALN 38

SERVICE — time, text, storage and events 40
4.1 SCHED — time base and task flags SERVICEN 40
4.2 RINGBUF — lock-free byte ring SERVICEN 43
4.3 TERM — UART terminal I/O SERVICEN 43
4.4 EVQ — event queue SERVICEN 46
4.5 ESTORE — EEPROM image with write-behind SERVICEN 46
4.6 USERDB — accounts SERVICEN 49
4.7 MAVG — 10-sample moving average SERVICEN 50
4.8 FMT — number ↔ text without printf SERVICEN 50
4.9 flash_str.h — text in flash 50

Subsystems — the APP layer 52
5.1 SEC — sessions, roles and the lockdown counter APPN 52
5.2 ALARM — latched lockdown APPN 53
5.3 LIGHT — lamps and dimmer APPN 54
5.4 DOOR — servo door, admin only APPN 54
5.5 CLIMATE — air conditioning with hysteresis APPN 55
5.6 HEATER — the water heater APPN 57
5.7 UILOC — keypad and LCD APPN 60
5.8 UIREM — the UART terminal APPN 62

The top-level application 64
6.1 The super-loop 64
6.2 Data ownership and event flow 64
6.3 Scenario 1 — remote login, set temperature, EEPROM save 66

6.4 Scenario 2 — a keypad user toggles a lamp 67
6.5 Scenario 3 — the third wrong login 67

System timing analysis 69
7.1 Method 69
7.2 From the clock to the timers 69
7.3 Task schedule, hyperperiod and I²C budget 70
7.4 Execution times and CPU load 72
7.5 Latency chains 73
7.6 Jitter, overruns and headroom 74

Verification 76
8.1 Test environments 76
8.2 The Proteus test plan and the 130 recorded APP steps 77
8.3 Requirement traceability 78
8.4 Independent cross-check in simavr 79
8.5 Design decisions and failures 79
8.6 Known limitations 80

Reference 82
A. Register quick reference 82
B. Full pin table 82
C. EEPROM byte map (24C08, block 0) 83
D. UART command list 83
E. Glossary 84
F. Interview questions about this project 84
G. Documentation errata (code is the reference) 85
H. Regenerating this guide 86

The system on one page

One ATmega32 runs two course projects as one product: a **smart home** (remote UART terminal, keypad + LCD, accounts in EEPROM, five lamps, a dimmable lamp, a servo door, temperature-controlled air conditioning, an alarm) and an **electric water heater** (own panel with Up/Down/ON-OFF buttons, two-digit display, heating and cooling elements, 10-sample averaged control with a ±5 °C band). The heater is one more appliance of the smart home: the terminal and the keypad show and change it as well.

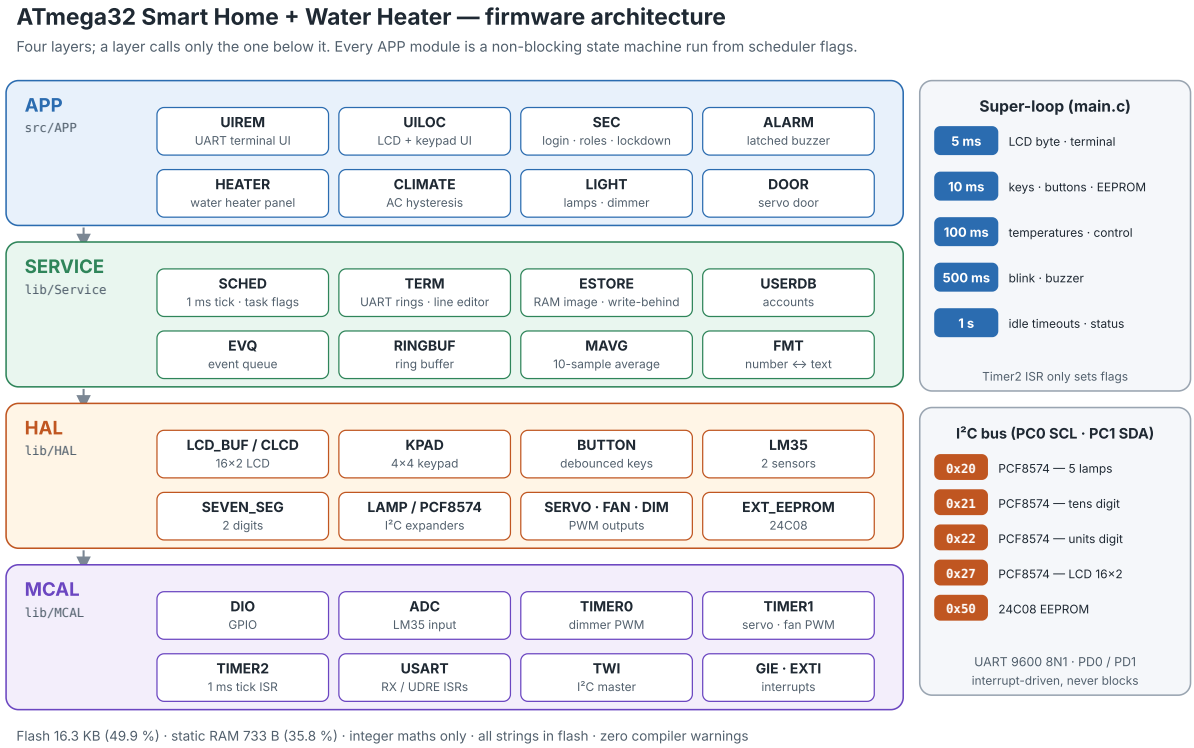


Figure 0.1 Firmware architecture (`docs/images/architecture.svg`). Four layers; a layer only calls the one below it. Two approved exceptions: `SCHED → TIMER2` and `TERM → USART` (D-2). reused from the repository

Key facts

CPU clock	16 MHz
System tick (Timer2 CTC)	1.000 ms
Periodic tasks	5 / 10 / 100 / 500 / 1000 ms
Worst loop pass (simavr)	2.05 ms
CPU load (simavr)	4.1–7.2 %
Flash / static RAM	16 344 B (49.9 %) / 733 B (35.8 %)
Modules	42 (10 MCAL, 16 HAL, 8 SERVICE, 8 APP)
I/O pins	24 used, 2 reserved, 6 spare

Interfaces

UART 9600 8N1	remote terminal (Bluetooth / USB-TTL)
I²C 100 kHz	LCD 0x27, lamps 0x20, 7-seg 0x21/0x22, 24C08 0x50
Timer0 PWM 7.8 kHz	dimmer (PB3)
Timer1 PWM 50 Hz	door servo (PD5), AC fan (PD4)
ADC, 2.56 V ref	LM35 ambient (PA0), water (PA1)
GPIO	keypad 4×4, 3 buttons, 2 elements, LED, buzzer

How it runs

The Timer2 ISR only counts and sets task flags. `main()` polls the flags and runs short, non-blocking state machines. The UART is interrupt-driven through ring buffers; I²C is polled from the main loop only, never from an ISR. EEPROM writes go out one page per 10 ms slot from a RAM image.

CHAPTER 1

System and architecture

What the system must do, how the firmware is cut into layers, and how the four scarce resources of an ATmega32 — 32 I/O pins, three timers, 32 KB of flash and 2 KB of RAM — were shared out.

1.1 Requirements

The course offered two separate projects, a Smart Home and an Electric Water Heater (`docs/course_brief_graduation_projects.pdf`). The specification merges them and gives every requirement an ID. Where the brief was ambiguous, a numbered decision was recorded (specification §12). The table lists all 43 requirement IDs, where each is implemented and which Proteus steps of `test_plan.md` cover it.

ID	Requirement (short form of specification §6)	Implemented in	Test steps
SEC-01	Two roles; admin logs in only over UART, users over keypad or UART	SEC, UIREM, UILOC	T-01, T-02
SEC-02	Separate keypad and remote user lists; keypad names numeric	USERDB, SEC	T-04
SEC-03	Admin adds/removes users of both lists from the terminal	UIREM, USERDB	T-03, T-05, T-06
SEC-04	Accounts survive power-off (EEPROM)	USERDB, ESTORE	T-07
SEC-05	3rd wrong login in a row (per source) → lockdown + alarm until reset	SEC, ALARM	T-08 – T-12, T-73
SEC-06	Users control everything except the door	UIREM, UILOC	T-13, T-14
SEC-07	Accounts: admin read/write, users read-only (write gate)	USERDB, SEC	T-15
SEC-08	Keypad works with a remote user; blocked while the admin is logged in until allowed	SEC, UILOC, UIREM	T-16 – T-20
SEC-09	Default admin written on first boot (magic byte)	ESTORE, USERDB	T-01, T-21
LUI-01…04	Keypad/LCD for users only; menu; status screen when idle; PIN shown as `*`	UILOC, LCD_BUF	T-22 – T-32
RUI-01…04	Two-way UART; every change reported; menus per role; clear errors	UIREM, TERM, EVQ	T-33 – T-39
LGT-01	5 on/off lamps	LIGHT, LAMP	T-24, T-40, T-41
LGT-02	1 dimmable lamp, PWM → 0–5 V, 10 % steps	LIGHT, DIMMER	T-25, T-42, T-43
DOR-01/02	Servo door, admin only; 50 Hz PWM, 0° closed, 90° open	DOOR, SERVO, UIREM	T-13, T-44, T-45
AC-01…03	LM35 ambient; fan on above 28 °C, off below 21 °C; PWM fan	CLIMATE, FAN, LM35	T-26, T-46 – T-50
HTR-01/02	First Up/Down press only enters setting mode; then ±5 °C per press	HEATER, BUTTON	T-54, T-55
HTR-03/04	Set range 35–75 °C, default 60; saved to EEPROM and restored	HEATER, ESTORE	T-56, T-60 – T-62
HTR-05/06	OFF at power-up; ON/OFF toggles on release; OFF = all heater outputs off	HEATER	T-51 – T-53, T-69
HTR-07/08	Water sampled every 100 ms; decisions on the average of 10, none before 10	HEATER, MAVG, LM35	T-63, T-64
HTR-09	avg < set−5 → heat; avg > set+5 → cool; between → keep	HEATER, RELAY	T-65, T-66
HTR-10…12	Display water temp; in setting mode the set value blinking at 1 s; 5 s timeout	HEATER, SEVEN_SEG	T-54 – T-59
HTR-13	Heater LED: blinks while heating, steady while cooling, off otherwise	HEATER, LED	T-67 – T-69
HTR-14	Terminal and keypad show and change the heater with the same rules	UIREM, UILOC, HEATER	T-27, T-70 – T-72
ALM-01/02	Buzzer on lockdown until reset; PIR (optional, not built)	ALARM, BUZZER	T-08, T-12, T-73 – T-75

ID	Requirement (short form of specification §6)	Implemented in	Test steps
EEP-01…03	External 24C08; layout with magic/version; writes never block	ESTORE, EXT_EEPROM	T-60, T-76 – T-78

Not built in v1 (decision 15 / D-16): the AC fan speed ramp (optional part of AC-03), the PIR alarm (ALM-02) and the LDR. Their pins stay reserved.

1.2 Layers and the rules between them

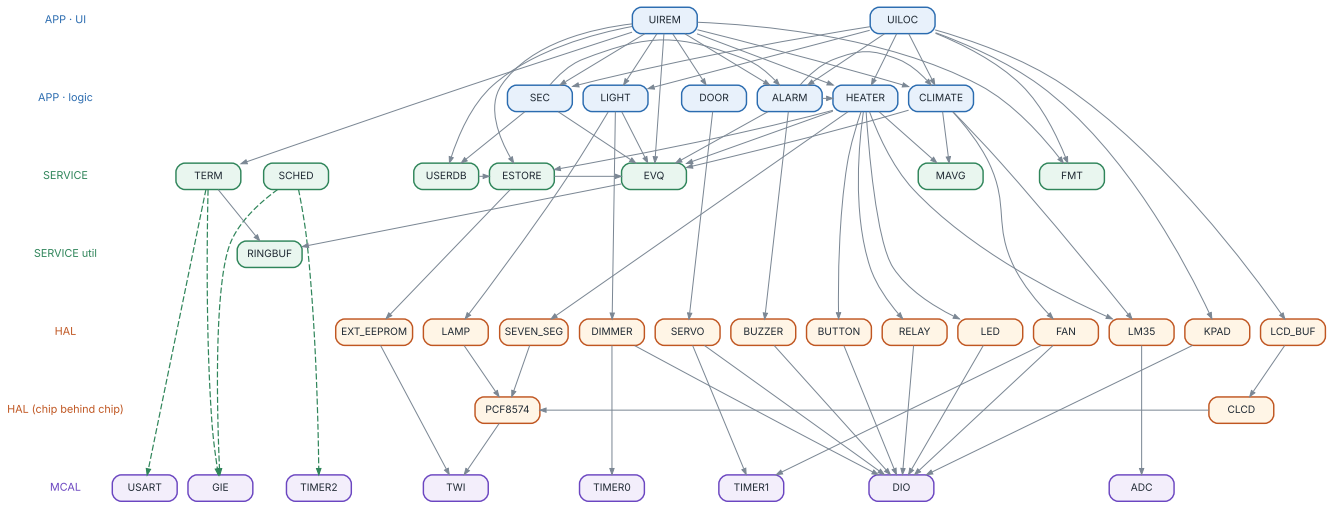


Figure 1.1 Module call graph as built: one arrow = one module calls another (from the `#include`s and calls of every `.c`). Dashed: the two approved SERVICE → MCAL shortcuts. No arrow goes upwards, and no APP module reaches MCAL. drawn from the source code

LAYER RULES (SPECIFICATION §5, ARCHITECTURE §1.1)

1. A layer only calls the layer directly below it; APP may also use SERVICE.
2. `SCHED → TIMER2` and `TERM → USART` skip the HAL because no chip sits in between (D-2).
3. SERVICE utilities (`RINGBUF`, `MAVG`, `FMT`, the type headers) touch no hardware and may be used anywhere.
4. A HAL module may use a lower HAL module when one chip sits behind another: `LCD_BUF → CLCD → PCF8574`, `LAMP → PCF8574`, `SEVEN_SEG → PCF8574`.
5. MCAL never includes an upper header. Interrupts reach upper layers only through callbacks (`MOD_SetCallBack…`).
6. Inside APP: UI → SEC → ALARM → feature, and UI → feature. Feature modules never call a UI; they post events to `EVQ`.
7. APP code never touches a register, a port or a pin number. All pins live in `_cfg.h` files.

Every module is three files, `MOD.c` / `MOD.h` / `MOD_cfg.h`, in its own folder. Of the 42 modules, 3 course drivers are used unchanged (`GIE` is used; `SPI` and `SSG` are compiled but never called), 9 were fixed or extended, and 30 are new.

1.3 Pin map and the 32-pin budget

Wired directly, the full feature list needs about 41 I/O lines (specification §7). The ATmega32 has 32. Four decisions closed the gap:

- the LCD went from a 4-bit parallel bus to a PCF8574 I²C expander (6 pins → 0 extra pins);
- the five lamps sit on a second PCF8574;
- the two 7-segment digits each got their own PCF8574 (D-20). This also removed the 7447 decoder and its 6 port pins;
- the EEPROM is an I²C 24C08.

The I²C bus costs two pins (PC0, PC1) for five chips.

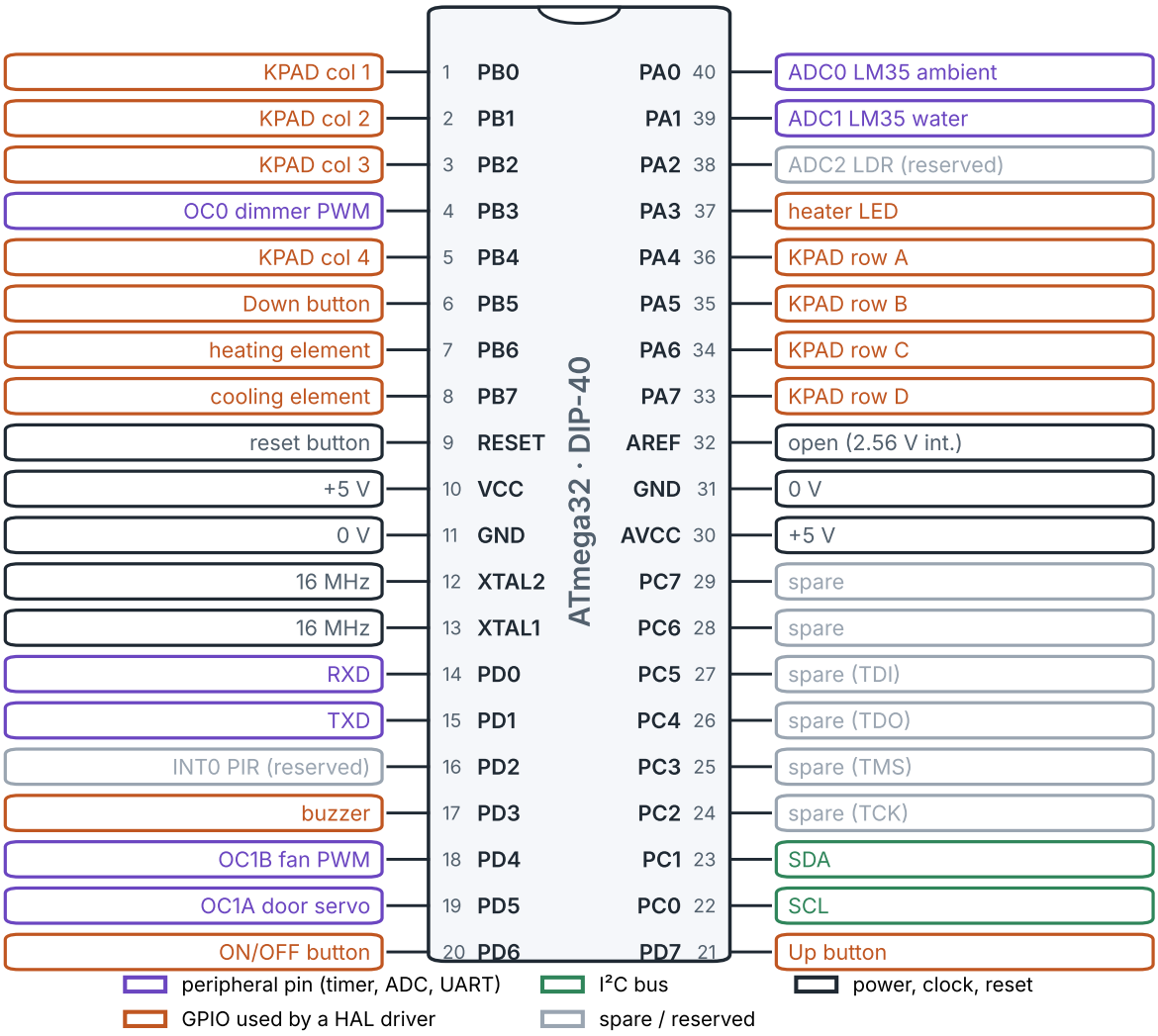


Figure 1.2 ATmega32 DIP-40 pin assignment as built (`docs/pin_map.md`, the `_cfg.h` files). drawn from the source code

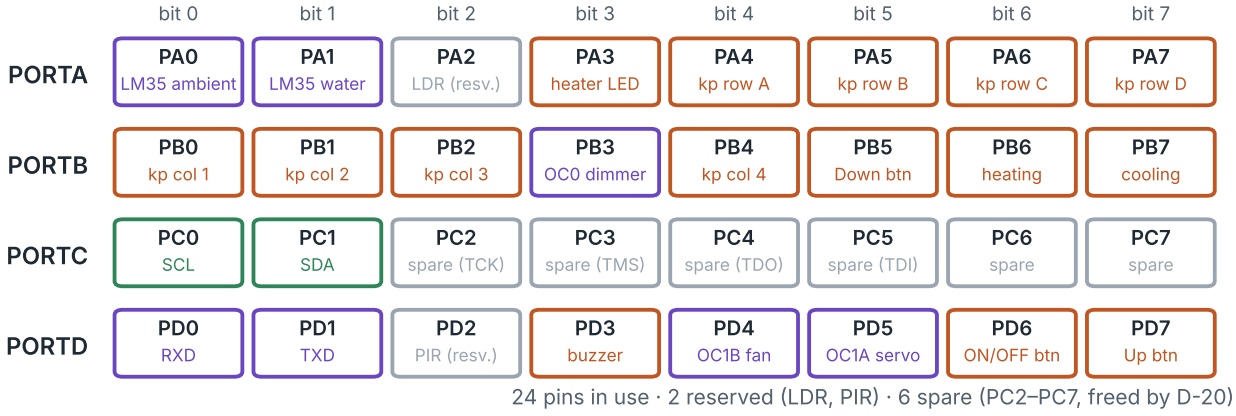


Figure 1.3 The 32 I/O pins by port and bit. drawn from the source code

HARDWARE NOTES KEPT FROM PIN_MAP.MD §4

PB5–PB7 are also the ISP pins (C-4): on a real board, add series resistors so programming does not toggle the heater outputs. AREF must stay open, because the ADC uses the internal 2.56 V reference (C-5). The buzzer needs an NPN driver stage on real hardware (C-9). PC2–PC5 are JTAG pins on a real chip, but they are unused since D-20, so `DIO_voidDisableJTAG()` is not called at boot.

1.4 I²C bus and address map

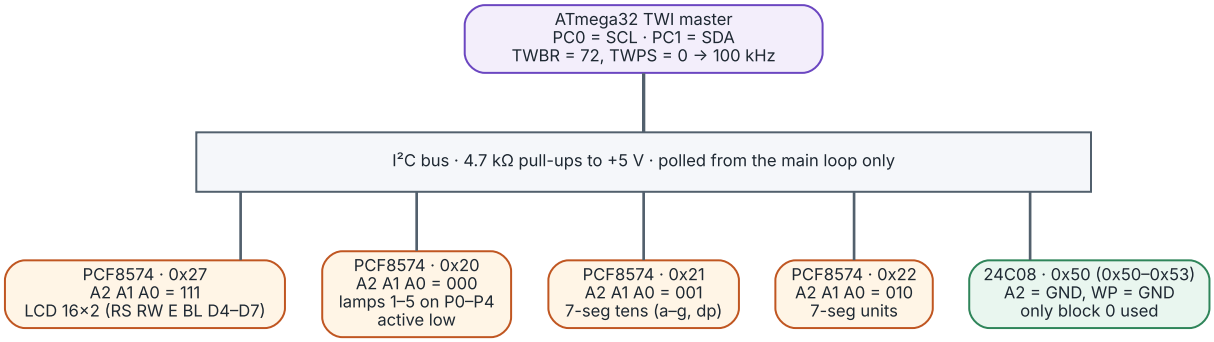


Figure 1.4 One I²C bus, five chips. A PCF8574 has the 7-bit address `0100 A2 A1 A0`; the 24C08 has `1010 A2 B1 B0`, where B1 B0 are address bits 9:8 of the memory. drawn from the source code

Device	7-bit	write / read byte	Driver	Port / content
PCF8574 — LCD	0x27	0x4E / 0x4F	CLCD (via LCD_BUF)	P0 RS, P1 RW, P2 E, P3 backlight, P4–P7 D4–D7
PCF8574 — lamps	0x20	0x40 / 0x41	LAMP	P0–P4 = lamps 1–5, active low; P5–P7 kept high
PCF8574 — tens digit	0x21	0x42 / 0x43	SEVEN_SEG	P0–P6 = segments a–g, P7 = dp; low = lit
PCF8574 — units digit	0x22	0x44 / 0x45	SEVEN_SEG	same as the tens digit
24C08 EEPROM	0x50	0xA0 / 0xA1	EXT_EEPROM (via ESTORE)	bytes 0x000–0x0CF used (block 0 only)

Only the main loop uses the bus; no ISR touches it, so two transactions can never interleave and no bus lock is needed. The PCF8574 is a 100 kHz part, which is why the bus runs at 100 kHz, not 400 kHz.

1.5 Timers and peripherals

Resource	Mode and numbers (computed, chapter 2)	Used by	Pin
Timer0	fast PWM, prescaler 8 → 7 812.5 Hz	DIMMER	PB3 (OC0)
Timer1	mode 14, TOP = ICR1 = 39999, prescaler 8 → 50.000 Hz, 0.5 µs/count	SERVO (A), FAN (B)	PD5, PD4
Timer2	CTC, prescaler 64, OCR2 = 249 → 1.000 ms compare interrupt	SCHED (system tick)	—
USART	9600 8N1, UBRR = 103, RX-complete + UDRE interrupts	TERM	PD0, PD1
TWI	master, polled, TWBR = 72, prescaler 1 → 100 kHz	PCF8574, EXT_EEPROM	PC0, PC1
ADC	internal 2.56 V reference, prescaler 128 → 125 kHz, polled	LM35	PA0, PA1
INT0	not used in v1 (reserved for the PIR)	—	PD2

1.6 Memory budget

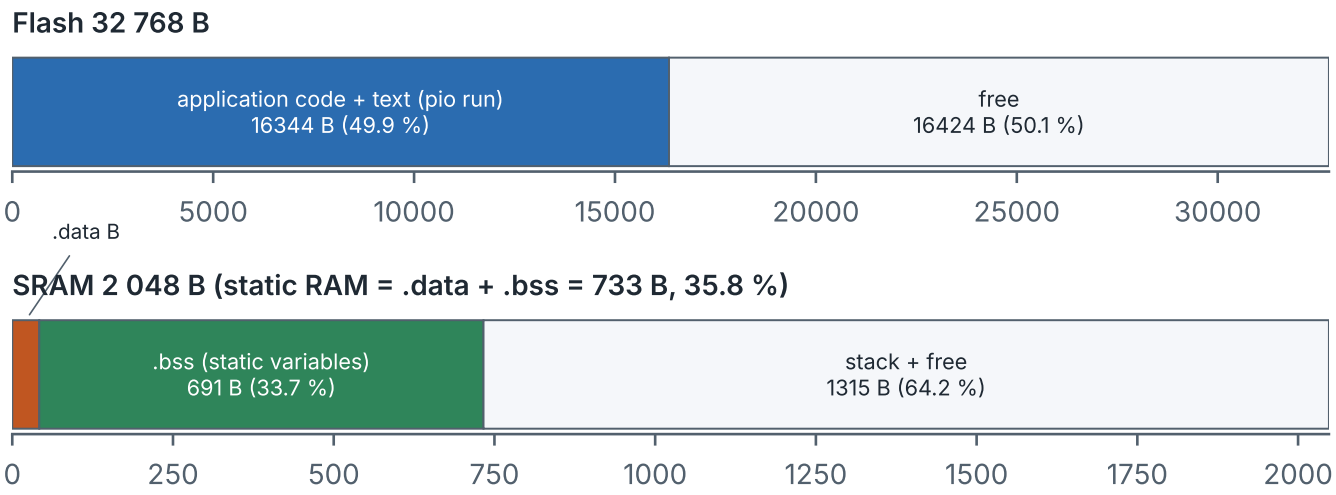


Figure 1.5 Flash and RAM of the `app` build as reported by `pio run` (README). An independent rebuild with avr-gcc 7.3.0 and the PlatformIO flags gives the same static RAM (42 B `.data` + 691 B `.bss` = 733 B) and 16 412 B of flash; the 68-byte flash difference comes from slightly different build flags. drawn from the source code

All text constants live in flash through the GCC `__flash` qualifier and the `FLASH_STR()` macro (§4.9). `<avr/pgmspace.h>` could not be used because it pulls in `<avr/io.h>`, which clashes with the course's `reg_def.h`. Static RAM per module, from each module compiled on its own (`avr-size`, `.data + .bss`):

Module	bytes	main contents	Module	bytes	main contents
ESTORE	215	208-byte EEPROM image + dirty mask + state	SCHED	17	tick (u32), flags, 5 down-counters, overruns
TERM	194	RX 33 + TX 129 storage, line 17, 2 rings	BUTTON	12	level, samples, 2 event flags × 3
LCD_BUF	68	wanted 32 + shown 32 + cursor/scan/wait	SEVEN_SEG	7	number, enabled, shown/known per digit
UIREM	41	state, print queue 6, name 17, target 9	MCAL callback pointers	20	TIMER0 ×2, TIMER2, ADC ×2, EXTI ×3, USART ×2
EVQ	39	33-byte ring storage + ring + mute	SEC	5	role ×2, fail ×2, local-allowed flag
HEATER	31	MAVG (24) + state, set, timer, phase	others	≤ 4 each	KPAD, LIGHT, ALARM, DOOR, USERDB, CLCD, LAMP
UILOC	26	state, ID 9, PIN 9, timers	switch tables (CSWTCH)	34	two compiler tables in `.data`
CLIMATE	25	MAVG (24) + state			

About 1.3 KB remain for the stack. The deepest call path is `main → UIREM → USERDB → ESTORE` with a 17-byte line buffer on the stack, plus one ISR frame (the Timer2 ISR pushes 15 registers and the return address).

1.7 Build environments

`platformio.ini` defines one environment for the application and one per layer test. All share `board = ATmega32`, `board_build.f_cpu = 16000000L` and the include paths `lib/MCAL lib/HAL lib/Service src/APP`; all build with `-Wall` and zero warnings.

Environment	Sources	Libraries	Purpose
`app`	`src/APP/main.c` + 8 APP modules	MCAL HAL Service	the firmware
`test_base`	`test_mains/test_base.c`	MCAL HAL	smoke test of the original course drivers
`test_mcal`	`test_mains/test_mcal.c`	MCAL	MCAL layer test (40 checks)
`test_hal`	`test_mains/test_hal.c`	MCAL HAL	HAL layer test (65 checks + manual steps)

Environment	Sources	Libraries	Purpose
`test_service`	`test_mains/test_service.c`	MCAL HAL Service	SERVICE layer test (140 checks)
`test_app`	`src/APP/` without `main.c` + `test_app.c`	MCAL HAL Service	APP automatic checks, then the full super-loop

`scripts/build_all_hex.ps1` rebuilds all six into `hex/<env>.hex`. Proteus needs the ATmega32 *Clock Frequency* property set to 16 MHz, equal to `F_CPU`; every timing in this guide depends on it.

1.8 Boot order

`main()` does three things: initialise every module in a fixed order, enable interrupts, and run the super-loop (chapter 6). The order matters:

1. **Outputs to a safe state first.** `RELAY`, `LED`, `BUZZER`, `FAN`, `SERVO`, `DIMMER` set the pin level before the pin becomes an output, so neither heater element can switch on during reset (HTR-06).
2. **Communication and the I²C parts:** `TERM` (UART + rings), then `LCD_BUF_voidInit`, the only long blocking call (CLCD power-up delays), then `LAMP`, `SEVEN_SEG`, `KPAD`, `BUTTON`, `LM35`.
3. **Storage:** `ESTORE_voidInit` reads the 208-byte image from the 24C08 (or loads the defaults on a blank chip), then `USERDB_voidInit` repairs a missing admin, then `EVQ_voidInit`.
4. **APP modules:** ALARM, SEC, LIGHT, DOOR, CLIMATE, HEATER, UILOC, UIREM. None of them posts an event or calls `SCHED`, so the queue is empty when the loop starts.
5. **Time base last:** `SCHED_voidInit`, `GIE_voidEnableGlobalInterrupt`, `SCHED_voidStart`. The tick starts only when everything it will drive is ready.

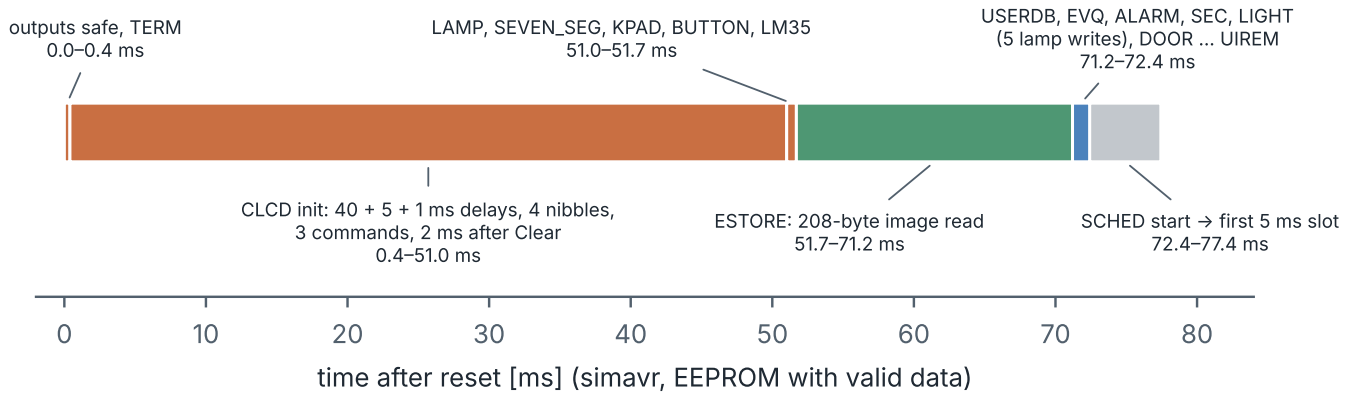


Figure 1.6 Boot phases measured in simavr from the I²C trace (EEPROM with valid data). The CLCD init alone is 50.5 ms computed (48 ms of `_delay_ms` + 2.5 ms of bus time), against 51.0 − 0.4 = 50.6 ms measured. `docs/architecture.md` §4.3 quotes 46 ms (see the discrepancy list). The scheduler starts 72.4 ms after reset, and the first 5 ms task runs at 77.4 ms.
measured in simavr (cycle-accurate simulation of app.elf)

On a blank 24C08 the boot takes the same time: the defaults are built in RAM and the 13 pages are written in the background by the 10 ms task, 20 ms per page. In simavr the page writes start at 84.7 ms and the last one at 324.7 ms, for both the real-chip model (5 ms write cycle) and the Proteus model (no write delay).

CHAPTER 2

MCAL — the register layer

The Microcontroller Abstraction Layer is the only code that touches registers. Most drivers come from the course base set by Eng. Eslam Hefny. They were kept and fixed in their own style rather than rewritten; TIMER1, TIMER2 and the restyled TWI are new.

2.0 Common ground: registers, bits and interrupts

All register addresses are in `lib/MCAL/reg_def.h` as volatile pointers in data-space addresses (I/O address + 0x20), with bit-name macros. No module includes `<avr/io.h>`. Bits are changed with the four macros of `lib/Service/Bit_math.h`:

`lib/Service/Bit_math.h` · lines 11–14 · read-modify-write on any register

```
12  #define SET_BIT(reg,bit)  (reg|=1<<bit)
13  #define CLR_BIT(reg,bit)  (reg&=~(1<<bit))
14  #define GET_BIT(reg,bit)  ((reg>>bit)&1)
```

`SET_BIT(reg,bit)` compiles to a single `sbi` or a read-modify-write. That is safe in the main loop, with one exception that matters here: registers whose flags are cleared by writing a one (`ADIF`, `TWINT`) and registers that share one address (`UBRRH`/`UCSRC`). Each of these is dealt with in its driver below.

PITFALL FOUND IN PHASE 0 — ISRS DROPPED BY LINK-TIME OPTIMISATION
The course drivers declared their ISRs as `void __vector_N() __attribute__((signal));`. Nothing in C calls an ISR by name, so PlatformIO's `-flto` treated the function as unused and removed it: the vector pointed at `__bad_interrupt`, which jumps to address 0, and the MCU reset whenever the interrupt fired. The fix, now used for every ISR, is:

`lib/MCAL/TIMER2/TIMER2.h` · lines 30–30

```
30 void __vector_4 () __attribute__ ((signal, used, externally_visible)) ;
```

Every ISR follows the course pattern: the vector function only calls a callback through a function pointer, after a `NULL` check, so the upper layer decides what happens without MCAL including any upper header. The callbacks of this project are short: the Timer2 callback counts, and the two USART callbacks move one byte.

Vector	Source	Driver	Callback in the application	Measured length (simavr)
`__vector_4`	TIMER2 COMP	TIMER2	`SCHED_voidTick`	235–261 cycles
`__vector_13`	USART RXC	USART	`TERM_voidRxCallback`	122 cycles
`__vector_14`	USART UDRE	USART	`TERM_voidTxCallback`	124–140 cycles
`__vector_1/2/3`, `_10/_11`, `_16`	INT0–2, TIMER0, ADC	EXTI, TIMER0, ADC	not enabled in `app`	—

Cycle counts are from the first instruction of the vector function to the end of `reti`. Add 4 cycles of interrupt response and 3 for the `jmp` in the vector table for the full cost (§7.4).

2.1 DIO — digital I/O MCAL F

PURPOSE

Pin and port access for every HAL driver that drives or reads a GPIO: KPAD, BUTTON, RELAY, LED, BUZZER, and the PWM pins in DIMMER, SERVO and FAN. One addition was made: `DIO_voidDisableJTAG`.

HARDWARE / REGISTERS

Register	Address	Meaning of a 1 bit
DDRA/B/C/D	0x3A / 0x37 / 0x34 / 0x31	the pin is an output
PORTA/B/C/D	0x3B / 0x38 / 0x35 / 0x32	output: drive high · input: enable the internal pull-up (≈ 20–50 kΩ)

Register	Address	Meaning of a 1 bit
PINA/B/C/D	0x39 / 0x36 / 0x33 / 0x30	the level on the pin (read only)
MCUCSR.JTD	0x54 bit 7	JTAG off, PC2–PC5 become I/O; must be written twice within 4 cycles

DESIGN AND API

Every function takes a port ID (`DIO_PORTA` … `DIO_PORTD` = 0…3) and a pin ID (0…7), checks the range and switches on the port. The pull-up function writes the PORT bit, which is what the hardware needs for an input.

```
void DIO_voidSetPinDirection(u8 Port, u8 Pin, u8 Dir);   /* DIO_PIN_OUTPUT / DIO_INPUT */
void DIO_voidSetPinValue    (u8 Port, u8 Pin, u8 Value); /* DIO_PIN_HIGH / DIO_PIN_LOW */
u8   DIO_u8GetPinValue      (u8 Port, u8 Pin);
void DIO_voidSetPortDirection(u8 Port, u8 Dir);  void DIO_voidSetPortValue(u8 Port, u8 Value);
u8   DIO_u8GetPortValue(u8 Port);   void DIO_voidEnablePullUp(u8 Port, u8 Pin);
void DIO_voidTogPin(u8 Port, u8 Pin);   void DIO_voidDisableJTAG();
```

lib/MCAL/DIO/DIO.c · lines 132–138

```
132 void DIO_voidDisableJTAG(){
133     /* PC2..PC5 are JTAG pins. JTD must be written twice within 4 clock cycles.
134      * MCUCSR also holds the reset flags (write 0 = no effect) and ISC2 (keep it). */
135     u8 Local_u8Value = (MCUCSR & (1<<MCUCSR_ISC2)) | (1<<MCUCSR_JTD) ;
136     MCUCSR = Local_u8Value ;
137     MCUCSR = Local_u8Value ;
138 }
```

EXAMPLE — A SAFE OUTPUT

RELAY sets the level first and then the direction: `DIO_voidSetPinValue(DIO_PORTB, DIO_PIN_6, LOW)`, then `DIO_voidSetPinDirection(DIO_PORTB, DIO_PIN_6, OUTPUT)`. If the order were reversed and PORTB.6 still held a 1, the heating element would switch on for a few cycles.

PITFALLS

- MCUCSR also holds the reset-cause flags and ISC2. The JTAG function keeps ISC2 and writes 0 to the flags (writing 0 has no effect on them).
- Reading a pin right after changing a nearby output can return the old level: the pin synchroniser needs about 1.5 cycles. KPAD therefore waits 5 µs after driving a column low (§3.5).

RE-IMPLEMENTATION CHECKLIST

1. Define DDR/PORT/PIN addresses for the four ports.
2. Write set-direction, set-value and get-value per pin with a `switch` on the port.
3. Pull-up = write 1 to PORT while DDR = 0.
4. Range-check the arguments.
5. JTAG disable: compute the MCUCSR value once, then write it twice back-to-back.

2.2 GIE — global interrupt enable MCAL E

Two one-line functions: `GIE_voidEnableGlobalInterrupt()` executes `SEI` and `GIE_voidDisableGlobalInterrupt()` executes `CLI` (SREG bit I). They are used in two places: once in `main()` before the tick starts, and as short critical sections in SERVICE — `SCHED` reads the 32-bit tick and does the test-and-clear of a task flag with interrupts off; `TERM` reads and clears its RX-activity flag the same way. Those sections run only in the main loop, where interrupts are always enabled, so a plain disable/enable pair is correct. A version that saves and restores SREG would only be needed if one were ever called from an ISR.

2.3 ADC — analog-to-digital converter MCAL F

PURPOSE

Ten-bit conversions for the two LM35 sensors (AC-01, HTR-07). The application uses only the blocking (polled) conversion, which takes 104 µs.

HARDWARE / REGISTERS

Bits	Value	Why
`ADMUX.REFS1:0`	11	internal 2.56 V reference: 2560 mV / 1024 = 2.5 mV per LSB, and the LM35 gives 10 mV/°C, so one LSB = 0.25 °C and the raw value is the temperature × 4
`ADMUX.ADLAR`	0	result right-adjusted: read the 16-bit `ADC` (ADCL first, then ADCH)
`ADMUX.MUX4:0`	0 / 1	channel ADC0 (ambient, PA0) or ADC1 (water, PA1); set before each conversion
`ADCSRA.ADPS2:0`	111	prescaler 128: 16 MHz / 128 = 125 kHz, inside the 50–200 kHz range needed for full 10-bit accuracy
`ADCSRA.ADEN`	1	ADC on
`ADCSRA.ADSC`	1 per conversion	start; the hardware clears it at the end
`ADCSRA.ADIF`	poll, then write 1	conversion complete; cleared by writing a one
`ADCSRA.ADIE`	0 in the sync path	an enabled ADC interrupt would clear ADIF and the poll would never end

TIMING (COMPUTED)

t_{conv} = 13 ADC clocks / 125 kHz = **104 µs**. The first conversion after ADEN takes 25 clocks = 200 µs; that is the very first HEATER sample at boot. Each 100 ms task does two conversions (water, then ambient): 208 µs of busy-waiting.

`lib/MCAL/ADC/ADC.c` · lines 28–41 · ADC_u16StartConversion, body

```
28      CLR_BIT(ADCSRA,ADCSRA_ADIE);
29      SET_BIT(ADCSRA,ADCSRA_ADIF);    /* writing 1 clears a stale flag */
30      /* select channel */
31      ADMUX &= 0b11100000;   // mask channel ID
32      ADMUX |= Copy_u8ChannelID;
33      /* start conversion */
34      SET_BIT(ADCSRA,ADCSRA_ADSC);
35      /*polling (wait until conversion finish )*/
36      while(GET_BIT(ADCSRA,ADCSRA_ADIF)==0){
37          /*Busy wait*/
38      }
39      /*clear flag */
40      SET_BIT(ADCSRA,ADCSRA_ADIF);
41      /*return ADC val */
```

API

```
void ADC_voidInit();                                /* 2.56 V ref , /128 , enable */
u16  ADC_u16StartConversion(u8 Channel);            /* blocking , 104 µs */
void ADC_voidStartConvertionAsyn(u8 Channel);       /* with ADIE (not used by the app) */
void ADC_voidDisableInterrupt();                    /* new */
void ADC_SetCallBack(void (*ptr)(void), u16 *data);
```

EXAMPLE

Water at 55 °C → LM35 output 550 mV → ADC = 550 / 2.5 = 220 = 55 × 4. HEATER compares this directly with `(set ± 5) × 4`, so no division or floating point is needed.

PITFALLS (FIXED IN PHASE 2)

- **Sync after async hung.** After one interrupt-driven conversion, ADIE stayed set, the ISR cleared ADIF, and the polling loop waited for ever. `ADC_u16StartConversion` now clears ADIE and a stale ADIF first.
- **Wrong comment.** The course code said AVCC but selected 2.56 V, which is what the LM35 needs; the comment was corrected.
- The ISR lacked `used, externally_visible` (LTO).
- AREF must stay unconnected (pin_map C-5). Tied to 5 V it would fight the internal reference and every temperature would read wrong.

RE-IMPLEMENTATION CHECKLIST

1. Choose the reference from the sensor range (0–255 °C fits 2.56 V).
2. Prescaler so that the ADC clock is 50–200 kHz.
3. Per conversion: clear ADIE and ADIF, set MUX, set ADSC.
4. Poll ADIF, clear it by writing 1, read ADCL then ADCH.
5. Keep the result integer: value × 2.5 mV.

2.4 TIMER0 — 8-bit PWM for the dimmer MCAL F

PURPOSE

Hardware PWM on OC0 (PB3) for the dimmable lamp (LGT-02). An RC filter (R6 10 kΩ, C1 10 µF) turns the PWM into a 0–5 V level for the lamp driver transistor.

HARDWARE / REGISTERS

Bits	Value	Why
`TCCR0.CS02:00`	010	clock F_CPU / 8 = 2 MHz (`DIMMER_TIMER_PRESCALER = TIMER0_DIV_8`)
`TCCR0.WGM01:00`	11	fast PWM, TOP = 0xFF
`TCCR0.COM01:00`	10 (or 00)	non-inverting: OC0 set at BOTTOM, cleared on compare match. 00 = pin disconnected (used for 0 %)
`OCR0`	p × 255 / 100	duty cycle

FORMULAS (COMPUTED)

f_{PWM} = F_CPU / (N × 256) = 16 MHz / (8 × 256) = **7 812.5 Hz**, period 128 µs. In fast PWM the pin is high for OCR0 + 1 counts, so the real duty is (OCR0 + 1) / 256. With the RC filter (τ = 100 ms) the ripple at 7.8 kHz is negligible, and a level change settles in about 0.5 s (5τ).

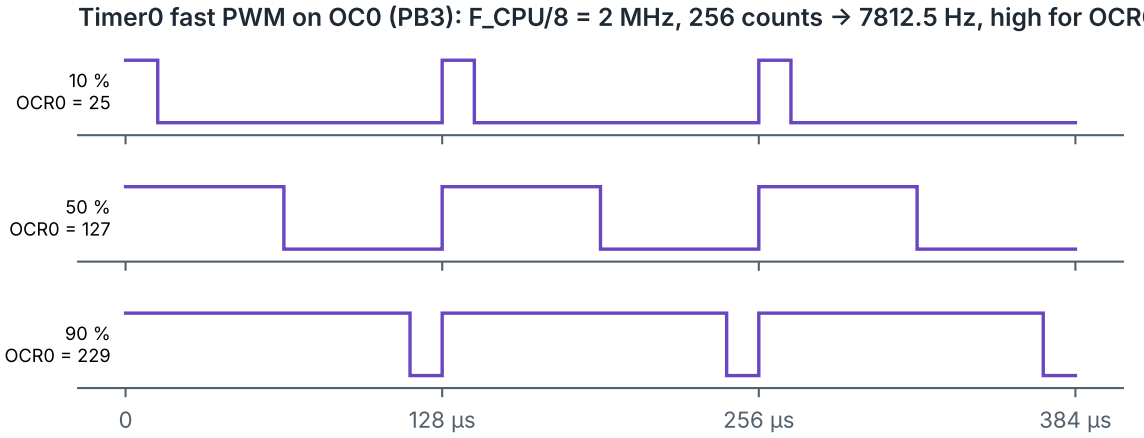


Figure 2.1 Timer0 fast PWM on OC0 for three dimmer levels. computed from the code and F_CPU

Level %	OCR0	High time	Real duty %	Level %	OCR0	High time	Real duty %
0	—	pin low, disconnected	0	60	153	77.0 µs	60.16
10	25	13.0 µs	10.16	70	178	89.5 µs	69.92
20	51	26.0 µs	20.31	80	204	102.5 µs	80.08
30	76	38.5 µs	30.08	90	229	115.0 µs	89.84
40	102	51.5 µs	40.23	100	255	constant high	100
50	127	64.0 µs	50.00				

lib/MCAL/TIMER0/TIMER0.c · lines 103–107 · TIMER0_GeneratePWM, non-inverting case after the fix

```
103     case TIMER0_PWM_NONINVERTED:
104         /* COM01:COM00 = 10 , non-inverting mode */
105         CLR_BIT(TCCR0,TCCR0_COM00);
106         SET_BIT(TCCR0,TCCR0_COM01);
107         OCR0 =((u16)DutyCycle*255)/100;    /* 100 % -> 255 (256 would wrap to 0) */
```

API

```
void TIMER0_voidInit(u8 Prescaler, u8 Mode);   /* TIMER0_DIV_8 , TIMER0_FAST_PWM */
void TIMER0_GeneratePWM(u8 Mode, u8 DutyCycle); /* TIMER0_PWM_DISCONNECTED / INVERTED / NONINVERTED , 0..100
*/
void TIMER0_voidSetOCR(u8);  TIMER0_SetCallBack_OV/OC(...);  (CTC and interrupt functions unused)
```

PITFALLS (FIXED IN PHASE 2)

- **Swapped COM bits.** The course code set COM01:00 = 11 for "non-inverting", which is the inverting mode, so the lamp was brightest at 0 %. `test_base` reported it as `[FAIL]`.
- **100 % wrapped to 0.** `OCR0 = Duty × 256 / 100` gave 256 for 100 %, which an 8-bit register stores as 0. It is now `× 255`, computed in 16 bits.
- **0 % is not OCR0 = 0.** In fast PWM, OCR0 = 0 still produces a one-count spike every period. DIMMER disconnects the pin (COM = 00) and drives it low instead.
- A `case` label carried the wrong name (`TIMER0_CTC_DISCONNECTED` for `TIMER0_NORMAL`; same value 0). The name was corrected.

RE-IMPLEMENTATION CHECKLIST

1. Prescaler from the wanted PWM frequency (F_CPU / (N · 256)).
2. WGM = fast PWM, COM = 10.
3. OCR = p · TOP / 100 in 16-bit arithmetic.
4. 0 % → disconnect and drive low; 100 % → OCR = TOP.
5. Pin = output before enabling COM.

2.5 TIMER1 — 16-bit PWM for the servo and the fan MCAL N

PURPOSE

Two PWM channels with one period: OC1A (PD5) drives the door servo (DOR-02), OC1B (PD4) the AC fan transistor (AC-03). Both SERVO and FAN call `TIMER1_voidInit`; a second call writes the same values and changes nothing.

HARDWARE / REGISTERS

Bits	Value	Why
`TCCR1A.WGM11:10, TCCR1B.WGM13:12`	10, 11	mode 14: fast PWM with TOP = ICR1, so the period is set by ICR1 and both OCR1A and OCR1B stay free for duty cycles
`TCCR1B.CS12:10`	010	F_CPU / 8 = 2 MHz → 0.5 µs per count
`ICR1`	39999	TOP = F_CPU / (8 × 50 Hz) − 1
`TCCR1A.COM1A1:0 / COM1B1:0`	10 or 00	non-inverting per channel; 00 = disconnected (fan at 0 %)
`OCR1A / OCR1B`	see tables	high time in counts

lib/MCAL/TIMER1/TIMER1_cfg.h · lines 12–18 · all numbers derived from F_CPU

```
12 #define TIMER1_PRESCALER          8UL                /* CS11 */
13 #define TIMER1_PWM_FREQ_HZ        50UL               /* 20 ms period for the servo */
14 /* 16 MHz : 16000000 / (8 * 50) - 1 = 39999 */
15 #define TIMER1_TOP_VALUE          ((u16)((F_CPU / (TIMER1_PRESCALER * TIMER1_PWM_FREQ_HZ)) - 1))
16 /* 16 MHz : 2000 timer ticks per ms (1 tick = 0.5 us) */
17 #define TIMER1_TICKS_PER_MS       ((u16)(F_CPU / (TIMER1_PRESCALER * 1000UL)))
```

16-bit registers go through the shared TEMP latch: the **high byte must be written first**, then the low byte, which commits both at once. `TIMER1_voidSetOCRA/B` and the ICR1 write in `TIMER1_voidInit` do exactly that.

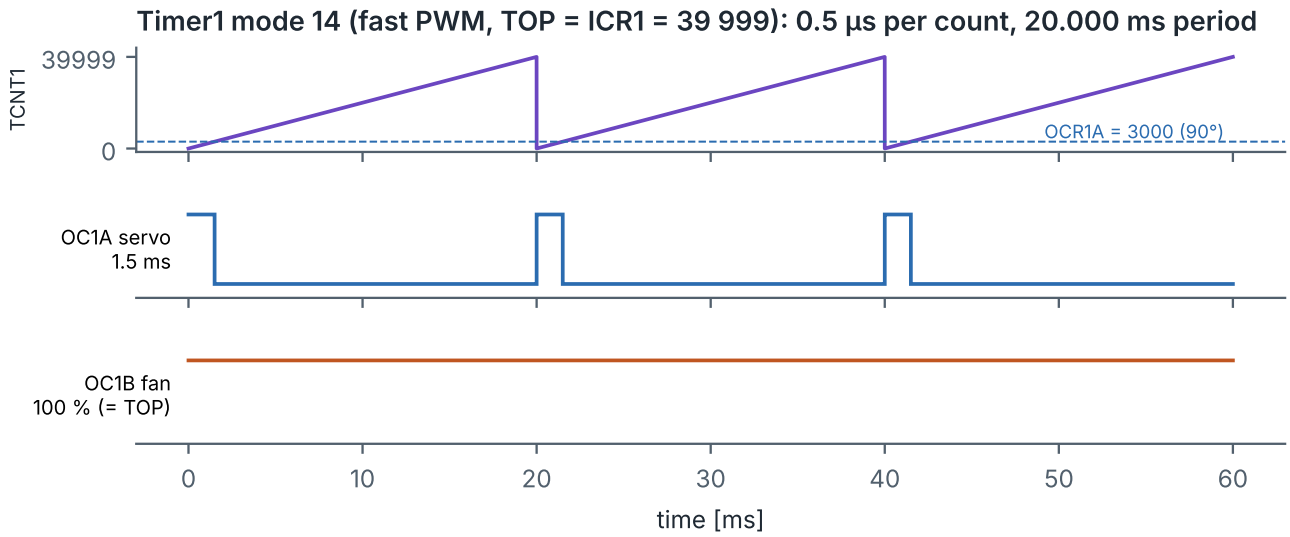


Figure 2.2 Timer1 mode 14: one 20 ms ramp, the servo pulse on OC1A and the fan on OC1B. computed from the code and F_CPU

Servo (OC1A): OCR1A = pulse_us × 2000 / 1000

Angle	Pulse µs	OCR1A	Real high time µs
0° (closed)	1000	2000	1000.5
45°	1250	2500	1250.5
90° (open)	1500	3000	1500.5
180°	2000	4000	2000.5

Fan (OC1B): OCR1B = p × (TOP + 1) / 100

Speed	OCR1B	Output
0 %	—	disconnected, pin low
10 %	4000	2.0005 ms of 20 ms
50 %	20000	10.0005 ms of 20 ms
100 %	39999	constant high (= TOP)

CLIMATE uses only 0 % and 100 % (`CLIMATE_FAN_SPEED 100`); the speed ramp is not built (D-16).

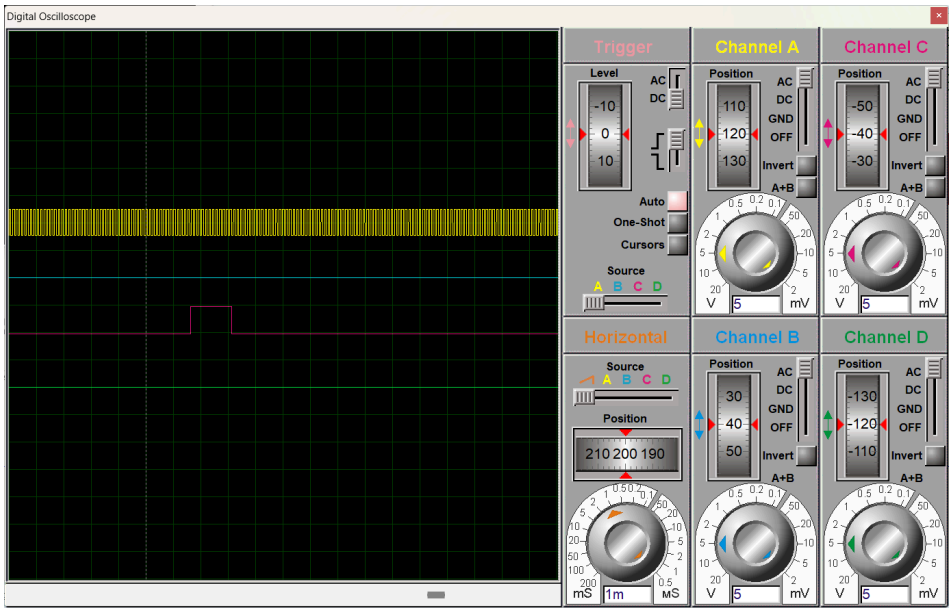


Figure 2.3 Proteus oscilloscope (presentation capture): channel A = dimmer PWM at 7.8 kHz, channel C = the servo pulse at 1 ms/div. measured in Proteus (repository capture)

API

```
void TIMER1_voidInit();  void TIMER1_voidSetOCRA(u16);  void TIMER1_voidSetOCRB(u16);
void TIMER1_voidGeneratePWM_A(u8 Mode);  void TIMER1_voidGeneratePWM_B(u8 Mode);   /* DISCONNECTED / INVERTED
/ NONINVERTED */
```

PITFALLS

- The first pin plan put the dimmer on Timer1 too; one TOP for both would have made the dimmer a 50 Hz PWM, too slow to filter into a steady 0–5 V. The dimmer moved to Timer0 (D-17, conflict C-1).

- The Proteus servo model maps 1–2 ms to −90…+90° by default; set its Min/Max Angle to 0/180.

RE-IMPLEMENTATION CHECKLIST

1. Mode 14, prescaler 8.
2. ICR1 = F_CPU/(8·f) − 1, high byte first.
3. OCR = pulse_µs × ticks_per_ms / 1000.
4. COM = 10 on the channels in use; pins as outputs.
5. Make init idempotent, because two drivers share the timer.

2.6 TIMER2 — the 1 ms system tick MCAL N

PURPOSE

The time base of the whole firmware. Timer2 counts in CTC mode and raises a compare interrupt exactly every millisecond; `SCHED` (§4.1) owns it.

HARDWARE / REGISTERS

Bits	Value	Why
`TCCR2.CS22:20`	100	F_CPU / 64 = 250 kHz. Timer2's prescaler codes differ from Timer0's (it adds /32 and /128), hence its own `TIMER2_DIV_x` macros
`TCCR2.WGM21:20`	10	CTC: the counter clears to 0 on compare match with OCR2
`TCCR2.COM21:20`	00	OC2 pin not used
`OCR2`	249	F_CPU / (64 × 1000) − 1
`TIMSK.OCIE2`	1	compare-match interrupt (`__vector_4`); set by `SCHED_voidStart`

FORMULA (COMPUTED)

T = (OCR2 + 1) × 64 / F_CPU = 250 × 4 µs = **1000.000 µs**, so the error is 0 ppm: the only error left is the crystal's tolerance. At 16 MHz two prescalers give an exact millisecond with an 8-bit compare value: ÷64 (OCR2 = 249) and ÷128 (OCR2 = 124); ÷32 would need 499, which does not fit. The code uses ÷64. A compile-time check stops a build with an `F_CPU` that cannot give an exact tick:

`lib/Service/SCHED/SCHED_cfg.h` · lines 13–19

```
#define SCHED_TIMER_PRESCALER     64UL                /* TIMER2_DIV_64 */
#define SCHED_TIMER_CLOCK         TIMER2_DIV_64       /* must be the code of SCHED_TIMER_PRESCALER */
#define SCHED_TICK_HZ             1000UL
#define SCHED_OCR_VALUE           ((u8)((F_CPU / (SCHED_TIMER_PRESCALER * SCHED_TICK_HZ)) - 1))    /* 249
*/
#if (F_CPU % (SCHED_TIMER_PRESCALER * SCHED_TICK_HZ)) != 0
#error "F_CPU does not give an exact 1 ms tick with prescaler 64"
#endif
```

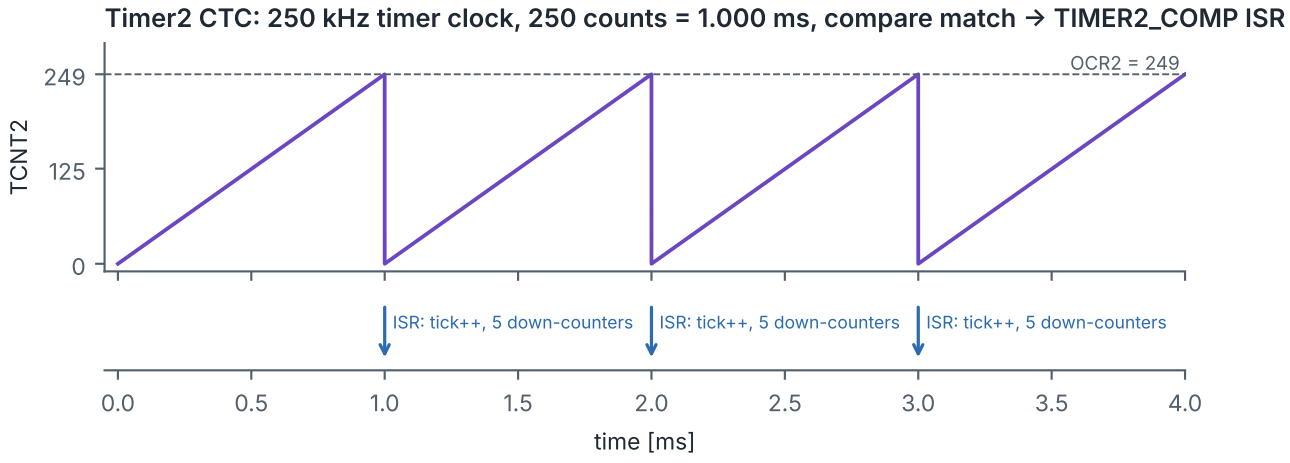


Figure 2.4 Timer2 in CTC mode: one compare match per millisecond. computed from the code and F_CPU

lib/MCAL/TIMER2/TIMER2.c · lines 47–51 · the vector only calls the callback

```
47 void __vector_4 () {
48     if (TIMER2_oc_ptr!=NULL){
49         TIMER2_oc_ptr();
50     }
51 }
```

API

```
void TIMER2_voidInit(u8 Prescaler, u8 Mode);   /* TIMER2_DIV_64 , TIMER2_CTC */
void TIMER2_voidSetOCR(u8);  void TIMER2_voidEnableOCInterrupt();  void TIMER2_voidDisableOCInterrupt();
void TIMER2_voidSetCallBack_OC(void (*ptr)(void));
```

RE-IMPLEMENTATION CHECKLIST

1. Pick N so that F_CPU/(N·1000) is an integer ≤ 256.
2. CTC, OCR = that integer − 1.
3. Callback pointer + `NULL` check in the ISR.
4. Enable OCIE only after everything the tick drives is initialised.
5. Keep the ISR body to counting (chapter 4).

2.7 USART — 9600 8N1, interrupt driven MCAL F

PURPOSE

The serial link to the remote terminal (RUI-01). The driver stays thin: it configures the hardware, exposes the RX-complete and data-register-empty interrupts through callbacks, and writes one byte without waiting. The ring buffers live one layer up in `TERM` (D-3).

HARDWARE / REGISTERS

Bits	Value	Why
`UBRRH : UBRRL`	0 : 103	baud divider from F_CPU (`USART_cfg.h`); UBRRH first
`UCSRA.U2X`	0	normal speed is accurate enough (+0.16 %)
`UCSRB.RXEN, TXEN`	1, 1	receiver and transmitter on
`UCSRB.RXCIE`	1	interrupt for every received byte (enabled by `TERM_voidInit`)
`UCSRB.UDRIE`	on demand	"data register empty" interrupt: on while the TX ring holds bytes, off when it is empty
`UCSRC`	0x86	URSEL = 1 (select UCSRC), UCSZ1:0 = 11 (8 bits), UPM = 00 (no parity), USBS = 0 (1 stop bit). Written once, never read-modify-write

FORMULAS (COMPUTED)

UBRR = F_CPU / (16 × baud) − 1 = 16 000 000 / 153 600 − 1 = 103.17 → **103** (integer division). Real baud = F_CPU / (16 × (UBRR + 1)) = **9 615.38**, error **+0.16 %**, far inside the ±2 % an 8N1 link tolerates. U2X would give UBRR = 207 with the same error, so it brings nothing. One byte is 10 bits = **1.040 ms**; the link carries at most 962 bytes per second in each direction.

lib/MCAL/USART/USART.c · lines 17–30

```
17 void USART_voidInit(){
18     /* Set Baud rate (USART_cfg.h , computed from F_CPU) */
19     UBRRH = (u8)(USART_UBRR_VALUE >> 8) ;
20     UBRRL = (u8)USART_UBRR_VALUE ;
21     /*enable Receiver and Transmitter */
22     SET_BIT(UCSRB,UCSRB_TXEN);
23     SET_BIT(UCSRB,UCSRB_RXEN);
25     /*set format : 8 bit , no parity , 1 stop bit
26      * UCSRC shares its address with UBRRH : one read returns UBRRH ,
27      * so SET_BIT/CLR_BIT (read-modify-write) would corrupt UBRRH.
28      * Write the whole register at once with URSEL = 1 (select UCSRC) */
29     UCSRC = (1<<UCSRC_URSEL) | (1<<UCSRC_UCSZ1) | (1<<UCSRC_UCSZ0) ;
30 }
```

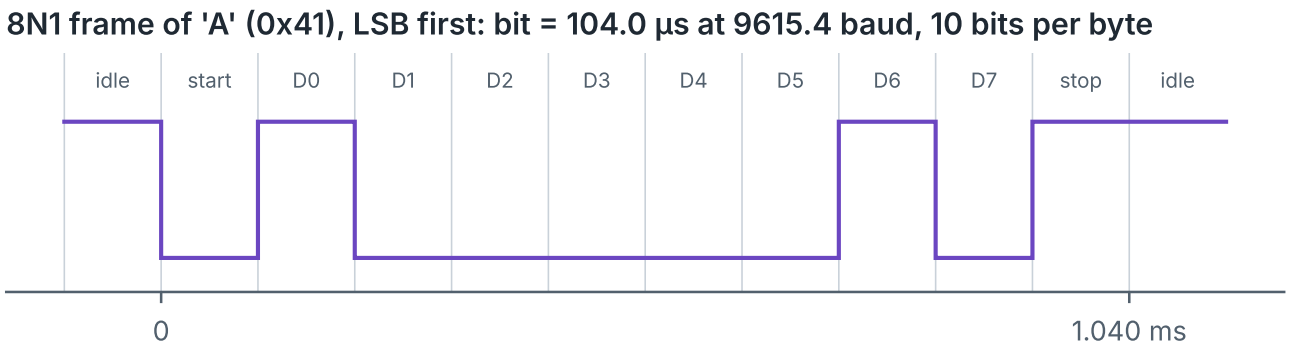


Figure 2.5 One 8N1 frame at 9615 baud. computed from the code and F_CPU

`lib/MCAL/USART/USART.c` · lines 75–89 · RX and UDRE vectors

```
75 void __vector_13 (){
76     /* always read UDR : it clears RXC , even when no callback is set */
77     u8 Local_u8data = UDR ;
78     if (USART_rx_ptr!=NULL){
79         USART_rx_ptr(Local_u8data);
80     }
81 }
82 void __vector_14 (){
83     if (USART_tx_ptr!=NULL){
84         USART_tx_ptr();
85     }else{
86         /* nobody to feed UDR : switch UDRIE off , or the interrupt fires forever */
87         CLR_BIT(UCSRB,UCSRB_UDRIE);
88     }
89 }
```

API

```
void USART_voidInit();
void USART_voidEnableRxInterrupt();  void USART_voidEnableTxInterrupt();  void USART_voidDisableTxInterrupt();
void USART_voidSetCallBack_RX(void (*ptr)(u8));   /* the ISR passes the received byte */
void USART_voidSetCallBack_TX(void (*ptr)(void)); /* called when UDR is empty */
void USART_voidWriteData(u8);                     /* writes UDR , no wait */
void USART_voidSend(u8); u8 USART_u8Recieve(); void USART_voidSendString(const c8*);  /* blocking , kept for
test_base */
```

PITFALLS (FIXED)

- **UBRRH and UCSRC share address 0x40.** A read of that address returns UBRRH on a real ATmega32, so `SET_BIT(UCSRC, …)` read UBRRH, set a bit and wrote it back as UCSRC without URSEL — corrupting the format. The register is now written once with URSEL = 1 (Phase 0). Proteus seems to return UCSRC on that read, so the bug did not show in simulation (specification §4, Proteus notes).
- **Hard-coded baud value.** `UBRRL = 103` was right only at 16 MHz. It is now computed from `F_CPU`.
- **Blocking TX/RX.** Printing the 408-byte admin menu with `USART_voidSend` would block the loop for 408 × 1.04 ms ≈ 0.42 s. The interrupt API and TERM's rings remove all waiting.
- The UDRE vector switches UDRIE off when no callback is set; otherwise the interrupt would fire for ever and lock the CPU.

RE-IMPLEMENTATION CHECKLIST

1. UBRR from F_CPU; check the error (< 2 %).
2. Write UBRRH, UBRRL, then UCSRC once with URSEL.
3. RX ISR: always read UDR (it clears RXC), then pass the byte on.
4. TX: enable UDRIE after putting a byte in the ring; in the ISR send one byte or disable UDRIE.
5. Keep the blocking functions only for test code.

2.8 TWI — the I²C master MCAL F

PURPOSE

The bus to the LCD, the lamps, both display digits and the EEPROM. The driver was restyled to the course conventions and extended with a timeout on every wait, error codes on every function, and an address probe used for 24C08 ACK polling. It stays polled, not interrupt-driven (D-4): every transfer is short and bounded (longest regular one: an EEPROM page, ≈ 1.7 ms).

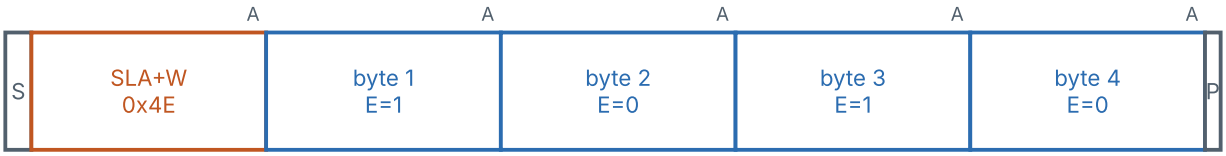
HARDWARE / REGISTERS

Register / bits	Value	Why
TWSR.TWPS1:0	00	prescaler 1
TWBR	72	SCL = F_CPU / (16 + 2·TWBR·4^{TWPS}) = 16 MHz / 160 = 100 kHz
TWCR.TWEN	1	TWI on; the TWI takes over PC0/PC1
TWCR.TWINT	write 1 to start an action	flag "action finished"; writing a one clears it and starts the next step
TWCR.TWSTA / TWSTO	1 for START / STOP	generate a (repeated) START or a STOP
TWCR.TWEA	1 / 0	when receiving: ACK (more bytes wanted) / NACK (last byte)
TWDR	address or data	byte to send / byte received
TWSR & 0xF8	status	0x08 START, 0x10 rep. START, 0x18/0x20 SLA+W ACK/NACK, 0x28/0x30 data ACK/NACK, 0x40/0x48 SLA+R, 0x50/0x58 data received ACK/NACK, 0x38 arbitration lost

TIMING (COMPUTED)

One SCL period is 10 µs; one byte on the wire, including its ACK bit, takes 9 periods = **90 µs**. A transaction of *n* bytes (address included) takes about 10 µs + n × 90 µs. Transaction-by-transaction numbers are in §7.3.

I²C transaction anatomy at 100 kHz (10 µs per SCL period, 90 µs per byte incl. ACK)



one LCD byte (CLCD, I²C mode) = START + 5 × 9 SCL periods
= 460 µs computed · 473 µs simavr · ≈ 480 µs measured in Proteus (hal_summary.md)

Figure 2.6 Anatomy of the most frequent transaction, one LCD byte. computed from the code and F_CPU

lib/MCAL/TWI/TWI.c · lines 19–33 · TWI_u8SendCommand: every wait has a timeout

```
19      TWCR = (1<<TWCR_TWINT) | (1<<TWCR_TWEN) | Copy_u8Flags ;
20      /* wait on TWINT (about 90 us per byte at 100 kHz) */
21      while(GET_BIT(TWCR,TWCR_TWINT) == 0){
22          if(Local_u16Timeout == 0){
23              /*1. release the bus */
24              TWCR = (1<<TWCR_TWINT) | (1<<TWCR_TWEN) | (1<<TWCR_TWSTO);
25              /*2. switch the unit off and on : clears any half-finished transfer */
26              TWCR = 0 ;
27              TWCR = (1<<TWCR_TWEN);
28              Local_u8Error = TWI_ERR_TIMEOUT ;
29              break ;
30          }
31          Local_u16Timeout-- ;
32      }
33      return Local_u8Error ;
```

The timeout counts loop passes, not time. `TWI_cfg.h` assumes 6 cycles per pass, giving 2666 passes ≈ 1 ms. The compiled loop (`in`, `sbrs`, `rjmp`, `sbiw`, `brne`) takes **8 cycles**, so the real timeout is 2666 × 8 / 16 MHz = **1.33 ms** (computed from the disassembly of the avr-gcc 7.3.0 build). It is still far longer than a normal byte (90 µs ≈ 180 passes) and inside the 2 ms that `test_mcal` checks.

`lib/MCAL/TWI/TWI.c` · lines 160–167 · START + SLA+W + STOP

```
160 u8 TWI_u8ProbeAddress(u8 Copy_u8SlaveAddress){
161     u8 Local_u8Error = TWI_u8SendStartCondition();
162     if(Local_u8Error == TWI_OK){
163         Local_u8Error = TWI_u8SendSlaveAddressWrite(Copy_u8SlaveAddress);
164     }
165     TWI_voidSendStopCondition();
166     return Local_u8Error ;
167 }
```

API

```
void TWI_voidMasterInit();  void TWI_voidDisable();  u8 TWI_u8GetStatus();
u8 TWI_u8SendStartCondition();  u8 TWI_u8SendRepeatedStart();  void TWI_voidSendStopCondition();
u8 TWI_u8SendSlaveAddressWrite(u8);  u8 TWI_u8SendSlaveAddressRead(u8);
u8 TWI_u8MasterWriteDataByte(u8);  u8 TWI_u8MasterReadDataByteAck(u8*);  u8 TWI_u8MasterReadDataByteNack(u8*);
u8 TWI_u8MasterTransmit(u8 addr, const u8 *data, u8 n);        /* START , SLA+W , n bytes , STOP */
u8 TWI_u8MasterReceive (u8 addr, u8 *data, u8 n);
u8 TWI_u8MasterWriteRead(u8 addr, const u8 *w, u8 nw, u8 *r, u8 nr);   /* write , repeated START , read */
u8 TWI_u8ProbeAddress(u8 addr);     /* TWI_OK = ACK , TWI_ERR_SLA_NACK = absent or busy */
/* return codes: TWI_OK 0, ERR_START 1, ERR_SLA_NACK 2, ERR_DATA_NACK 3, ERR_ARB_LOST 4, ERR_BUS 5,
ERR_TIMEOUT 6 */
```

PITFALLS

- **A stuck bus hung the loop.** The course driver waited for TWINT for ever. With a slave holding SDA low, every loop pass stopped. Each wait now ends after ≈ 1.33 ms, the bus is released (STOP, then TWEN off/on), and `TWI_ERR_TIMEOUT` goes up the call chain. LCD_BUF, LAMP, SEVEN_SEG and ESTORE each react to it (chapter 3).
- A STOP does not set TWINT, so `TWI_voidSendStopCondition` does not wait. The hardware finishes the STOP before it starts the next START.
- Bus timing depends on the pull-ups: 4.7 kΩ at 100 kHz is right for the short Proteus bus.

RE-IMPLEMENTATION CHECKLIST

1. TWBR from F_CPU and the SCL frequency.
2. One `SendCommand(flags)` = write TWCR, wait TWINT with a timeout.
3. Check TWSR after every step and return a code.
4. Build transmit, receive and write-read on top; always end with STOP.
5. Add a probe (START + SLA+W + STOP) for presence checks and EEPROM ACK polling.

2.9 EXTI — external interrupts MCAL F

INT0 (PD2), INT1 (PD3) and INT2 (PB2) with selectable sense (MCUCR.ISC0x/ISC1x: low level, any change, falling, rising; MCUCSR.ISC2 for INT2) and enable bits in GICR. The application does not use EXTI in v1; PD2 stays reserved for the PIR sensor (ALM-02, not built). `test_base` and `test_mcal` exercise it with a push button on PD2.

Fixed in Phase 2: the three ISRs got `used, externally_visible`; the callback setters `EXTI_voidINT0/1/2_callBack` are now declared in `EXTI.h`; and each ISR checks the callback for `NULL` before calling it. Before the fix, an interrupt with no callback set jumped to address 0.

`lib/MCAL/EXTI/EXTI.c` · lines 87–92

```
 87 void __vector_1 (){
 88 if(INT0_p != NULL){
 89     INT0_p();
 90 }
 92 }
```

Also present but unused: `lib/MCAL/SPI` (course stub; its missing `return` was fixed in Phase 0). No SPI device is fitted.

CHAPTER 3

HAL — one driver per part

A HAL module hides one part, or one kind of part, behind functions that speak in the part's own terms: a lamp number, a temperature, a key, an angle. Above the HAL nobody knows a pin, an I²C address or a register. Chips that sit behind other chips form small HAL chains: `LCD_BUF → CLCD → PCF8574 → TWI`.

Module	Part	Uses	Called from	Blocking time per call (computed)
PCF8574	8-bit I²C expander	TWI	CLCD, LAMP, SEVEN_SEG	190 µs per one-byte write
CLCD	HD44780 LCD behind a PCF8574	PCF8574	LCD_BUF	460 µs per byte; init 50.5 ms (boot)
LCD_BUF	non-blocking LCD screen	CLCD	UILOC (writes), main loop (5 ms)	≤ one LCD byte per call
KPAD	4×4 calculator keypad	DIO	UILOC, main loop (10 ms)	≈ 25 µs per scan
BUTTON	3 push buttons	DIO	HEATER, main loop (10 ms)	a few µs
LM35	2 temperature sensors	ADC	HEATER, CLIMATE (100 ms)	104 µs per reading
SEVEN_SEG	2 common-anode digits, one PCF8574 each	PCF8574	HEATER	0, 190 or 380 µs (only changed digits)
LAMP	5 LEDs on a PCF8574	PCF8574	LIGHT	190 µs
RELAY, LED, BUZZER	GPIO outputs	DIO	HEATER, ALARM	< 2 µs
DIMMER, SERVO, FAN	PWM outputs	TIMER0 / TIMER1, DIO	LIGHT, DOOR, CLIMATE	< 10 µs (register writes)
EXT_EEPROM	24C08 EEPROM	TWI	ESTORE	page write 1.63 ms, poll 100 µs

3.1 PCF8574 — I²C port expander HAL F

PURPOSE

The PCF8574 turns one I²C byte into eight pin levels. Four of them on one bus give the LCD (0x27), the five lamps (0x20) and the two display digits (0x21, 0x22), which is how the design fits into 32 pins (§1.3).

HARDWARE

The address is `0100 A2 A1 A0`, set by three pins. The pins are *quasi-bidirectional*: writing 0 turns on a strong pull-down (sinks up to 25 mA), writing 1 leaves only a weak pull-up (≈ 100 µA). So a PCF8574 drives LEDs **active low**, and a pin used as an input must first be written 1. At power-up all pins are 1.

DESIGN AND API

```
void PCF8574_voidInit();                                        /* TWI_voidMasterInit ; a second call is
harmless */
u8   PCF8574_u8WritePort (u8 Address, u8 Value);                /* START , addr+W , value , STOP */
u8   PCF8574_u8ReadPort  (u8 Address, u8 *Value);
u8   PCF8574_u8WriteBytes(u8 Address, const u8 *Data, u8 Length); /* new: several port values in ONE
transaction */
```

Every function returns the TWI status, so the caller can see a missing chip. `PCF8574_u8WriteBytes` is the addition made for CLCD: the chip latches every byte as it arrives, so four bytes in one transaction produce four successive pin states, 90 µs apart.

PITFALL

Reading back a PCF8574 whose pins drive LEDs returns the pin *levels*, not the written latch. With segment LEDs pulling the pins, the value read in Proteus differed from the one written while the display was right. The 7-segment read-back checks were removed from `test_hal` (specification §4).

RE-IMPLEMENTATION CHECKLIST

1. Address 0x20 + A2A1A0.
2. Write = one-byte TWI transmit.
3. Multi-byte write for timed sequences.
4. Return the TWI status.
5. Drive loads active low.

3.2 CLCD — character LCD over I²C HAL F

PURPOSE

Driver for the 16×2 HD44780-compatible LCD (LM016L), in the course's I²C mode (`CLCD_MODE = CLCD_I2C_MODE`). It is blocking, with about 0.46 ms per byte. Only `LCD_BUF` calls it after boot.

HARDWARE

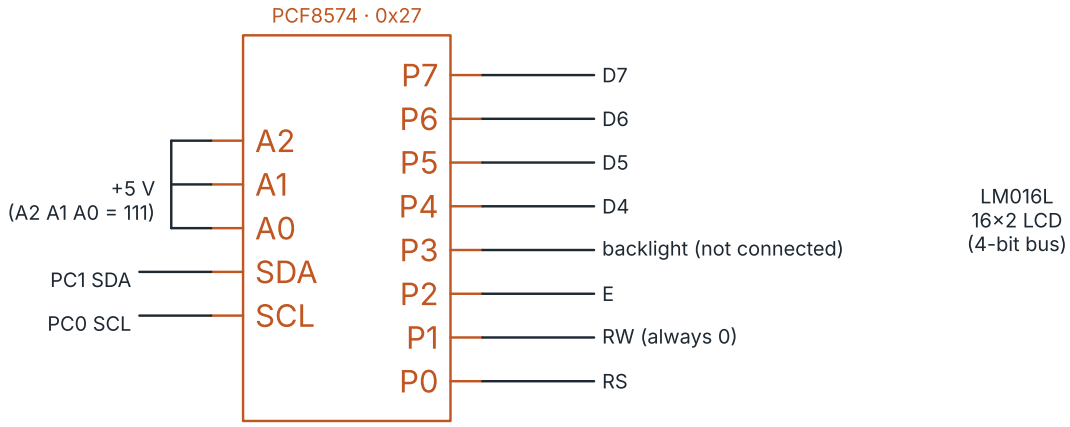


Figure 3.1 Wiring of the LCD expander (`CLCD_cfg.h`, pin_map.md §2). drawn from the source code

PCF8574 0x27 port bits → LCD pins (CLCD_cfg.h); BL is always written 1, RW always 0

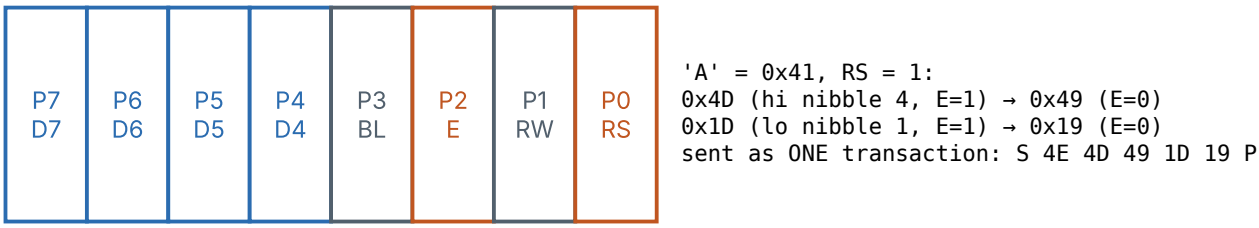


Figure 3.2 One LCD byte becomes four port values: the high nibble with E = 1, then E = 0, then the low nibble the same way. The LCD latches data on the falling edge of E. drawn from the source code

DESIGN

In 4-bit mode the LCD takes every byte as two nibbles on D4–D7, each latched by a falling edge of E. The course driver sent each nibble edge as its own I²C transaction: 4 transactions × 3 bytes ≈ 1.2 ms per character. The fixed driver builds all four port values and sends them in one transaction of 5 bytes, 460 µs computed, ≈ 0.48 ms measured in Proteus (hal_summary.md). The E pulse is one I²C byte long (90 µs), far above the 450 ns the LCD needs, so no delay is added.

`lib/HAL/CLCD/CLCD.c` · lines 44–53

```
44 static void I2C_voidSendByte(u8 Copy_u8Byte, u8 Copy_u8RS){
45     u8 Local_u8High = I2C_u8BuildPort(Copy_u8Byte>>4,Copy_u8RS);
46     u8 Local_u8Low  = I2C_u8BuildPort(Copy_u8Byte,Copy_u8RS);
47     u8 Local_au8Data[4] ;
48     Local_au8Data[0] = Local_u8High | (1<<CLCD_I2C_E_BIT) ;
49     Local_au8Data[1] = Local_u8High ;
50     Local_au8Data[2] = Local_u8Low  | (1<<CLCD_I2C_E_BIT) ;
51     Local_au8Data[3] = Local_u8Low ;
52     Global_u8LastStatus = PCF8574_u8WriteBytes(CLCD_I2C_ADDRESS,Local_au8Data,4);
53 }
```

Init sequence (`CLCD_voidInit`, boot only): wait 40 ms after power-up; send nibble 0x3 three times (5 ms and 1 ms apart) to force 8-bit mode from any state; send 0x2 to switch to 4-bit mode; Function Set `0x28` (4-bit, 2 lines, 5×8); Display Control `CLCD_DISPLAY_CTRL = 0x0C` (display on, cursor off, blink off); Clear `0x01` followed by 2 ms (Clear and Home need 1.52 ms). Total 50.5 ms computed, 50.6 ms in simavr.

Cursor: `CLCD_voidSetCursorPosition(x, y)` sends `0x80 | (x + 0x40·y)`: row 0 starts at DDRAM 0x00, row 1 at 0x40.

API

```
void CLCD_voidInit();  void CLCD_voidSendCommand(u8);  void CLCD_voidSendData(u8);
void CLCD_voidSetCursorPosition(u8 x, u8 y);  void CLCD_voidSendString(char*);  void CLCD_voidClearDisp();
void CLCD_voidSendFlashString(const __flash c8 *);   /* new */
u8   CLCD_u8GetStatus();                             /* new: TWI status of the last write */
void CLCD_voidCreatSpecialChar(u8 Index, u8 *Pattern);
```

PITFALLS (FIXED IN PHASE 2)

- Cursor and blink were switched on at init; on a status screen that shows a blinking block. Now set from `CLCD_DISPLAY_CTRL`.
- `CLCD_voidClearDisp` existed but was not declared in the header.
- Writes could fail silently. `CLCD_u8GetStatus()` now reports the last TWI result, so LCD_BUF can retry.
- The parallel modes (unused) still have a 10 ms enable pulse; they would block the loop if ever selected.

RE-IMPLEMENTATION CHECKLIST

1. Map PCF8574 bits to RS, RW, E, BL, D4–D7.
2. Byte = 4 port values (nibble with E high, then low) in one transaction.
3. Init: 40 ms, 0x3 ×3, 0x2, 0x28, 0x0C, 0x01 + 2 ms.
4. Cursor = 0x80 | address (row 1 at 0x40).
5. Keep the last TWI status.

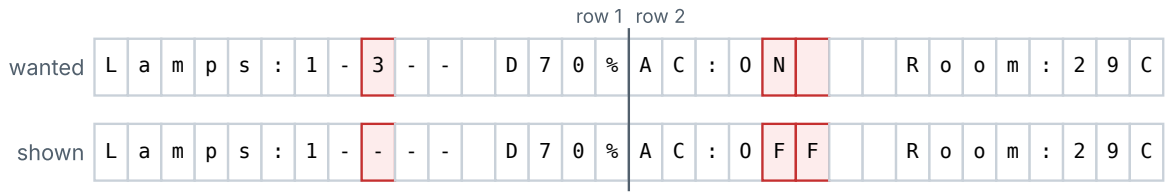
3.3 LCD_BUF — the non-blocking LCD HAL N

PURPOSE

Writing a 16-character line with CLCD blocks for 16 × 0.46 ms ≈ 7.5 ms, and longer with set-cursor commands. LCD_BUF keeps the loop free: writers only change RAM, and a 5 ms task copies the differences to the LCD, one byte per call (LUI-02, LUI-03).

DESIGN

Two 32-byte arrays: `Global_c8Wanted` (what the application wants) and `Global_c8Shown` (what the LCD shows), plus the LCD's cursor position as LCD_BUF believes it to be. `LCD_BUF_voidUpdate` searches round-robin from the last position for a cell where the two differ. If the cursor is not on that cell, the call sends one set-cursor command. If it is, the call sends the character, marks the cell shown and advances the cursor, because the LCD auto-increments. At the end of a row the cursor is marked unknown, because the next DDRAM address (0x10) is not on screen.



each LCD_BUF_voidUpdate (5 ms) finds the next differing cell (round robin) and sends ONE thing: a set-cursor command if the cursor is elsewhere, else the character. Here: cursor→8, "3", cursor→20, "N", " " ... 3 differing cells + 2 cursor moves = 5 calls = 25 ms.

Figure 3.3 The wanted/shown comparison. Only differing cells cost bus time. drawn from the source code

lib/HAL/LCD_BUF/LCD_BUF.c · lines 91–105 · end of LCD_BUF_voidUpdate

```
91      /*4. Cursor is there : this call sends the character */
92      CLCD_voidSendData(Global_c8Wanted[Local_u8Cell]);
93      if(CLCD_u8GetStatus() == 0){
94          Global_c8Shown[Local_u8Cell] = Global_c8Wanted[Local_u8Cell] ;
95          /* the LCD moves its cursor right ; at the end of a row it leaves the visible area */
96          if(((Local_u8Cell + 1) % LCD_BUF_COLS) == 0){
97              Global_u8Cursor = LCD_BUF_CURSOR_UNKNOWN ;
98          }else{
99              Global_u8Cursor = Local_u8Cell + 1 ;
100         }
101         Local_u8Cell++;
102         if(Local_u8Cell >= LCD_BUF_CELLS){
103             Local_u8Cell = 0 ;
104         }
105         Global_u8Scan = Local_u8Cell ;
```

Error handling: if CLCD reports an I²C error, the byte is not marked shown, the cursor becomes unknown, and the module stays quiet for `LCD_BUF_RETRY_UPDATES` = 200 calls = 1 s. An unplugged LCD therefore costs one failed transaction per second, not one per 5 ms.

TIMING (COMPUTED)

One character changed (cursor elsewhere)	2 calls → visible after 5–10 ms
Full screen redraw (32 cells + 2 cursor moves)	34 calls × 5 ms = 170 ms
Longest call	one transaction, 460 µs computed · 0.49 ms Proteus (hal_summary.md)

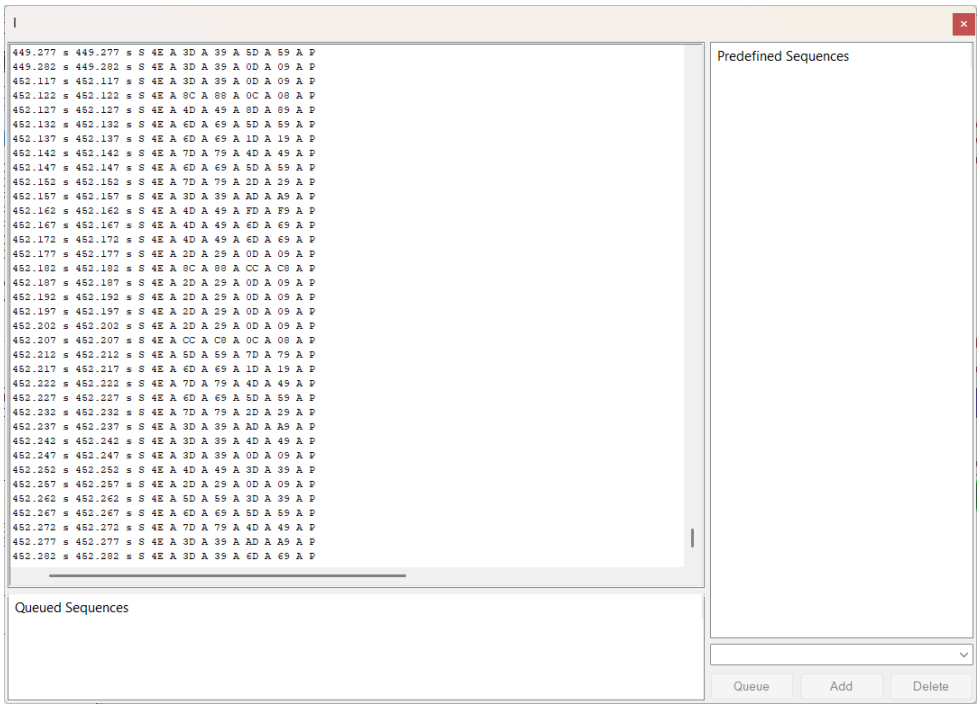


Figure 3.4 Proteus I²C debugger (presentation capture): one LCD transaction `S 4E xx xx xx xx P` every 5 ms (452.122 s, 452.127 s, …) — the LCD_BUF cadence.
measured in Proteus (repository capture)

API

```
void LCD_BUF_voidInit();     /* CLCD_voidInit (blocking , boot) + both buffers blank */
void LCD_BUF_voidClear();    /* RAM only */
void LCD_BUF_voidWriteChar (u8 Row, u8 Col, c8 Char);
void LCD_BUF_voidWriteFlash(u8 Row, u8 Col, const __flash c8 *Text);   /* cut at the row end */
void LCD_BUF_voidWriteRam  (u8 Row, u8 Col, const c8 *Text);
void LCD_BUF_voidUpdate();   /* every 5 ms : at most ONE byte to the LCD */
```

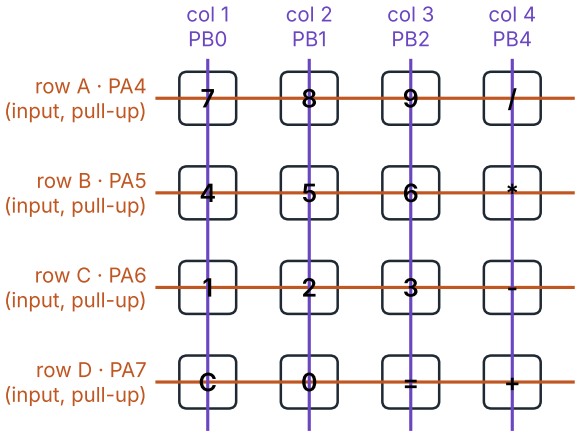
RE-IMPLEMENTATION CHECKLIST

1. Two cell arrays, wanted and shown.
2. Writers touch only wanted.
3. Update: find the next difference; set the cursor or send one char.
4. Track the auto-incremented cursor; forget it at row ends and on errors.
5. Back off after an error.

3.4 KPAD — 4×4 keypad HAL F

PURPOSE

Reads the calculator keypad (`7 8 9 /  · 4 5 6 *  · 1 2 3 -  · C 0 = +`) for UILOC (LUI-02). It adds a non-blocking, debounced scan next to the course's blocking function.



scan (KPAD_voidUpdate, every 10 ms): for each column: drive LOW, wait 5 µs, read the 4 rows; a LOW row = key at (row, column); drive the column HIGH. Accepted after 2 equal scans (20 ms); reported once (KPAD_u8GetKey).

Figure 3.5 Keypad matrix and scan (`KPAD_cfg.h`, `KPAD_MAT`). drawn from the source code

DESIGN

`KPAD_voidUpdate` runs every 10 ms. It drives each column low in turn, waits 5 µs for the row lines to settle through the pull-ups, reads the four rows, and drives the column high again; the first low row gives the key. A key becomes the stable value after `KPAD_DEBOUNCE_SCANS` = 2 equal scans. A new stable key is stored once in `Global_u8Pending`; `KPAD_u8GetKey()` returns it once and then `KPAD_NO_KEY` (0xFF), so a held key gives exactly one event. The key table and the pin arrays are `static const __flash`: plain `const` data is still copied to RAM on the AVR, and `__flash` keeps the 24 bytes in program memory.

`lib/HAL/KPAD/KPAD.c` · lines 81–95 · KPAD_voidUpdate

```
81      u8 Local_u8Raw = KPAD_u8ScanMatrix();
82      /*1. Count how many scans in a row gave the same result */
83      if(Local_u8Raw == Global_u8Candidate){
84          if(Global_u8SameScans < KPAD_DEBOUNCE_SCANS){
85              Global_u8SameScans++;
86          }
87      }else{
88          Global_u8Candidate = Local_u8Raw ;
89          Global_u8SameScans = 1 ;
90      }
91      /*2. Accept a new level after KPAD_DEBOUNCE_SCANS equal scans */
92      if((Global_u8SameScans >= KPAD_DEBOUNCE_SCANS) && (Global_u8Candidate != Global_u8Stable)){
93          Global_u8Stable = Global_u8Candidate ;
94          if(Global_u8Stable != KPAD_NO_KEY){
95              /* new key accepted : report it once (a held key gives no second event) */
```

TIMING

A clean press is accepted on the 2nd scan that sees it, 10–20 ms after the contact closes (computed). UILOC reads the key in the same 10 ms slot, right after `KPAD_voidUpdate`. simavr measured 14.6 ms from key-down to the lamp write.

PITFALLS (FIXED)

- The course table was transposed (row/column swapped), and one column was left low after a key was found (Phase 0).
- `KPAD_u8GetKeyPressed` waits until the key is released, which blocks the loop for as long as the key is held. It is kept only for `test_base`.

RE-IMPLEMENTATION CHECKLIST

1. Rows = inputs with pull-ups, columns = outputs, idle high.
2. Scan: column low, short settle, read rows, column high.
3. Debounce: N equal scans.
4. Report a new key once.
5. Keep the tables in flash.

3.5 BUTTON — heater push buttons HAL N

PURPOSE

Debounced level and press/release events for ON/OFF (PD6), Up (PD7) and Down (PB5) (HTR-01, HTR-02, HTR-05).

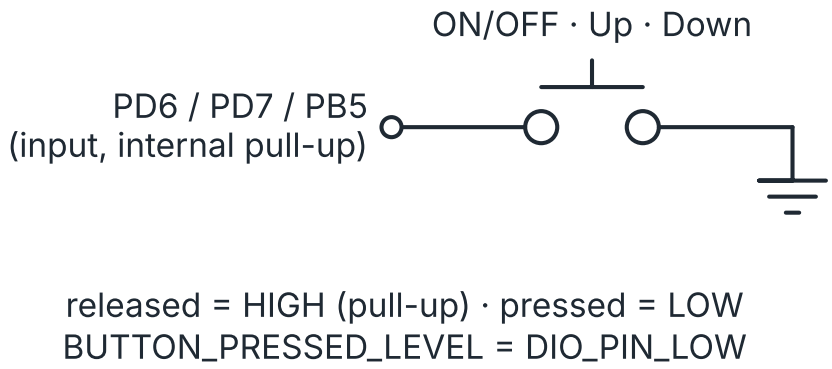


Figure 3.6 Each button connects its pin to ground; the internal pull-up holds the pin high when released. drawn from the source code

DESIGN

Every 10 ms `BUTTON_voidUpdate` reads the three pins. If a pin differs from the debounced level, a counter runs; after `BUTTON_DEBOUNCE_SAMPLES` = 3 differing samples in a row the level changes and a press or release flag is set. A sample equal to the old level restarts the count, so a bounce is ignored. The event flags are cleared by reading them.

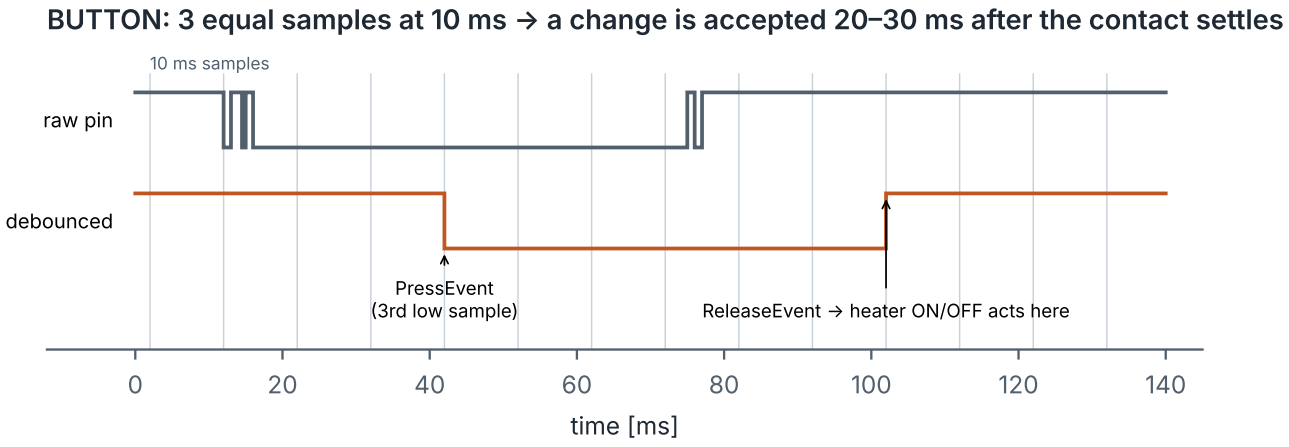


Figure 3.7 Debounce timing. computed from the code and F_CPU

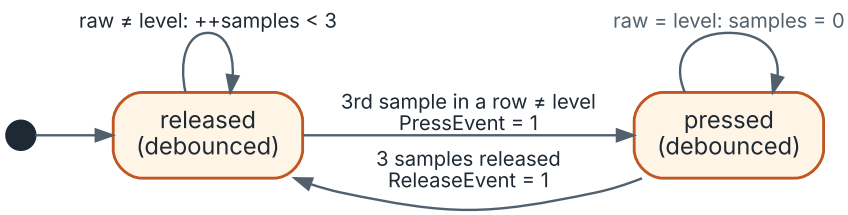


Figure 3.8 BUTTON debounce as a state machine (per button). drawn from the source code

HEATER acts on the **release** of ON/OFF (HTR-05), so holding the button does nothing, and on the **press** of Up and Down.

API

```
void BUTTON_voidInit();  void BUTTON_voidUpdate();          /* every 10 ms */
u8 BUTTON_u8IsPressed(u8 ID);  u8 BUTTON_u8GetPressEvent(u8 ID);  u8 BUTTON_u8GetReleaseEvent(u8 ID);
/* IDs: BUTTON_ONOFF 0 , BUTTON_UP 1 , BUTTON_DOWN 2 */
```

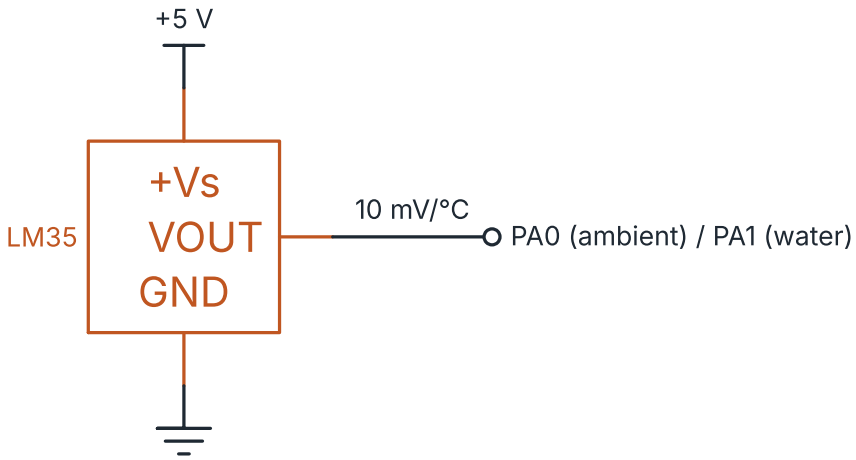
RE-IMPLEMENTATION CHECKLIST

1. Inputs with pull-ups; pressed = low.
2. Count consecutive differing samples.
3. Change the level after N samples and raise an event.
4. Reset the count on a bounce.
5. Clear events on read.

3.6 LM35 — temperature sensors HAL N

PURPOSE

Two LM35s: ambient air for the AC (PA0, AC-01) and water for the heater (PA1, HTR-07).



ADC reference = internal 2.56 V → AREF pin left open (C-5)

Figure 3.9 LM35 wiring. drawn from the source code

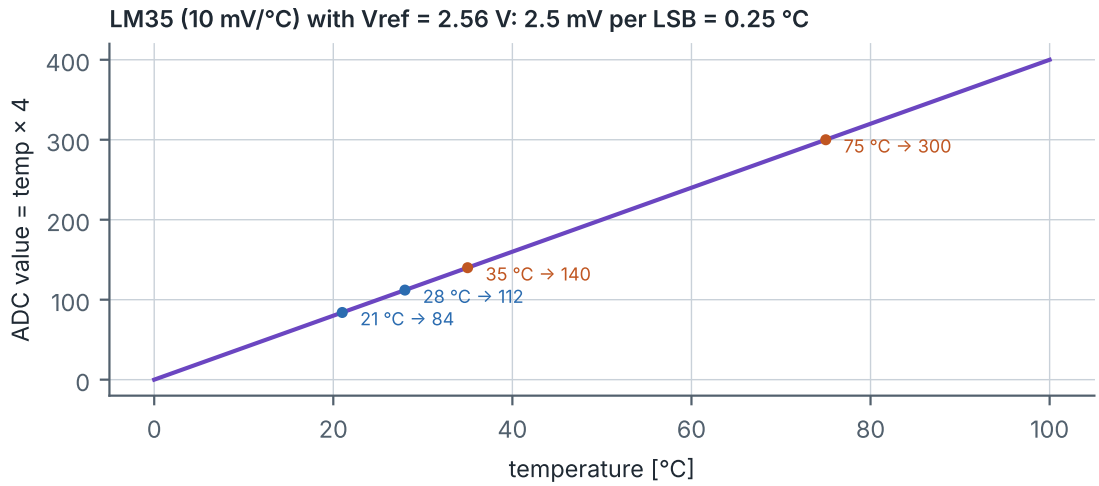


Figure 3.10 ADC value against temperature. The thresholds used by CLIMATE and HEATER are marked. computed from the code and F_CPU

DESIGN

With the 2.56 V reference, one ADC step is 2.5 mV = 0.25 °C, so **the raw ADC value is the temperature in quarter degrees**. `LM35_u16ReadTempX4` returns it unchanged; CLIMATE and HEATER keep everything in this unit and round to whole degrees only for display: `(x4 + 2) / 4`. An unknown sensor ID returns 0. The range ends at 1023 / 4 = 255.75 °C, well above the 0–100 °C an LM35 delivers here.

`lib/HAL/LM35/LM35.c` · lines 15–24

```
15 u16 LM35_u16ReadTempX4(u8 Copy_u8SensorID){
16     u16 Local_u16Value = 0 ;
17     switch(Copy_u8SensorID){
18     case LM35_AMBIENT: Local_u16Value = ADC_u16StartConversion(LM35_AMBIENT_CHANNEL); break ;
19     case LM35_WATER  : Local_u16Value = ADC_u16StartConversion(LM35_WATER_CHANNEL);   break ;
20     default: /* error */ break ;
21     }
22     /* 2.56 V / 1024 = 2.5 mV per step , LM35 = 10 mV per C --> value = temperature x LM35_STEPS_PER_C (=
4) */
23     return Local_u16Value ;
24 }
```

RE-IMPLEMENTATION CHECKLIST

1. Pick Vref so that 1 LSB is a round fraction of a degree.
2. Return raw counts.
3. Compare thresholds in counts (× 4).
4. Round only for display.
5. Leave AREF open with the internal reference.

3.7 SEVEN_SEG — two-digit display HAL N

PURPOSE

The heater's display: water temperature, or the blinking set temperature (HTR-10, HTR-11). Each common-anode digit has its own PCF8574 (tens 0x21, units 0x22), with no multiplexing (D-20). The history of this design is in §8.5.

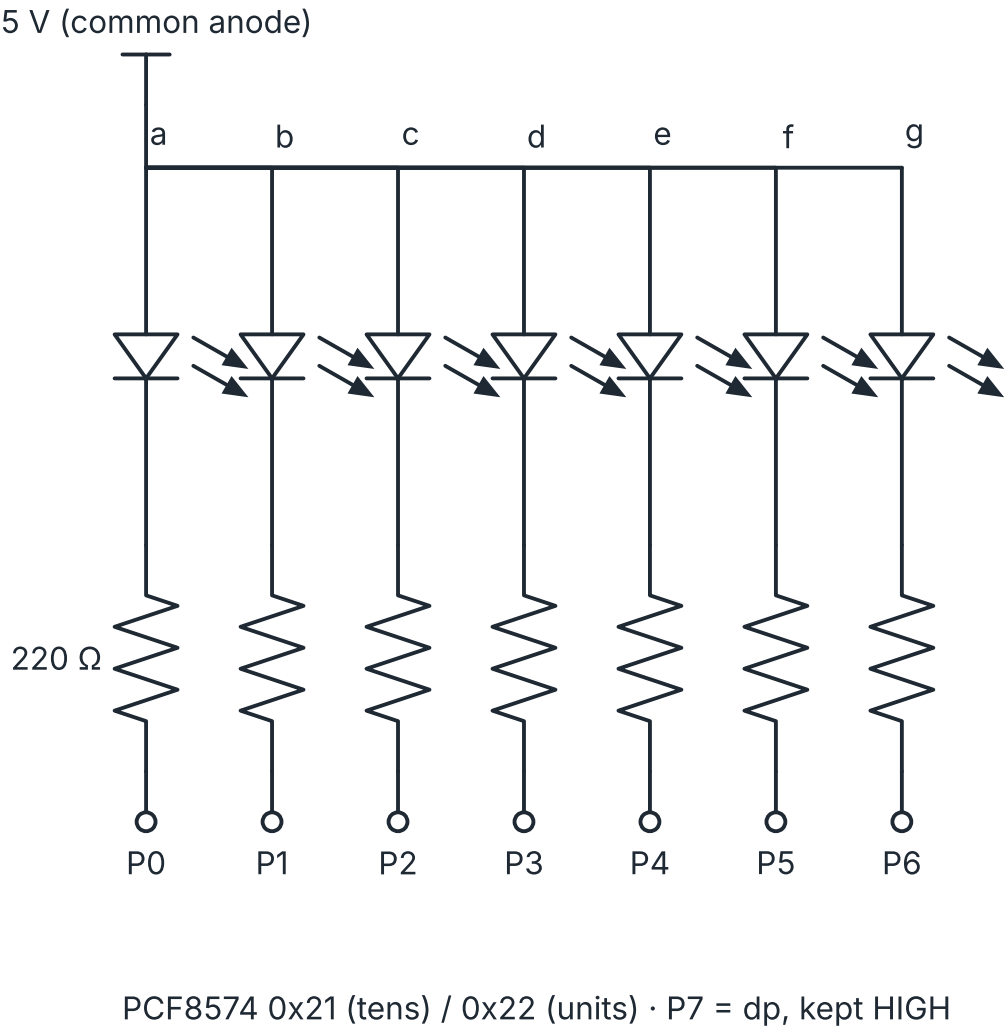
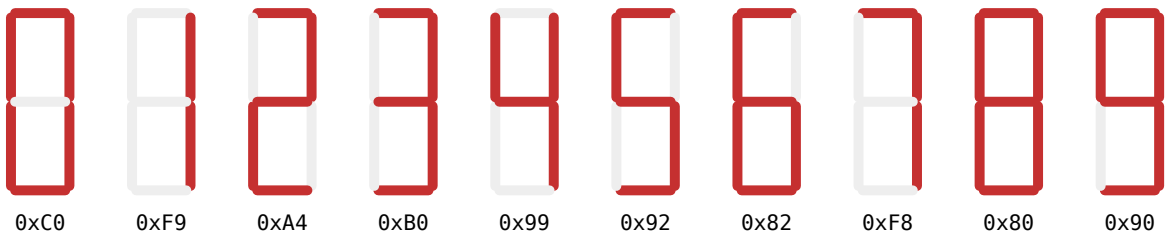


Figure 3.11 One digit: common anode on +5 V; each segment is lit when its PCF8574 pin is low. drawn from the source code



SEVEN_SEG_TABLE: bit 0 = a … bit 6 = g, bit 7 = dp; a 0 bit lights the segment (common anode, PCF8574 sinks). 0xFF = blank.

Figure 3.12 Segment patterns of `SEVEN_SEG_TABLE`. drawn from the source code

DESIGN

The module remembers what each chip shows (`Shown`) and whether that is known (`Known`). `SetNumber`, `Enable` and `Disable` compute both patterns and write a chip **only if its pattern differs** from what it shows. Changing 55 to 56 costs one 190 µs write, not two; repeating a number costs nothing. A failed write marks that digit unknown, so the next call writes it again. Numbers above 99 show 99; a number below 10 shows a leading zero.

lib/HAL/SEVEN_SEG/SEVEN_SEG.c · lines 50–63

```
 50 static u8 SEVEN_SEG_u8WriteDigit(u8 Copy_u8Address, u8 Copy_u8Pattern, u8 * Copy_pu8Shown, u8 *
Copy_pu8Known){
 51     u8 Local_u8Status = TWI_OK ;
 52     if((*Copy_pu8Known == 0) || (*Copy_pu8Shown != Copy_u8Pattern)){
 53         Local_u8Status = PCF8574_u8WritePort(Copy_u8Address,Copy_u8Pattern);
 54         if(Local_u8Status == TWI_OK){
 55             *Copy_pu8Shown = Copy_u8Pattern ;
 56             *Copy_pu8Known = 1 ;
 57         }else{
 58             /* the chip may have missed it : try again at the next call */
 59             *Copy_pu8Known = 0 ;
 60         }
 61     }
 62     return Local_u8Status ;
 63 }
```

API

```
void SEVEN_SEG_voidInit();                 /* blank */
void SEVEN_SEG_voidSetNumber(u8 Number);    /* 0..99 */
void SEVEN_SEG_voidEnable();  void SEVEN_SEG_voidDisable();   /* show / blank (0xFF on both chips) */
u8   SEVEN_SEG_u8GetStatus();              /* TWI status of the last write */
```

RE-IMPLEMENTATION CHECKLIST

1. Pattern table, a 0 bit = lit (common anode).
2. One expander per digit: no refresh needed.
3. Write only changed digits.
4. Mark a digit unknown after a failed write.
5. Call from the main loop only (the bus is polled).

3.8 LAMP — five lamps on a PCF8574 HAL N

PURPOSE

On/off control of lamps 1–5 on P0–P4 of the expander at 0x20 (LGT-01).

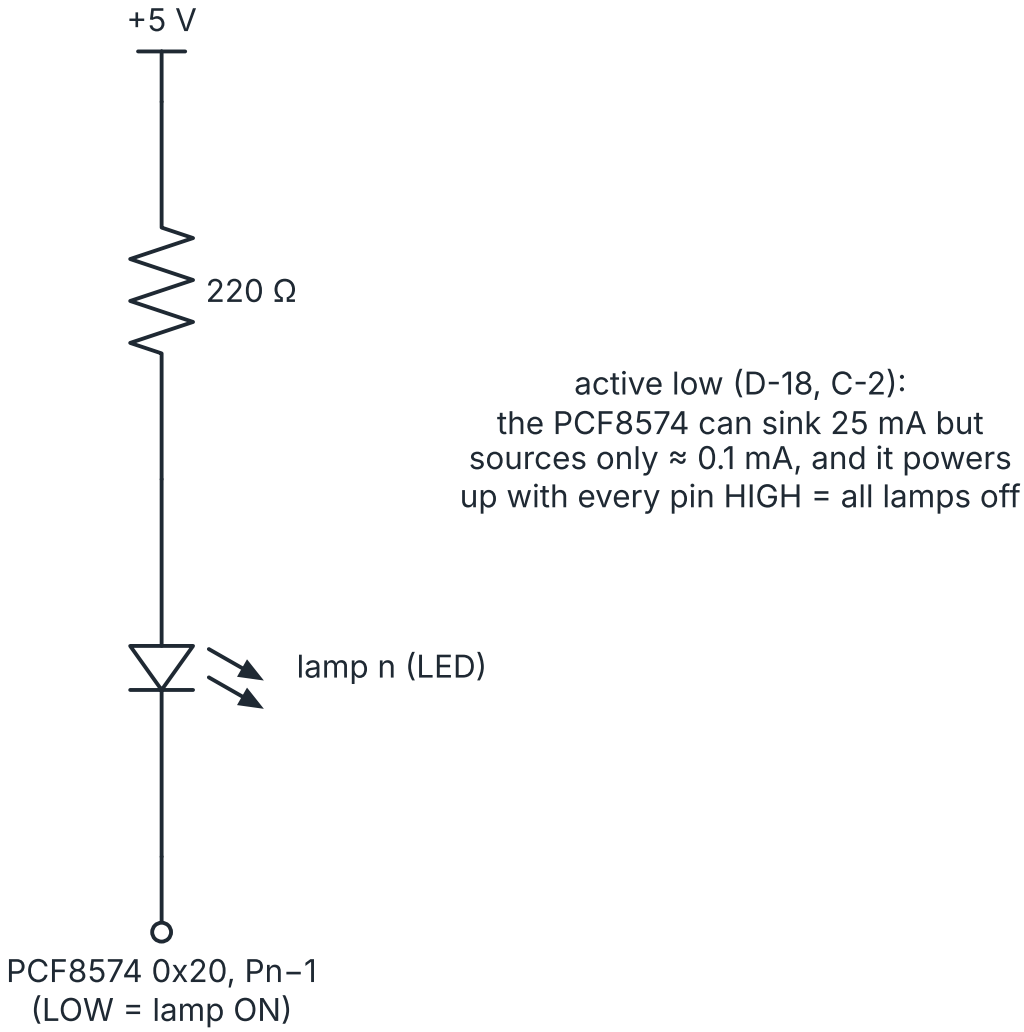


Figure 3.13 One lamp channel (Proteus uses an LED per lamp). drawn from the source code

DESIGN

The PCF8574 cannot be read back reliably as an output latch, so LAMP keeps a RAM copy of the port, `Global_u8Port`, starting at 0xFF (all off, unused pins high). To switch a lamp it changes one bit in a *copy* of that byte, writes the whole byte, and stores it as the new state **only if the chip acknowledged**. After an I²C error the copy still matches the chip. The level for "on" comes from `LAMP_ON_LEVEL = 0`.

`lib/HAL/LAMP/LAMP.c` · lines 36–48 · inside LAMP_u8SetState

```
36          u8 Local_u8Bit = LAMP_FIRST_BIT + (Copy_u8LampNumber - 1) ;
37          u8 Local_u8NewPort = Global_u8Port ;
38          /* pin level for "ON" is LAMP_ON_LEVEL , for "OFF" the opposite */
39          if((Copy_u8State == LAMP_ON) == (LAMP_ON_LEVEL == 1)){
40              SET_BIT(Local_u8NewPort,Local_u8Bit);
41          }else{
42              CLR_BIT(Local_u8NewPort,Local_u8Bit);
43          }
44          Local_u8Error = PCF8574_u8WritePort(LAMP_I2C_ADDRESS,Local_u8NewPort);
45          if(Local_u8Error == TWI_OK){
46              /* the copy changes only when the chip really got the value */
47              Global_u8Port = Local_u8NewPort ;
48          }
```

API

```
void LAMP_voidInit();                       /* all off */
u8   LAMP_u8SetState(u8 Lamp, u8 State);    /* lamp 1..5 , LAMP_ON / LAMP_OFF ; TWI status or
LAMP_ERR_NUMBER (0xFF) */
u8   LAMP_u8GetState(u8 Lamp);              /* last state written successfully */
```

RE-IMPLEMENTATION CHECKLIST

1. Shadow byte of the port.
2. Change a bit in a copy, write the byte.
3. Commit the copy only on ACK.
4. Active-low level as a cfg constant.
5. Distinct error code for a bad lamp number.

3.9 RELAY, LED, BUZZER — plain outputs HAL N

Three small drivers with the same shape: an ID, `_voidOn`, `_voidOff`, and an init that **sets the off level before making the pin an output**, so nothing switches on during reset. RELAY has `RELAY_HEATING` (PB6) and `RELAY_COOLING` (PB7); LED has `LED_HEATER` (PA3); BUZZER is PD3. All are active high.

`lib/HAL/RELAY/RELAY.c` · lines 21–27

```
21 void RELAY_voidInit(){
22     /* level first , then output : the element never switches on by accident */
23     RELAY_voidSet(RELAY_HEATING,RELAY_OFF_LEVEL);
24     RELAY_voidSet(RELAY_COOLING,RELAY_OFF_LEVEL);
25     DIO_voidSetPinDirection(RELAY_HEATING_PORT,RELAY_HEATING_PIN,DIO_PIN_OUTPUT);
26     DIO_voidSetPinDirection(RELAY_COOLING_PORT,RELAY_COOLING_PIN,DIO_PIN_OUTPUT);
27 }
```

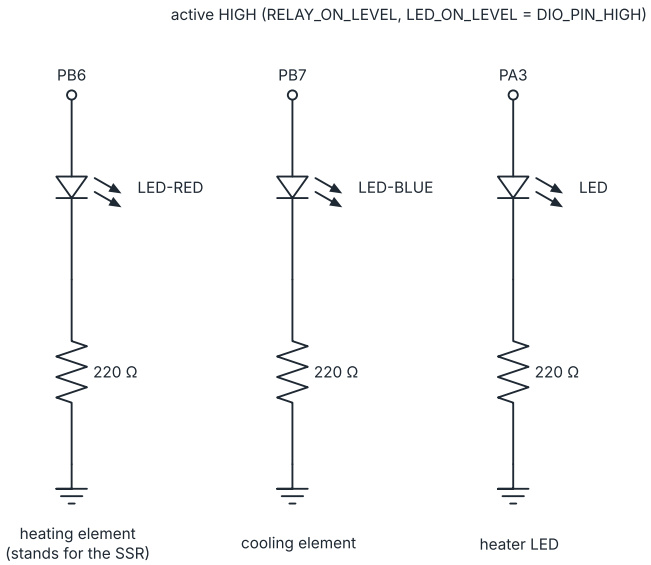


Figure 3.14 Heater outputs in Proteus: LEDs stand for the solid-state relays. drawn from the source code

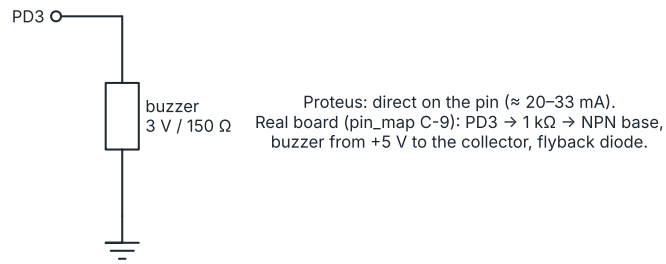
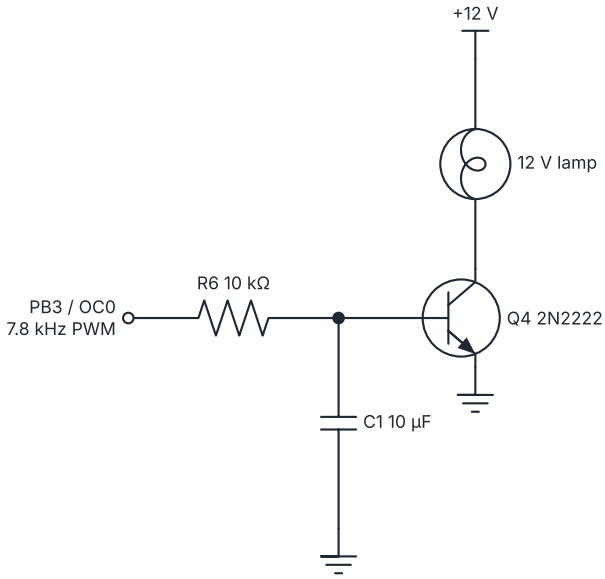


Figure 3.15 Buzzer: direct in Proteus; the driver stage a real board needs. drawn from the source code

HEATER guarantees that the two elements are never on together: it always switches one off before switching the other on (§5.6).

3.10 DIMMER, SERVO, FAN — PWM outputs HAL N

Three drivers on the PWM channels of chapter 2. They share the same corner-case rules: **0 %** = compare output disconnected and the pin driven low (a compare value of 0 still gives a one-count spike), **100 %** = compare value equal to TOP (constant high), and values above the range are clamped.



RC filter τ = R6·C1 = 100 ms → PWM becomes a 0–5 V level

Figure 3.16 Dimmer: PWM → RC filter → transistor → 12 V lamp. drawn from the source code

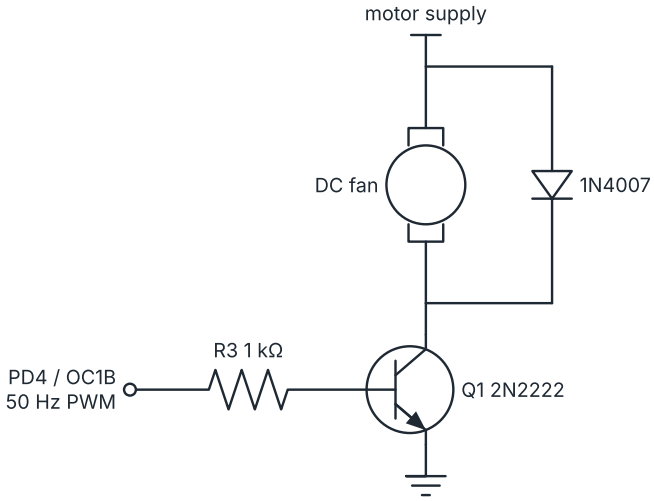


Figure 3.17 AC fan: OC1B → 1 kΩ → NPN; flyback diode across the motor. drawn from the source code

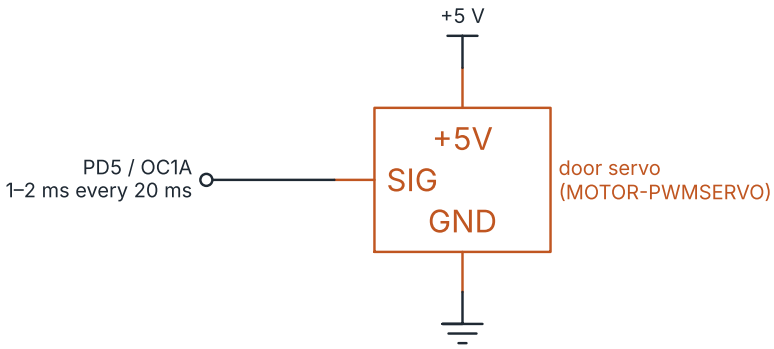


Figure 3.18 Door servo: the signal pin takes the 1–2 ms pulse directly. drawn from the source code

Driver	Pin / timer	Conversion	API
DIMMER	PB3 / Timer0 fast PWM, 7 812.5 Hz	OCR0 = p × 255 / 100	`DIMMER_voidInit()`, `DIMMER_voidSetLevel(u8 percent)`
SERVO	PD5 / Timer1 OC1A, 50 Hz	pulse = 1000 + angle × 1000 / 180 µs; OCR1A = pulse × 2000 / 1000	`SERVO_voidInit()` (→ 0°), `SERVO_voidSetAngle(u8 deg)`
FAN	PD4 / Timer1 OC1B, 50 Hz	OCR1B = p × 40000 / 100 (100 % = TOP)	`FAN_voidInit()` (stopped), `FAN_voidSetSpeed(u8 percent)`

lib/HAL/SERVO/SERVO.c · lines 20–30

```
20 void SERVO_voidSetAngle(u8 Copy_u8Angle){
21     u32 Local_u32PulseUs ;
22     if(Copy_u8Angle > SERVO_MAX_ANGLE){
23         Copy_u8Angle = SERVO_MAX_ANGLE ;
24     }
25     /*1. angle --> pulse width : 1000 us + angle x 1000 us / 180 */
26     Local_u32PulseUs = SERVO_MIN_PULSE_US + (((SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US) * Copy_u8Angle) /
SERVO_MAX_ANGLE) ;
27     /*2. pulse width --> timer ticks : 1.0 ms = 2000 , 1.5 ms = 3000 , 2.0 ms = 4000 at 16 MHz */
28     TIMER1_voidSetOCRA((u16)((Local_u32PulseUs * TIMER1_TICKS_PER_MS) / 1000UL));
29     TIMER1_voidGeneratePWM_A(TIMER1_PWM_NONINVERTED);
30 }
```

lib/HAL/DIMMER/DIMMER.c · lines 21–33

```
21 void DIMMER_voidSetLevel(u8 Copy_u8Percent){
22     if(Copy_u8Percent > 100){
23         Copy_u8Percent = 100 ;
24     }
25     if(Copy_u8Percent == 0){
26         /* compare value 0 would still give a one-tick spike : disconnect the pin and drive it low */
27         TIMER0_GeneratePWM(TIMER0_PWM_DISCONNECTED,0);
28         DIO_voidSetPinValue(DIMMER_PORT,DIMMER_PIN,DIO_PIN_LOW);
29     }else{
30         /* OCR0 = percent x 255 / 100 (100 % = 255 = constantly high) */
31         TIMER0_GeneratePWM(TIMER0_PWM_NONINVERTED,Copy_u8Percent);
32     }
33 }
```

PITFALL

The dimmer lamp stage: with R6 = 10 kΩ the base current is at most (5 − 0.7) V / 10 kΩ = 0.43 mA, while a 12 V / 24 Ω lamp needs 0.5 A. Unless the transistor gain is above about 1000, the lamp stays dim even at 100 %. pin_map.md notes the remedies (R6 = 1 kΩ, or a lighter lamp load in Proteus); no firmware change is involved.

3.11 EXT_EEPROM — the 24C08 HAL N

PURPOSE

Raw access to the 1 KB I²C EEPROM that holds the accounts and the heater set temperature (EEP-01). It knows the chip's two quirks, blocks and pages; the RAM image and the write-behind logic are one layer up in ESTORE.

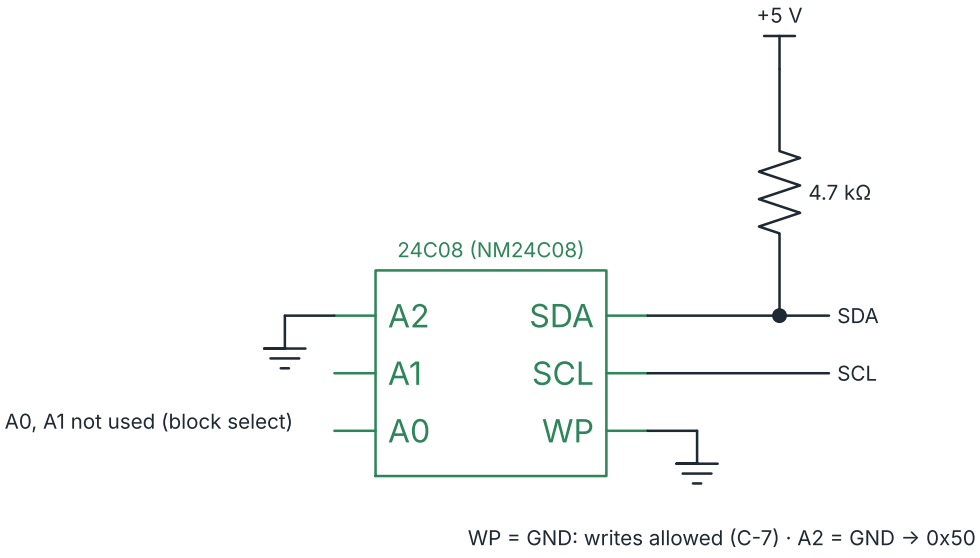


Figure 3.19 24C08 wiring: A2 and WP to ground. drawn from the source code

HARDWARE

- **Blocks:** 1024 bytes need 10 address bits, but one word-address byte carries 8. Bits 9:8 (B1 B0) travel in the slave address: block 0 = 0x50 ... block 3 = 0x53. A read is split at 256-byte borders. The project uses block 0 only (bytes 0x00–0xCF).

- **Pages:** a write may carry 1–16 bytes, all inside one 16-byte page; past the page end the chip wraps to the page start. `EXT_EEPROM_u8WritePage` refuses anything that would cross a page.
- **Write cycle:** after the STOP the chip programs the page for up to 5 ms and does not acknowledge its address meanwhile. That NACK is the "ready" signal: **ACK polling**.

`lib/HAL/EXT_EEPROM/EXT_EEPROM.c` · lines 62–65

```
62 u8 EXT_EEPROM_u8IsReady(){
63     /* the chip does not acknowledge its address while it is inside a write cycle (ACK polling) */
64     return (TWI_u8ProbeAddress(EXT_EEPROM_I2C_ADDRESS) == TWI_OK) ;
65 }
```

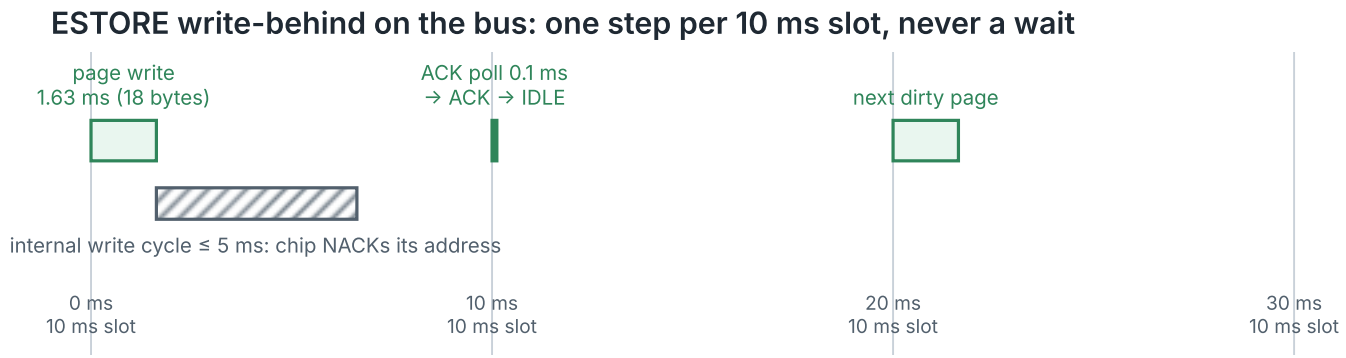


Figure 3.20 How ESTORE uses the three functions: a page write in one 10 ms slot, one ACK poll in the next.
computed from the code and F_CPU

API

```
void EXT_EEPROM_voidInit();
u8   EXT_EEPROM_u8ReadBlock(u16 Address, u8 *Data, u8 Length);       /* blocking , ≈ 90 µs per byte : boot
only */
u8   EXT_EEPROM_u8WritePage(u16 Address, const u8 *Data, u8 Length); /* 1..16 bytes in ONE page ; returns
after the STOP */
u8   EXT_EEPROM_u8IsReady();                                         /* ONE probe : 1 = write cycle over ;
never waits */
```

TIMING (COMPUTED)

Page write = START + address + word address + 16 data = 18 bytes → 1630 µs. ACK poll = START + address + STOP → 100 µs. Boot read of 208 bytes = write phase (2 bytes) + repeated START + read phase (209 bytes) → 19.01 ms (simavr: 19.3 ms).

PITFALL

The Proteus 24C08 model has no write-cycle delay: it acknowledges right after a write, so the first poll always succeeds. Real chips need the polling, so it is written for them (specification §4). `pin_map.md` §2 still says the model has `TD_WRITE = 10 ms`; that contradicts the other documents and the observed behaviour.

RE-IMPLEMENTATION CHECKLIST

1. Slave address = 0x50 | address bits 9:8.
2. Read: write the word address, repeated START, read with ACK … NACK.
3. Write: one page at most, never across a border.
4. Ready = one probe; never loop on it.
5. Leave timing decisions to the caller.

CHAPTER 4

SERVICE — time, text, storage and events

The SERVICE layer holds everything that is neither hardware nor product behaviour. **System services** — SCHED, TERM, ESTORE, USERDB, EVQ — own a resource and its rules. **Utilities** — RINGBUF, MAVG, FMT, flash_str.h — are pure code and may be used from any layer. This is also where the ISR world meets the main loop, so each section states exactly what is shared and how it is protected.

Module	Owns	Called from	RAM (B)	Shared with an ISR?
SCHED	Timer2, tick counter, 5 task flags	main loop; Timer2 ISR	17	yes — tick and flags
TERM	USART, RX/TX rings, line editor	UIREM; USART ISRs	194	yes — rings and RX-activity flag
ESTORE	24C08, 208-byte RAM image	USERDB, HEATER, UIREM; 10 ms task	215	no
USERDB	account rules, write gate	SEC, UIREM	1	no
EVQ	event ring	feature modules post, UIREM reads	39	no (main loop only)
RINGBUF · MAVG · FMT	— (data passed in)	TERM, EVQ · HEATER, CLIMATE · UIs, TERM	0	RINGBUF is ISR-safe by design

4.1 SCHED — time base and task flags SERVICE N

PURPOSE

A cooperative time-triggered scheduler: the 1 ms Timer2 interrupt sets one flag per period (5, 10, 100, 500, 1000 ms) and the super-loop runs a task when its flag is set. It replaces every `_delay_ms` of the course code with a structure in which no task ever waits.

DATA STRUCTURE

```
static volatile u32 Global_u32Tick;          /* ms since SCHED_voidInit          ISR writes , main reads
(atomically) */
static volatile u8  Global_u8Flags;          /* bit n = task n due               ISR sets   , main tests-and-
clears   */
static volatile u16 Global_u16Overruns;      /* flag set again before served     ISR writes , main reads
(atomically) */
static u16 Global_u16DownCount[5];           /* ISR only (loaded before the interrupt is enabled)
*/
static const __flash u16 SCHED_PERIOD_ARR[5] = {5,10,100,500,1000};
```

ALGORITHM

`lib/Service/SCHED/SCHED.c` · lines 32–46 · SCHED_voidTick: the whole ISR work

```
32      Global_u32Tick++;
33      /*2. one down-counter per task */
34      for(i = 0 ; i < SCHED_TASK_COUNT ; i++){
35          Global_u16DownCount[i]--;
36          if(Global_u16DownCount[i] == 0){
37              Global_u16DownCount[i] = SCHED_PERIOD_ARR[i] ;
38              if((Global_u8Flags & Local_u8Mask) != 0){
39                  /* the main loop did not serve the last one in time */
40                  Global_u16Overruns++;
41              }
42              Global_u8Flags |= Local_u8Mask ;
43          }
44          Local_u8Mask <<= 1 ;
45      }
46  }
```

Each counter starts at `period + offset` with offsets 0, 2, 1, 3, 4 ms (`SCHED_cfg.h`), so the tasks are first due on ticks **5, 12, 101, 503, 1004** and then every period. Modulo 5 these phases are 0, 2, 1, 3, 4 — all different. Since every period is a multiple of 5 ms, two flags are therefore **never set on the same tick**, and a long task delays the next one by at most the time it overruns its own millisecond (chapter 7).

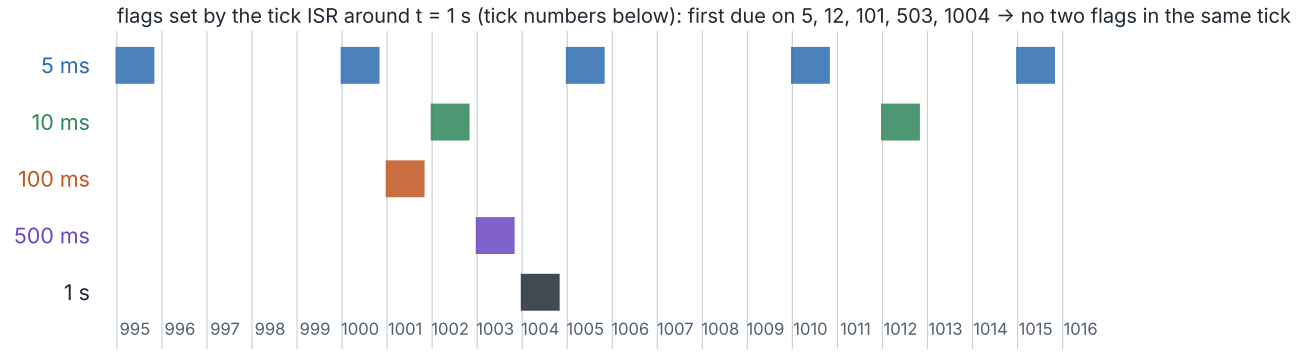


Figure 4.1 Flags set by the tick ISR around t = 1 s. computed from the code and F_CPU

ISR / MAIN-LOOP SHARING

- The **test-and-clear** of a flag in `SCHED_u8IsTaskDue` runs with interrupts off. The ISR sets other bits in the same byte; a read-modify-write that the ISR interrupts would lose a flag.
- The 32-bit tick and the 16-bit overrun count are copied with interrupts off: the AVR reads one byte at a time, and a tick between two byte reads gives a torn value (for example 0x00FF → 0x0100 read as 0x01FF).
- The down-counters belong to the ISR alone; the main loop never touches them after `SCHED_voidInit`, so they need no protection.
- Timeouts are counted in calls of the owning task (5 s = 50 calls of the 100 ms task), never by comparing 32-bit tick values, so no module needs `SCHED_u32GetTickMs` in the loop.

`lib/Service/SCHED/SCHED.c` · lines 68–82

```
68 u8 SCHED_u8IsTaskDue(u8 Copy_u8TaskID){
69     u8 Local_u8Due = 0 ;
70     if(Copy_u8TaskID < SCHED_TASK_COUNT){
71         /* test-and-clear must not be cut by the tick ISR (it sets bits in the same byte) */
72         GIE_voidDisableGlobalInterrupt();
73         if(GET_BIT(Global_u8Flags,Copy_u8TaskID)){
74             CLR_BIT(Global_u8Flags,Copy_u8TaskID);
75             Local_u8Due = 1 ;
76         }
77         GIE_voidEnableGlobalInterrupt();
78     }else{
79         //error
80     }
81     return Local_u8Due ;
82 }
```

INVARIANTS

- A flag is set at most once per period; if it is still set when the ISR sets it again, `Global_u16Overruns` counts the lost period. `test_service` checks 2000/1000/100/20/10 flags in 10 s with 0 overruns; the simavr runs of the application also end with 0 overruns (§7.6).
- A late task causes **jitter, not drift**: the counters keep running in the ISR, independent of when the loop serves a flag.

API AND COST

```
void SCHED_voidInit();   /* Timer2 CTC + callback ; interrupt still off ; may be called again to restart at 0
*/
void SCHED_voidStart();  /* OCIE2 on */
u8   SCHED_u8IsTaskDue(u8 TaskID);   /* SCHED_TASK_5MS / 10MS / 100MS / 500MS / 1S ; 1 once per period */
u32  SCHED_u32GetTickMs();  u16 SCHED_u16GetOverruns();
```

RAM 17 bytes. ISR length 235–261 cycles (simavr) = 14.7–16.3 µs, i.e. about 1.6 % of the CPU. architecture.md §4.1 estimates "under 10 µs = 1 %"; the measured value is higher (see the discrepancy list).

RE-IMPLEMENTATION CHECKLIST

1. Exact 1 ms CTC interrupt.
2. ISR: tick++, one down-counter per period, set a bit, count overruns.
3. Distinct start offsets so no two flags share a tick.
4. Atomic test-and-clear in the main loop.
5. Count timeouts in task calls.

4.2 RINGBUF — lock-free byte ring SERVICE N

PURPOSE

A FIFO of bytes between exactly one producer and one consumer, safe between an ISR and the main loop **without disabling interrupts**. Used three times: TERM RX (ISR → loop), TERM TX (loop → ISR), and EVQ (loop → loop).

DATA STRUCTURE

```
typedef struct{
    volatile u8 * Buffer ;   /* storage given by the user                          */
    u8   Size ;              /* bytes of storage ; capacity = Size - 1            */
    volatile u8 Head ;       /* next free slot  - written ONLY by the producer    */
    volatile u8 Tail ;       /* next byte to read - written ONLY by the consumer */
}RINGBUF_t;
```

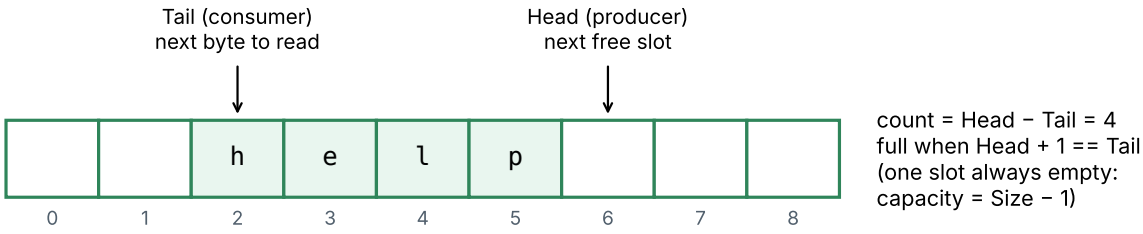


Figure 4.2 Head and Tail; one slot always stays empty. drawn from the source code

ALGORITHM AND WHY IT IS SAFE

Empty is `Head == Tail`; full is `Head + 1 == Tail` (modulo Size). Keeping one slot empty means the two states never look the same, so no shared counter is needed. Each index is written by one side only, and the AVR writes a `u8` in a single instruction, so the other side always sees either the old or the new value, never a torn one.

The order inside `Put` is the whole trick: **store the byte first, then publish the new Head**. If the ISR consumer interrupted between the two, it would still see the old Head and not read the unfinished slot. `volatile` on the buffer stops the compiler (with LTO) from reordering or caching these accesses.

INVARIANTS AND COST

- 0 ≤ count ≤ Size − 1; `GetCount` may be one step stale when read across an interrupt, which only makes the answer conservative.
- Users size their storage as capacity + 1: TERM RX 33, TX 129, EVQ 33. `Size` is a `u8`, so a ring holds at most 254 bytes.
- No RAM of its own; 5 bytes per `RINGBUF_t`.

RE-IMPLEMENTATION CHECKLIST

1. Head written by the producer only, Tail by the consumer only.
2. 8-bit indexes (atomic on the AVR).
3. One empty slot distinguishes full from empty.
4. Write data, then publish the index.
5. Mark the buffer and indexes volatile.

4.3 TERM — UART terminal I/O SERVICE N

PURPOSE

Turns the USART into a terminal for UIREM (RUI-01): interrupt-driven RX and TX through rings, a line editor with echo, masked echo for passwords, and output that **never waits**.

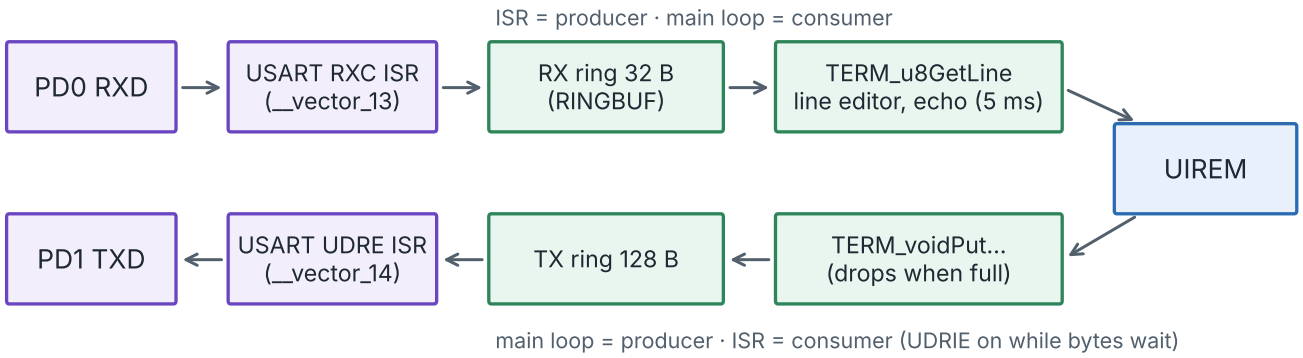


Figure 4.3 Data path. Each ring has one ISR side and one main-loop side. drawn from the source code

DATA STRUCTURE

```
static volatile u8 Global_u8RxStorage[33] , Global_u8TxStorage[129] ;   /* TERM_RX/TX_BUFFER_SIZE + 1 */
static RINGBUF_t Global_RxRing , Global_TxRing ;
static volatile u8 Global_u8RxActive ;                                  /* set by the RX ISR , read-and-clear
atomically */
static c8 Global_c8Line[17] ; u8 Length , TooLong , LastWasCR , Echo ;   /* line editor : main loop only */
```

ALGORITHM

lib/Service/TERM/TERM.c · lines 43–50 · UDRE ISR callback

```
43 static void TERM_voidTxCallback(void){
44     u8 Local_u8Data ;
45     if(RINGBUF_u8Get(&Global_TxRing,&Local_u8Data) == RINGBUF_OK){
46         USART_voidWriteData(Local_u8Data);
47     }else{
48         USART_voidDisableTxInterrupt();
49     }
50 }
```

lib/Service/TERM/TERM.c · lines 73–80 · main-loop side

```
73 void TERM_voidPutChar(c8 Copy_c8Char){
74     if(RINGBUF_u8Put(&Global_TxRing,(u8)Copy_c8Char) == RINGBUF_OK){
75         /* the byte is in the ring before the interrupt is enabled , so it can not be missed */
76         USART_voidEnableTxInterrupt();
77     }else{
78         /* ring full : the character is dropped , the loop never waits for the UART */
79     }
80 }
```

TX: `TERM_voidPut…` puts bytes in the TX ring and enables UDRIE. The UDRE interrupt then sends one byte per frame until the ring is empty and switches itself off. The byte is in the ring *before* UDRIE is enabled, so it cannot be missed. A full ring **drops** the character; callers check `TERM_u8TxFree()` first (UIREM prints only when ≥ 64 bytes are free).

RX and line editor (`TERM_u8GetLine`, every 5 ms from UIREM): bytes are taken from the RX ring and edited.

- Printable 0x20–0x7E are stored and echoed (or `*` in masked mode).
- Backspace 0x08 or Delete 0x7F removes the last character and echoes `"\b \b"`.
- CR, LF and CR LF each end **one** line: the LF right after a CR is skipped, even when it arrives in a later call.
- Other control codes are ignored.
- A line longer than 16 characters keeps its first 16 on screen, ignores the rest, and returns `TERM_LINE_TOO_LONG` once when its end arrives.

ISR / MAIN-LOOP SHARING

The rings need no locking (§4.2). `Global_u8RxActive` (used by UIREM's 120 s idle timer) is read and cleared with interrupts off, because the ISR sets it. The line editor belongs to the main loop alone.

TIMING (COMPUTED)

The 32-byte RX ring holds 33.3 ms of continuous input, far more than the 5 ms between two `TERM_u8GetLine` calls (at most ≈ 5 bytes arrive meanwhile). The 128-byte TX ring drains in 133 ms. The 408-byte admin menu therefore goes out over ≈ 0.42 s, one line per UIREM step, without the loop ever waiting.

API

```
void TERM_voidInit();  u8 TERM_u8TxFree();
void TERM_voidPutChar(c8); void TERM_voidPutFlash(const __flash c8*); void TERM_voidPutRam(const c8*); void
TERM_voidPutNumber(u16); void TERM_voidNewLine();
void TERM_voidSetEcho(u8);              /* TERM_ECHO_OFF / NORMAL / MASKED */
u8   TERM_u8GetLine(c8 *Line);          /* TERM_LINE_NONE / READY / TOO_LONG ; buffer of 17 bytes */
u8   TERM_u8IsLineEmpty();  u8 TERM_u8IsRxActive();
```

RE-IMPLEMENTATION CHECKLIST

1. RX ISR → ring; TX ring → UDRE ISR.
2. Enable UDRIE after queuing; disable when empty.
3. Drop on full, never wait.
4. Line editor with CR/LF/CRLF, backspace, length limit.
5. Masked echo for passwords.

4.4 EVQ — event queue SERVICE N

PURPOSE

The one-way link from feature modules to the remote terminal (RUI-02). A feature module must not call a UI (layer rule 6), yet the terminal must report every change made elsewhere. Features post `(event, argument)` pairs; UIREM prints them as `[INFO]` lines.

Event	ID	Argument	Posted by	Printed as
EVQ_LAMP	1	lamp number	LIGHT	`[INFO] Lamp 2 is now ON`
EVQ_DIMMER	2	percent	LIGHT	`[INFO] Dimmer is now 30%`
EVQ_AC	3	1 / 0	CLIMATE	`[INFO] AC is now ON`
EVQ_HEATER_POWER	4	1 / 0	HEATER	`[INFO] Heater is now ON`
EVQ_HEATER_ELEMENT	5	0 / 1 / 2	HEATER	`[INFO] Heater: HEATING`
EVQ_HEATER_SET	6	°C	HEATER	`[INFO] Heater set temperature is now 65C`
EVQ_LOCAL_SESSION	7	1 login / 0 logout	SEC	`[INFO] Keypad user logged in`
EVQ_STORAGE_FAULT	8	—	ESTORE	`[WARN] EEPROM not responding…`
EVQ_LOCKDOWN	9	—	ALARM	not printed: UIREM prints the alert from `ALARM_u8IsActive()`

DESIGN

A RINGBUF of 33 bytes holds 16 events of 2 bytes. `EVQ_voidPost` checks that 2 bytes are free and then stores both, so an event is never stored half. When the queue is full the *new* event is dropped and the older ones keep their order. **Mute:** while UIREM executes one of its own commands it mutes the queue, because that change is already answered with `[OK]`; the queue therefore only ever holds changes made somewhere else. Main-loop context only: no ISR posts events.

lib/Service/EVQ/EVQ.c · lines 25–35

```
25 void EVQ_voidPost(u8 Copy_u8Event, u8 Copy_u8Arg){
26     if(Global_u8Muted == 1){
27         /* UIREM runs its own command : that change is already answered with [OK] */
28     }else if(RINGBUF_u8GetFree(&Global_Ring) < EVQ_BYTES_PER_EVENT){
29         /* full : the new event is dropped , the older ones stay in order */
30     }else{
31         /* both bytes or none : an event is never stored half */
32         RINGBUF_u8Put(&Global_Ring,Copy_u8Event);
33         RINGBUF_u8Put(&Global_Ring,Copy_u8Arg);
34     }
35 }
```

RE-IMPLEMENTATION CHECKLIST

1. Fixed-size records in a ring.
2. Check room for the whole record first.
3. Drop new on full.
4. Mute switch for self-caused changes.
5. Consumer drains when idle.

4.5 ESTORE — EEPROM image with write-behind SERVICE N

PURPOSE

All persistent data — magic/version, heater set temperature, admin and 10 user records — in a 208-byte RAM image that mirrors bytes 0x00–0xCF of the 24C08 (EEP-01…03, D-5). Reads never touch the chip; writes go out in the background, one page per 10 ms slot, with ACK polling. No caller ever waits for the EEPROM.

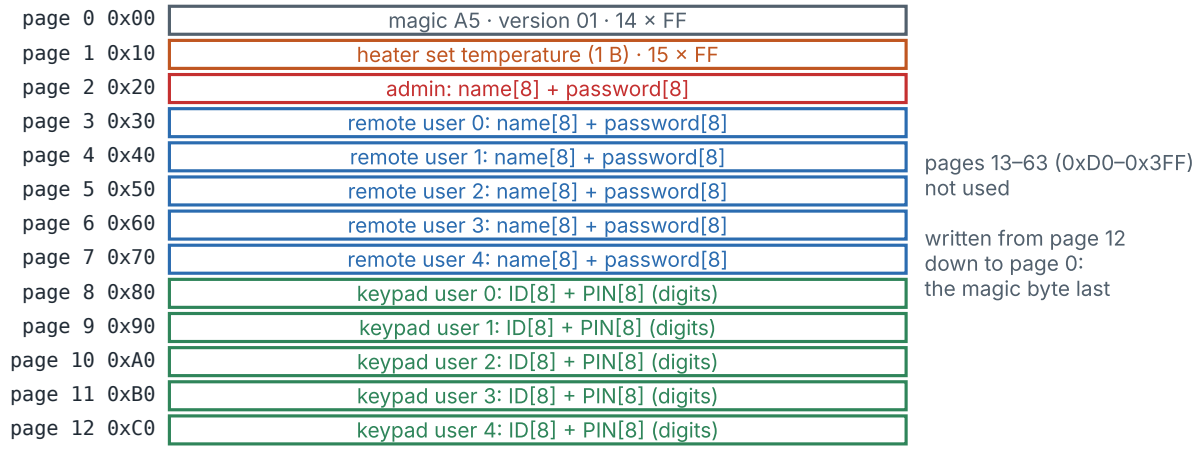


Figure 4.4 EEPROM map (`ESTORE_cfg.h`, eeprom_map.md). Every record is exactly one 16-byte page. drawn from the source code

DATA STRUCTURE

```
static u8  Global_u8Image[208] ;   /* = chip bytes 0x00..0xCF                */
static u16 Global_u16Dirty ;       /* bit n = page n differs from the chip (13 bits) */
static u8  Global_u8State ;        /* IDLE / WRITING / FAULT                  */
static u8  Global_u8Polls , Global_u8Status ; static u16 Global_u16PageWrites ;
```

ALGORITHM

Boot (`ESTORE_voidInit`, before the scheduler): one blocking read of 208 bytes (≈ 19 ms). If the read fails: defaults in RAM, state FAULT, nothing is ever written. If the magic (0xA5) or the version (0x01) is wrong: first boot, so load the defaults and mark all 13 pages dirty.

Write: `ESTORE_voidWriteByte` compares first; an unchanged byte costs nothing, which spares the 1 000 000-cycle endurance. A changed byte updates the image and sets the page's dirty bit.

Write-behind (`ESTORE_voidUpdate`, 10 ms task): one step per call.

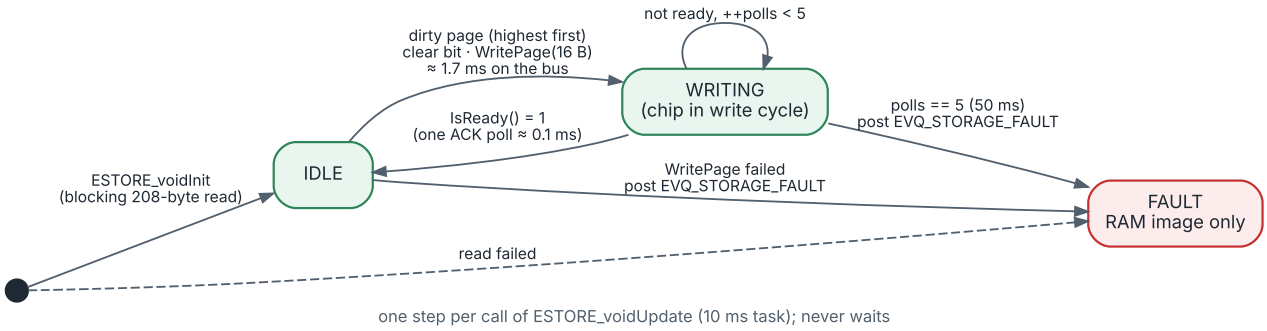


Figure 4.5 ESTORE write-behind state machine. drawn from the source code

`lib/Service/ESTORE/ESTORE.c` · lines 93–107 · IDLE step

```
 93     case ESTORE_STATE_IDLE:
 94         if(Global_u16Dirty != 0){
 95             /*1. highest dirty page first : on first boot the magic page 0 is written last */
 96             Local_u8Page = ESTORE_PAGE_COUNT - 1 ;
 97             while(GET_BIT(Global_u16Dirty,Local_u8Page) == 0){
 98                 Local_u8Page--;
 99             }
100             /*2. clear the bit BEFORE the write : a byte changed during the write cycle marks the page
again */
101             Global_u16Dirty &= ~((u16)1 << Local_u8Page) ;
102             Global_u8Polls = 0 ;
103             if(EXT_EEPROM_u8WritePage((u16)Local_u8Page * ESTORE_PAGE_SIZE,&Global_u8Image[Local_u8Page *
ESTORE_PAGE_SIZE],ESTORE_PAGE_SIZE) == EXT_EEPROM_OK){
104                 Global_u16PageWrites++;
105                 Global_u8State = ESTORE_STATE_WRITING ;
106             }else{
107                 ESTORE_voidEnterFault();
```

Two details carry the reliability:

- **Highest dirty page first.** On first boot, page 0 with the magic byte is written last. If power fails during the first-boot writes, the magic is still missing, so the next boot simply starts again.
- **Dirty bit cleared before the write.** If a byte of that page changes during the write cycle, the bit is set again and the page is written once more. A change can never be lost between "sent" and "done".

TIMING

One page = one 10 ms slot for the write (1.63 ms on the bus) + one slot for the poll = **20 ms per page**. A real 24C08 (≤ 5 ms write cycle) is always ready by the next slot; the Proteus model is ready at once. First boot: 13 pages = 260 ms in the background (simavr: page writes from 84.7 ms to 324.7 ms). After 5 failed polls (50 ms) the chip is declared gone: FAULT, `EVQ_STORAGE_FAULT`, and the system keeps working from RAM.

API

```
void ESTORE_voidInit();  void ESTORE_voidUpdate();  void ESTORE_voidLoadDefaults();   /* factory reset */
u8   ESTORE_u8ReadByte(u8 Addr);  void ESTORE_voidReadBlock(u8 Addr, u8 *Data, u8 Len);
void ESTORE_voidWriteByte(u8 Addr, u8 Data);  void ESTORE_voidWriteBlock(u8 Addr, const u8 *Data, u8 Len);
u8   ESTORE_u8IsBusy();     /* 0 in FAULT : nobody waits for a write that will never happen */
u8   ESTORE_u8GetStatus();  /* ESTORE_OK / ESTORE_FIRST_BOOT / ESTORE_FAULT */
u16  ESTORE_u16GetPageWrites();   /* diagnostic for test_service */
```

INVARIANTS

- The image is the truth; the chip follows within 20 ms per dirty page.
- Every record is one page, so a record write is atomic (D-6).
- Page 0 is written only by first boot and factory reset; the heater set temperature has its own page 1, so saving it can never touch the magic byte.

RE-IMPLEMENTATION CHECKLIST

1. Mirror the used area in RAM; read it once at boot.
2. Magic + version → defaults when invalid.
3. Write = image + dirty bit, skip unchanged bytes.
4. Background: write one page, poll once per slot, bounded retries.
5. Highest page first; clear the dirty bit before writing.

4.6 USERDB — accounts SERVICE N

PURPOSE

The rules for the admin record and the two user lists (SEC-02, 03, 04, 07, 09), stored in the ESTORE image.

DATA STRUCTURE

A record is 16 bytes, one page: name[8] + password[8], padded with 0x00 and **not** terminated when 8 characters long. A slot is empty when its first name byte is 0x00 or 0xFF (no separate valid flag, D-6). Remote users are at 0x30–0x7F (5 slots), keypad users at 0x80–0xCF (5 slots), the admin at 0x20. The only RAM is `Global_u8WriteAccess`.

ALGORITHM AND RULES

Rule	Remote user / admin	Keypad user
Name	1–8 printable characters 0x21–0x7E (no space)	1–8 digits
Password	4–8 printable characters	4–8 digits
Unique	inside the list, and not the admin's name	inside the list

`lib/Service/USERDB/USERDB.c` · lines 58–72 · compare all 8 positions

```
58 static u8 USERDB_u8IsFieldEqual(u8 Copy_u8Address, const c8 * Copy_pc8Text){
59     u8 Local_u8Length = USERDB_u8TextLength(Copy_pc8Text);
60     u8 Local_u8Char ;
61     u8 i ;
62     if(Local_u8Length > USERDB_NAME_SIZE){
63         return 0 ;
64     }
65     for(i = 0 ; i < USERDB_NAME_SIZE ; i++){
66         Local_u8Char = (i < Local_u8Length) ? (u8)Copy_pc8Text[i] : 0x00 ;
67         if(ESTORE_u8ReadByte(Copy_u8Address + i) != Local_u8Char){
68             return 0 ;
69         }
70     }
71     return 1 ;
72 }
```

Every comparison covers all 8 positions, treating the C string as 0x00-padded. "bob" therefore never matches "bobby", and an 8-character name, which has no terminator in the image, still compares correctly. Only used slots are searched, so an empty name can never match an empty slot. Add and remove write a whole record through `ESTORE_voidWriteBlock` — one page write, atomic. Remove fills the record with 0x00, so no old password is left behind.

Write gate (SEC-07, D-12): add, remove and set-admin-password return `USERDB_ERR_READ_ONLY` unless `USERDB_voidSetWriteAccess(1)` was called, which SEC does only while the admin is logged in. `USERDB_voidInit` repairs a missing or unprintable admin name with `admin` / `1234`; this repair is not a user action and bypasses the gate.

API

```
void USERDB_voidInit();
u8 USERDB_u8CheckAdmin(const c8 *Name, const c8 *Pass);                 /* 1 = match */
u8 USERDB_u8CheckUser (u8 List, const c8 *Name, const c8 *Pass);        /* USERDB_LIST_REMOTE / KEYPAD */
u8 USERDB_u8AddUser   (u8 List, const c8 *Name, const c8 *Pass);        /* OK ,
ERR_FULL/EXISTS/NOT_FOUND/BAD_NAME/BAD_PASS/READ_ONLY */
u8 USERDB_u8RemoveUser(u8 List, const c8 *Name);
u8 USERDB_u8GetUserName(u8 List, u8 Slot, c8 *Name);   /* 1 = slot used ; buffer of 9 */
u8 USERDB_u8SetAdminPassword(const c8 *Pass);  void USERDB_voidSetWriteAccess(u8 State);
```

Passwords are stored and compared as plain text (D-13): this is a simulation project, and hashing is a listed improvement.

4.7 MAVG — 10-sample moving average SERVICE N

The averaging behind HTR-08 (water) and the AC decision (ambient). `MAVG_t` holds the last 10 samples, their running sum, an index and a count: 24 bytes per instance. Adding a sample subtracts the oldest from the sum once the window is full, so each update costs O(1), not 10 additions. The average is rounded to nearest in integer arithmetic: `(Sum + Count/2) / Count`. The sum of 10 samples of at most 1023 is 10 230, which fits a `u16`. `MAVG_u8IsFull` tells the caller when 10 samples exist; HEATER makes no decision before that.

lib/Service/MAVG/MAVG.c · lines 19–33

```
19 void MAVG_voidAddSample(MAVG_t * avg, u16 Copy_u16Sample){
20     if(avg->Count == MAVG_WINDOW){
21         /*1. window full : the oldest sample leaves the sum */
22         avg->Sum -= avg->Samples[avg->Index] ;
23     }else{
24         avg->Count++;
25     }
26     /*2. the new sample takes its place */
27     avg->Samples[avg->Index] = Copy_u16Sample ;
28     avg->Sum += Copy_u16Sample ;
29     avg->Index++;
30     if(avg->Index >= MAVG_WINDOW){
31         avg->Index = 0 ;
32     }
33 }
```

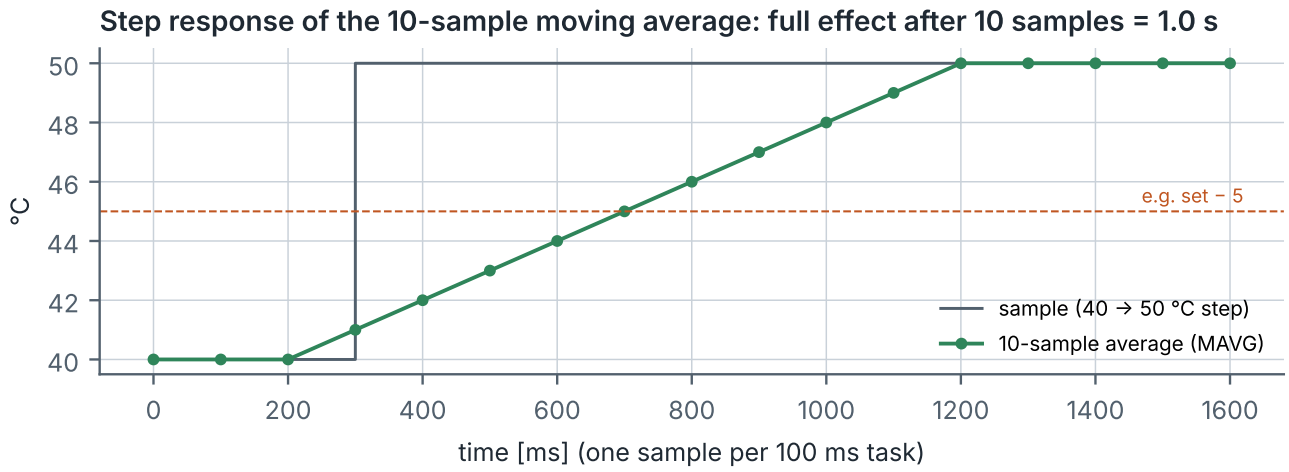


Figure 4.6 Step response at 100 ms per sample. A step reaches a threshold in the middle of the band after about half the window and its full value after 10 samples = 1 s. This is the "average fill time" of the latency chains in chapter 7. computed from the code and F_CPU

4.8 FMT — number ↔ text without printf SERVICE N

`FMT_u8NumberToText(u16, c8*)` writes decimal digits (repeated `% 10`, then reversed) and returns the length; the buffer needs 6 bytes ("65535" + '\0'). `FMT_u8TextToNumber(const c8*, u16*)` accepts digits only, rejects an empty string, and checks for overflow *before* the multiplication (value > 6553, or 6553 with a digit > 5). This avoids `printf` / `sscanf`, which cost about 1.5 KB of flash and need far more stack.

4.9 flash_str.h — text in flash

RAM is 2 KB, and the terminal text alone is about 2 KB, so every constant string lives in flash. `<avr/pgmspace.h>` (and `PSTR()`) pulls in `<avr/io.h>`, which redefines the registers of `reg_def.h`. The project uses GCC's named address space `__flash` instead:

lib/Service/flash_str.h · lines 15–15

```
15 #define FLASH_STR(str)    (__extension__({ static const __flash c8 Local_c8Text[] = (str);
&Local_c8Text[0]; }))
```

Each `FLASH_STR("…")` creates a static array in program memory and yields a `const __flash c8 *`. Functions that print text take that pointer type (`TERM_voidPutFlash`, `LCD_BUF_voidWriteFlash`, `CLCD_voidSendFlashString`), and the compiler emits `lpm` instructions to read it.

> **LESSON KEPT IN UIREM.C**
> A `switch` whose cases only return a text pointer was turned by the compiler into a pointer table in RAM (it cost about 100 bytes). The cases now print the text themselves (`UIREM_u8PutFixedText`).

CHAPTER 5

Subsystems — the APP layer

Eight modules carry the behaviour: two user interfaces (UIREM, UILOC), the security core (SEC, ALARM) and four features (LIGHT, DOOR, CLIMATE, HEATER). Each is a non-blocking state machine: it keeps its state in `static` variables, runs to completion every time it is called, and never waits. The state tables follow the code, not the design documents.

Module	Called by the loop	Calls	Requirement IDs
SEC	— (called by the UIs)	USERDB, ALARM, EVQ	SEC-01, 02, 05, 06, 07, 08
ALARM	500 ms	HEATER, CLIMATE, BUZZER, EVQ	ALM-01, SEC-05
LIGHT	—	LAMP, DIMMER, EVQ	LGT-01, LGT-02
DOOR	—	SERVO	DOR-01, DOR-02
CLIMATE	100 ms	LM35, MAVG, FAN, EVQ	AC-01, 02, 03
HEATER	10, 100, 500 ms	BUTTON, LM35, MAVG, RELAY, LED, SEVEN_SEG, ESTORE, EVQ	HTR-01 … HTR-14
UILOC	10 ms, 1 s	KPAD, LCD_BUF, FMT, SEC, LIGHT, CLIMATE, HEATER, ALARM	LUI-01 … 04, SEC-06, SEC-08
UIREM	5 ms, 1 s	TERM, EVQ, FMT, SEC, USERDB, ESTORE, LIGHT, DOOR, CLIMATE, HEATER, ALARM	RUI-01 … 04, SEC-01/03/06, DOR-01, HTR-14

5.1 SEC — sessions, roles and the lockdown counter APP N

PURPOSE

SEC decides who is logged in where, what role they have, and when the system locks. It keeps one session and one failure counter per login source: `SEC_SOURCE_REMOTE` (UART) and `SEC_SOURCE_LOCAL` (keypad).

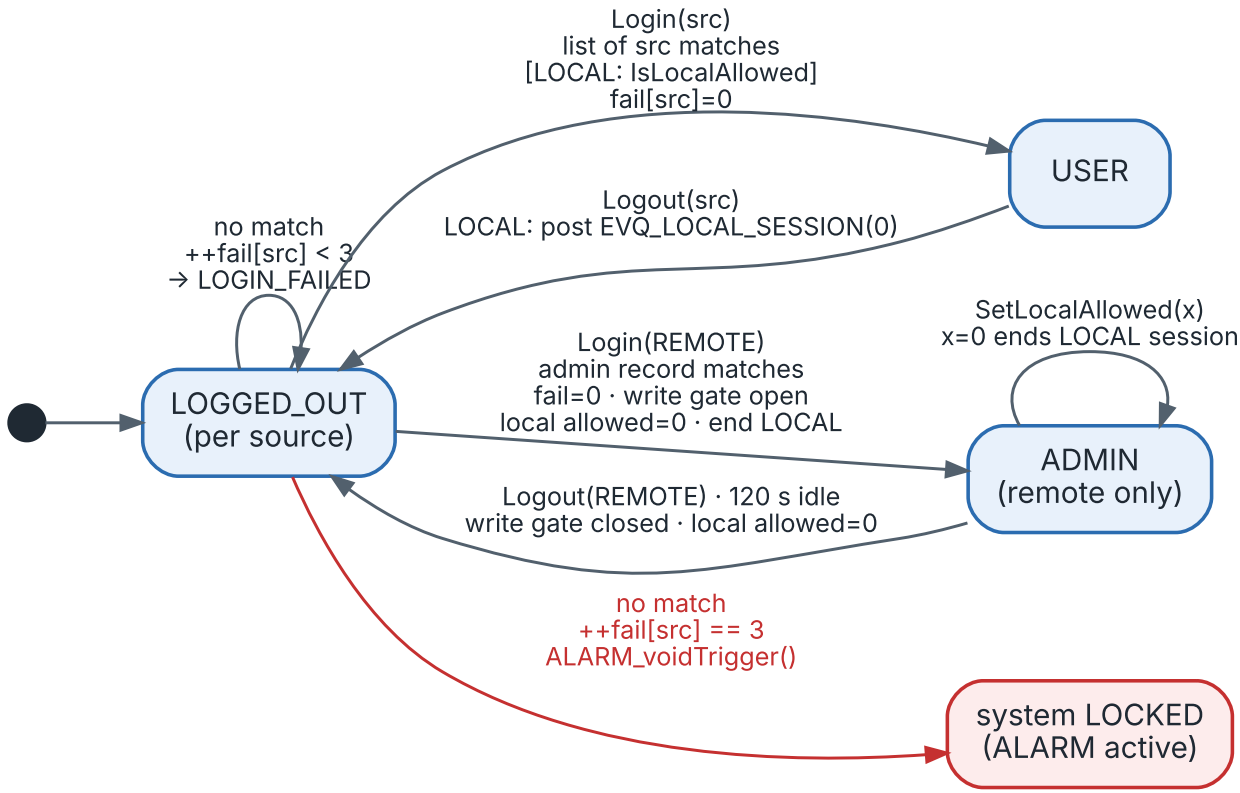


Figure 5.1 SEC session machine (per source) and the lockdown transition. drawn from the source code

EVENTS AND TRANSITIONS

- **Login(src, name, pass)** — one attempt = one name+password pair checked together; the reply never says which half was wrong.
  - If the alarm is active, the result is `SEC_LOGIN_LOCKED` and nothing is checked.
  - REMOTE: the admin record first (only for this source, SEC-01), then the remote list. LOCAL: the keypad list, and only if `SEC_u8IsLocalAllowed()`.
  - On success: `fail[src] = 0` and a role is set. An admin login also opens the USERDB write gate, sets local-allowed = 0 and ends any keypad session (decision 1). A keypad login posts `EVQ_LOCAL_SESSION(1)`.
  - On failure: `++fail[src]`; at 3 (`SEC_MAX_FAILED_LOGINS`) it calls `ALARM_voidTrigger()` and returns `SEC_LOGIN_LOCKED` (SEC-05: the 3rd failure, decision 2).
- **Logout(src)** — ends the session. For the admin it closes the write gate and resets local-allowed; for the keypad it posts `EVQ_LOCAL_SESSION(0)`.
- **SetLocalAllowed(x)** — effective only while the admin is logged in; x = 0 also ends a keypad session (SEC-08).

`src/APP/SEC/SEC.c` · lines 89–94 · end of SEC_u8Login

```
89          /* REQ-SEC-05 : the 3rd failure in a row on this source locks the system */
90          Global_u8FailCount[Copy_u8Source]++;
91          if(Global_u8FailCount[Copy_u8Source] >= SEC_MAX_FAILED_LOGINS){
92              ALARM_voidTrigger();
93              Local_u8Result = SEC_LOGIN_LOCKED ;
94          }
```

`SEC_u8IsLocalAllowed()` = not locked AND NOT (remote role is ADMIN and local-allowed = 0). A successful login on the same source resets its counter, and the two counters are independent, so 2 wrong on the terminal + 2 wrong on the keypad does not lock (test T-11).

Permissions (enforced by the UIs through `SEC_u8GetRole`, SEC-06, decision D-9/D-12)

Action	Admin (UART)	Remote user	Keypad user
Lamps, dimmer, view AC / heater / status	yes	yes	yes
Change heater set temperature	yes	yes	yes
Open / close the door	yes	no	no (no menu entry)
Accounts, admin password, factory reset, keypad allow/block	yes	no	no

INTEGRATION

`SEC_voidInit()` after `USERDB_voidInit` and `ALARM_voidInit`. No task function: SEC reacts only to calls from UIREM and UILOC. Note: a LOCAL login attempt while the keypad is blocked would count as a failure; UILOC prevents this by showing "Keypad blocked" instead of calling SEC (§5.7).

5.2 ALARM — latched lockdown APP N

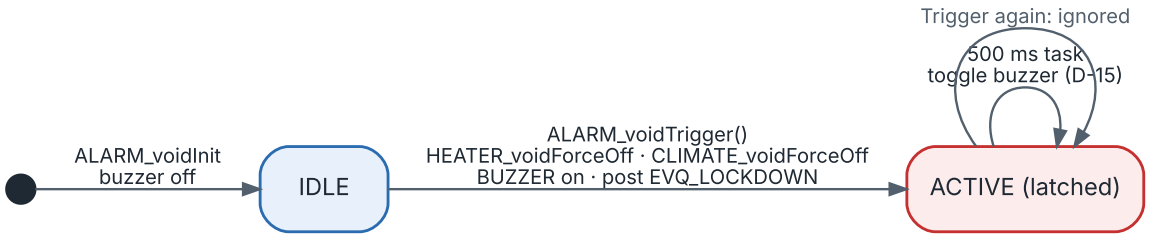


Figure 5.2 ALARM: once active, only an MCU reset leaves the state. drawn from the source code

`ALARM_voidTrigger()` brings the system to its safe state (decision 2): `HEATER_voidForceOff()` (both elements off, display blank, panel LOCKED), `CLIMATE_voidForceOff()` (fan off, FORCED_OFF), buzzer on, post `EVQ_LOCKDOWN`. Lamps, dimmer and door keep their state. The 500 ms task toggles the buzzer, 0.5 s on / 0.5 s off (`ALARM_BUZZER_BEEP 1`; 0 makes it continuous, D-15). The lockdown state is not stored in EEPROM, so a reset always clears it (SEC-05, ALM-01). Both UIs poll `ALARM_u8IsActive()` and switch to their own LOCKED state, which ignores all input.

Call order: ALARM calls HEATER and CLIMATE, and SEC calls ALARM, so the modules were built and tested bottom-up: LIGHT → DOOR → CLIMATE → HEATER → ALARM → SEC.

5.3 LIGHT — lamps and dimmer APP N

No modes, only data: a 5-bit lamp mask and the dimmer percentage; both start at 0 at boot (lamp and dimmer states are not stored, D-14).

- **`LIGHT_u8ToggleLamp(n)`** (n = 1…5) flips the bit and calls `LAMP_u8SetState`. Only if the chip acknowledged does it post `EVQ_LAMP(n)`; on an I²C error the bit is flipped back, so LIGHT, LAMP and the chip stay consistent.
- **`LIGHT_voidSetDimmer(p)`** clamps p to 100, rounds down to a multiple of 10 (`LIGHT_DIMMER_STEP`), sets the PWM, and posts `EVQ_DIMMER` only if the value changed.

`src/APP/LIGHT/LIGHT.c` · lines 35–44 · LIGHT_u8ToggleLamp

```
35          TOG_BIT(Global_u8LampMask,(Copy_u8LampNumber - 1));
36          Local_u8State = GET_BIT(Global_u8LampMask,(Copy_u8LampNumber - 1));
37          if(LAMP_u8SetState(Copy_u8LampNumber,Local_u8State) == 0){
38              EVQ_voidPost(EVQ_LAMP,Copy_u8LampNumber);
39          }else{
40              /* I2C error : the lamp did not change , so the bit goes back and nothing is reported */
41              TOG_BIT(Global_u8LampMask,(Copy_u8LampNumber - 1));
42              Local_u8State = GET_BIT(Global_u8LampMask,(Copy_u8LampNumber - 1));
43          }
44      }else{
```

5.4 DOOR — servo door, admin only APP N

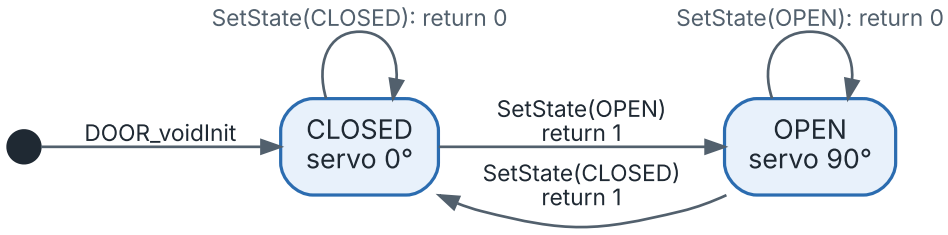


Figure 5.3 DOOR: two states; `SetState` returns whether the servo moved. drawn from the source code

Closed = `DOOR_CLOSED_ANGLE` 0° (1.0 ms pulse), open = `DOOR_OPEN_ANGLE` 90° (1.5 ms). Only UIREM calls DOOR, and only for the admin (commands 7 and 8). Because nothing else can move the door, DOOR posts no event. Boot = closed (D-14).

5.5 CLIMATE — air conditioning with hysteresis APP N

PURPOSE

Keep the room between 21 and 28 °C with an on/off fan (AC-01, AC-02, AC-03).

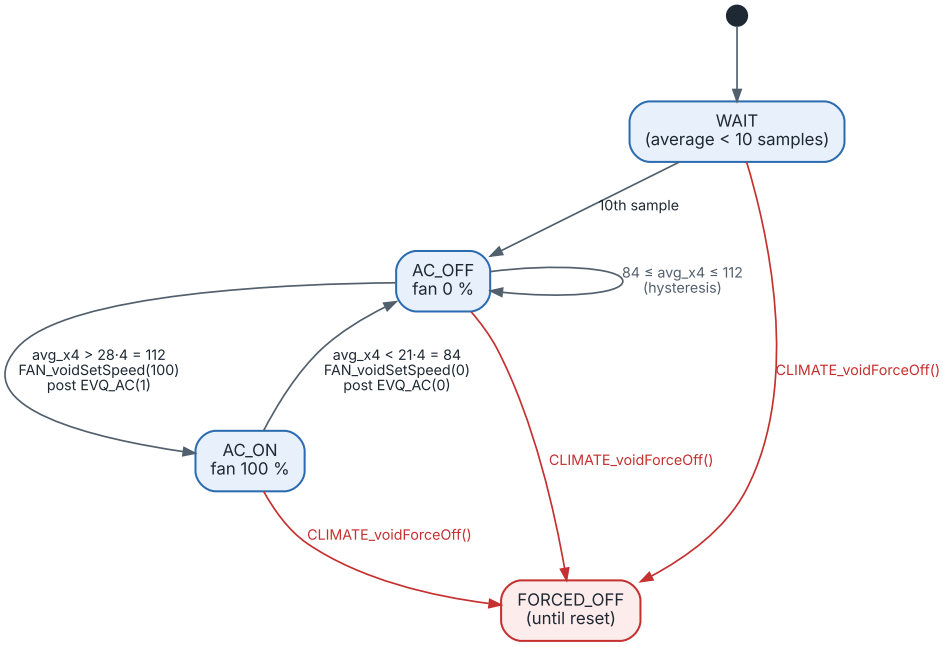


Figure 5.4 CLIMATE state machine. drawn from the source code

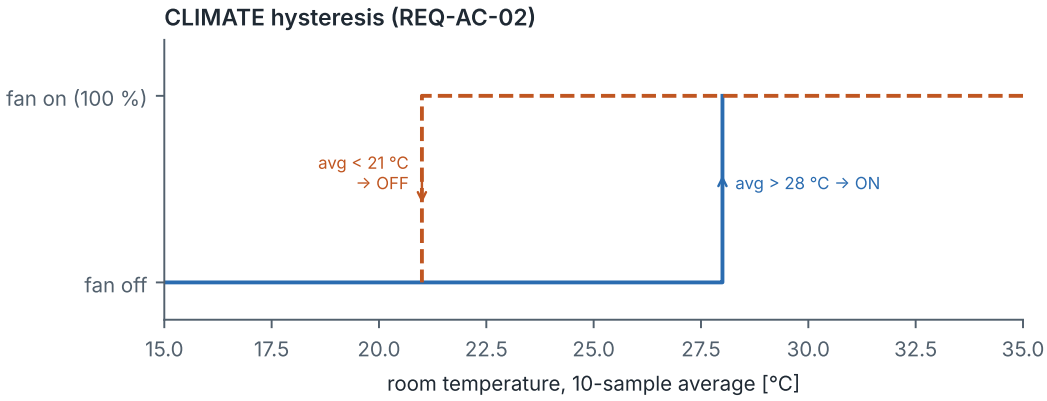


Figure 5.5 Hysteresis: between 21 and 28 °C the fan keeps its state. computed from the code and F_CPU

ALGORITHM

Every 100 ms: one ambient sample into a `MAVG_t`, in every state, so the UIs can always show the room temperature. In WAIT nothing is decided until 10 samples exist (1 s after boot). Then, in quarter degrees: AC_OFF → AC_ON when avg_x4 > 28 × 4 = 112 (strictly above 28 °C); AC_ON → AC_OFF when avg_x4 < 21 × 4 = 84. Each change sets the fan (`CLIMATE_FAN_SPEED` = 100 % or 0) and posts `EVQ_AC`. FORCED_OFF (lockdown) stays until reset. The 7-degree band prevents the fan from chattering around a single threshold.

src/APP/CLIMATE/CLIMATE.c · lines 49–63 · decision part of CLIMATE_voidFeedSample

```
49      case CLIMATE_STATE_AC_OFF:
50          /* REQ-AC-02 : higher than 28 C -> on */
51          if(Local_u16AvgX4 > CLIMATE_ON_ABOVE_X4){
52              FAN_voidSetSpeed(CLIMATE_FAN_SPEED);       /* REQ-AC-03 : fixed speed (ramp not built , D-16)
*/
53              Global_u8State = CLIMATE_STATE_AC_ON ;
54              EVQ_voidPost(EVQ_AC,1);
55          }
56          break ;
57      case CLIMATE_STATE_AC_ON:
58          /* REQ-AC-02 : lower than 21 C -> off ; 21..28 keeps the state (hysteresis) */
59          if(Local_u16AvgX4 < CLIMATE_OFF_BELOW_X4){
60              FAN_voidSetSpeed(0);
61              Global_u8State = CLIMATE_STATE_AC_OFF ;
62              EVQ_voidPost(EVQ_AC,0);
63          }
```

INTEGRATION

`CLIMATE_voidInit()` after FAN and LM35; `CLIMATE_voidTask100ms()` in the 100 ms slot. `CLIMATE_voidFeedSample` is public so that `test_app` can inject temperatures. Getters: `CLIMATE_u8IsAcOn`, `CLIMATE_u8IsTempValid` (0 during the first second), `CLIMATE_u8GetRoomTemp` (rounded °C).

Measured in simavr: ambient 25 → 30 °C at t = 22 s; `[INFO] AC is now ON` completed 699 ms later. The 7th sample after the step lifts the average above 28 °C: (7 × 30 + 3 × 25) / 10 = 28.5.

5.6 HEATER — the water heater APP N

PURPOSE

The complete second course project as one module: panel buttons, set temperature with setting mode and timeout, 100 ms sampling with a 10-sample average, a ±5 °C two-element control law, a blinking display and a blinking heater LED, EEPROM persistence, and the hooks for the terminal and the keypad (HTR-01 … HTR-14).

TWO STATE MACHINES

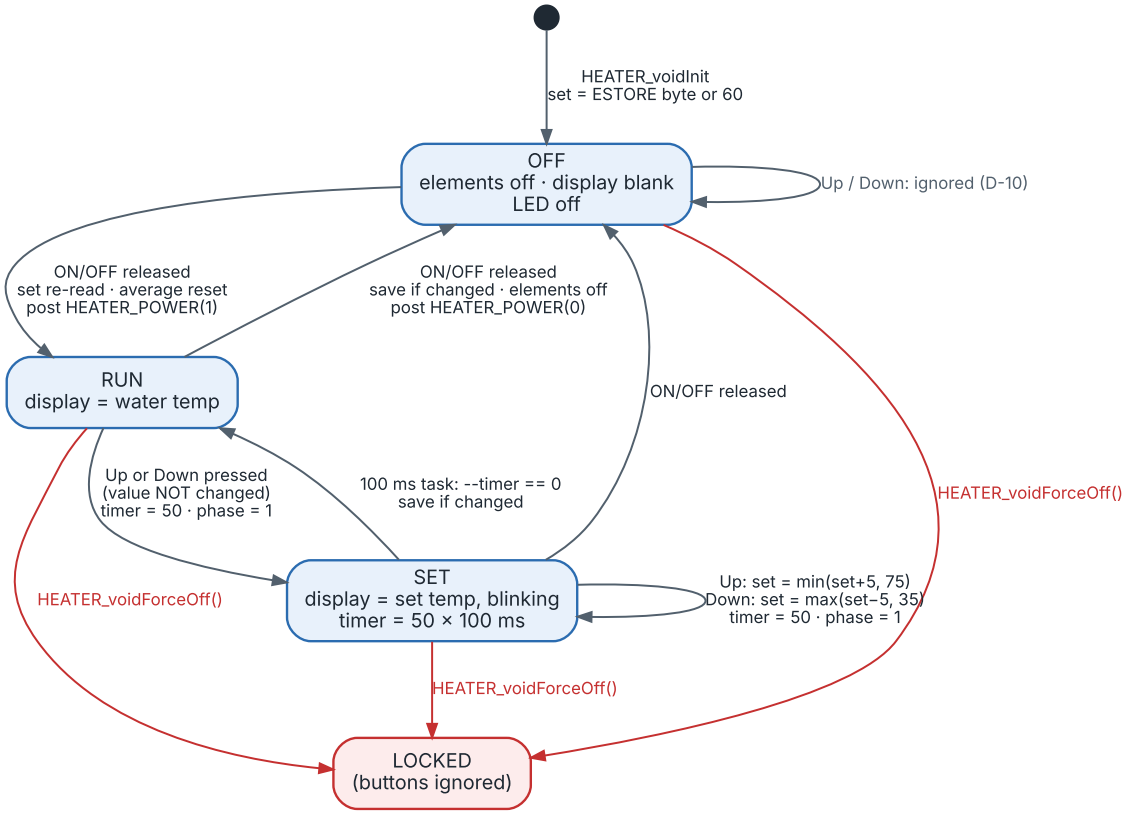


Figure 5.6 Panel states (HEATER.c). drawn from the source code

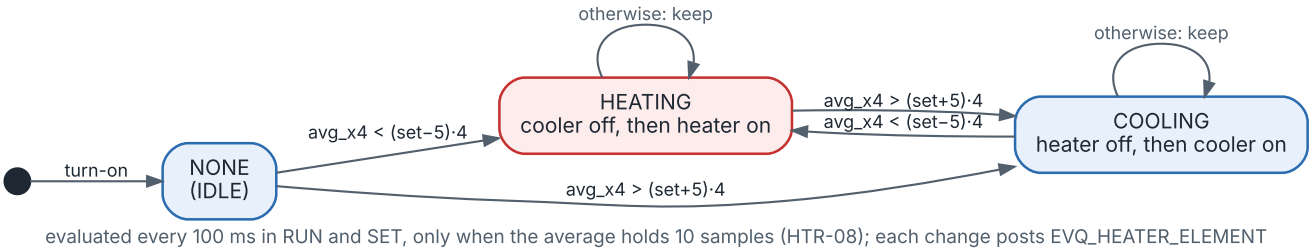


Figure 5.7 Element control, evaluated only while ON and only with a full average. drawn from the source code

Panel transitions as built (HEATER_voidTask10ms, HEATER_voidTask100ms, HEATER_u8SetSetTemp, HEATER_voidForceOff)

State	Event	Action	Next
OFF	ON/OFF **released**	set = stored value (or 60 if invalid); reset the water average; post HEATER_POWER(1)	RUN
OFF	Up / Down	ignored (events are read and discarded, D-10)	OFF
RUN	Up or Down pressed	**value not changed** (HTR-01); timer = 50; blink phase = 1	SET
SET	Up / Down pressed	set ± 5, clamped 35…75 (HTR-02, 03); timer = 50; phase = 1	SET
SET	100 ms task, `--timer == 0`	save if changed (write byte 0x10, post HEATER_SET)	RUN

State	Event	Action	Next
RUN / SET	ON/OFF released	save if changed; post HEATER_POWER(0); both elements off	OFF
any	`HEATER_u8SetSetTemp(t)`	if 35 ≤ t ≤ 75 and t % 5 == 0: set = t, save now; in SET restart the 5 s	same
any	`HEATER_voidForceOff()`	as ON/OFF released	LOCKED

CONTROL LAW (HTR-07 … HTR-09)

The water LM35 is sampled every 100 ms in every state, so the UIs can show the water temperature while the heater is off. Decisions are made only in RUN/SET and only when `MAVG_u8IsFull`: after turn-on the average restarts (decision 5), so the first decision comes after 10 samples, about 1 s (HTR-08). All comparisons are in quarter degrees, so no division is needed:

`src/APP/HEATER/HEATER.c` · lines 258–267

```
258         if(Local_u16AvgX4 < ((u16)(Global_u8SetTemp - HEATER_BAND) * LM35_STEPS_PER_C)){
259             Local_u8Element = HEATER_ELEMENT_HEATING ;
260         }else if(Local_u16AvgX4 > ((u16)(Global_u8SetTemp + HEATER_BAND) * LM35_STEPS_PER_C)){
261             Local_u8Element = HEATER_ELEMENT_COOLING ;
262         }else{
263         }
264         if(Local_u8Element != Global_u8Element){
265             HEATER_voidSetElement(Local_u8Element);
266             EVQ_voidPost(EVQ_HEATER_ELEMENT,Local_u8Element);      /* uart_protocol.md 9 */
267         }
```

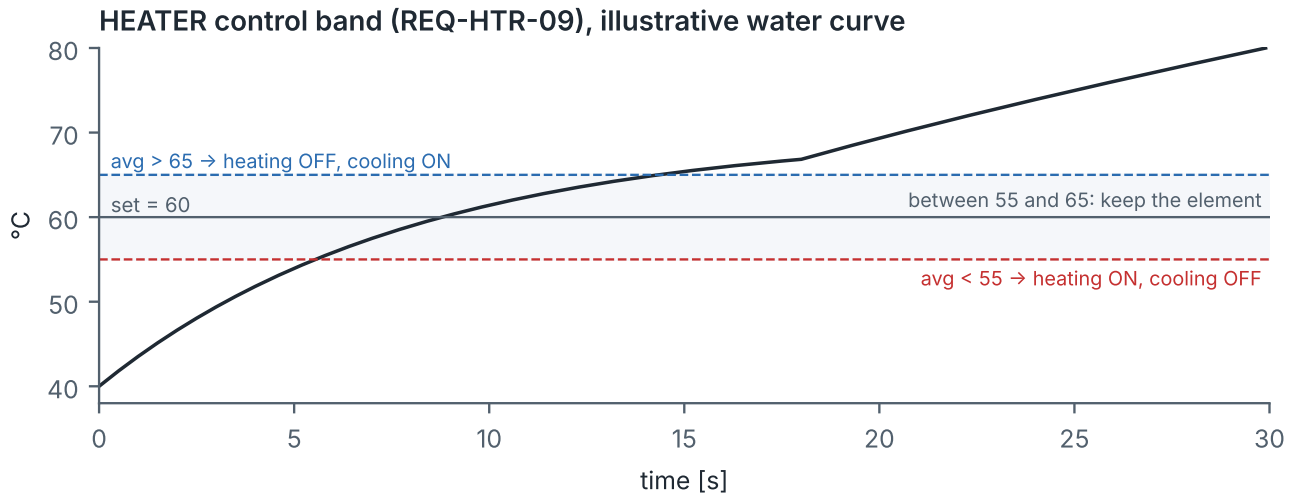


Figure 5.8 The ±5 °C band around the set temperature. computed from the code and F_CPU

`HEATER_voidSetElement` always switches one element off before switching the other on, so heating and cooling are never on together, not even for one instruction.

OUTPUTS (HTR-06, 10, 11, 13)

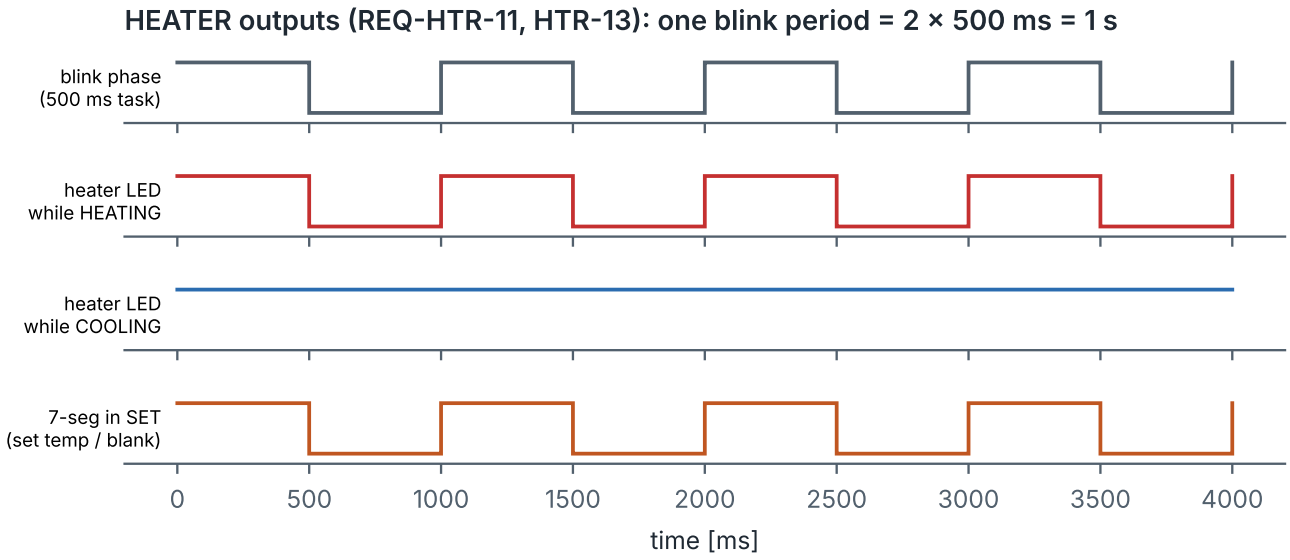


Figure 5.9 Blink phase, heater LED and display in setting mode. computed from the code and F_CPU

Panel state	7-segment	Heater LED
OFF, LOCKED	blank	off
RUN	water temperature `(avg_x4 + 2) / 4`, limited to 99	HEATING: follows the 500 ms phase (1 s period) · COOLING: steady on · NONE: off
SET	set temperature while phase = 1, blank while 0	

The outputs are refreshed by the 100 ms task and by the 500 ms task (which toggles the phase); `HEATER_voidShow` writes the display only when the shown number or the blank state changes, so a stable value costs no I²C traffic.

> **TWO TIMING EFFECTS OF THIS DESIGN (MEASURED IN SIMAVR, SEE §7.5)**
>
> - On turn-off the elements go off at once, but the display and the LED follow in the next 100 ms or 500 ms task, i.e. up to 100 ms later. architecture.md §5.6 describes all four as immediate.
> - After an Up/Down press the set value appears at the next 100 ms task. If that task falls 2 ms before a 500 ms phase toggle, the value is blanked again 2 ms later and reappears at the next "on" phase, up to 0.5 s later. The comment in `HEATER_voidRestartSetting` ("the value is visible at once") describes the intent; about one press in five hits this window.

PERSISTENCE (HTR-04, DECISION 6)

The working copy lives in HEATER; the stored copy is byte 0x10 of the ESTORE image. Saving happens when setting mode times out, on turn-off, and at once for a remote or keypad change — in each case only if the value differs from the stored byte, so an unchanged value costs no EEPROM write and no event. At boot an invalid stored byte (not 35…75 in steps of 5) gives 60; that 60 is written only at the next save (eeprom_map.md §4 says "stores it", which suggests at once).

INTEGRATION

`HEATER_voidInit()` after ESTORE, RELAY, LED, SEVEN_SEG, BUTTON and LM35. In the loop: `HEATER_voidTask10ms()` after `BUTTON_voidUpdate()` in the same slot, so a button event is handled in the pass that detects it; `HEATER_voidTask100ms()`; `HEATER_voidTask500ms()`. Getters for the UIs: `HEATER_u8IsOn`, `HEATER_u8GetElement`, `HEATER_u8GetWaterTemp`, `HEATER_u8GetSetTemp`.

5.7 UILOC — keypad and LCD APP N

PURPOSE

The local interface for keypad users (LUI-01 ... 04): login with numeric ID and PIN, a menu for lamps, dimmer, AC and heater, and a status screen when nobody is using the keypad.

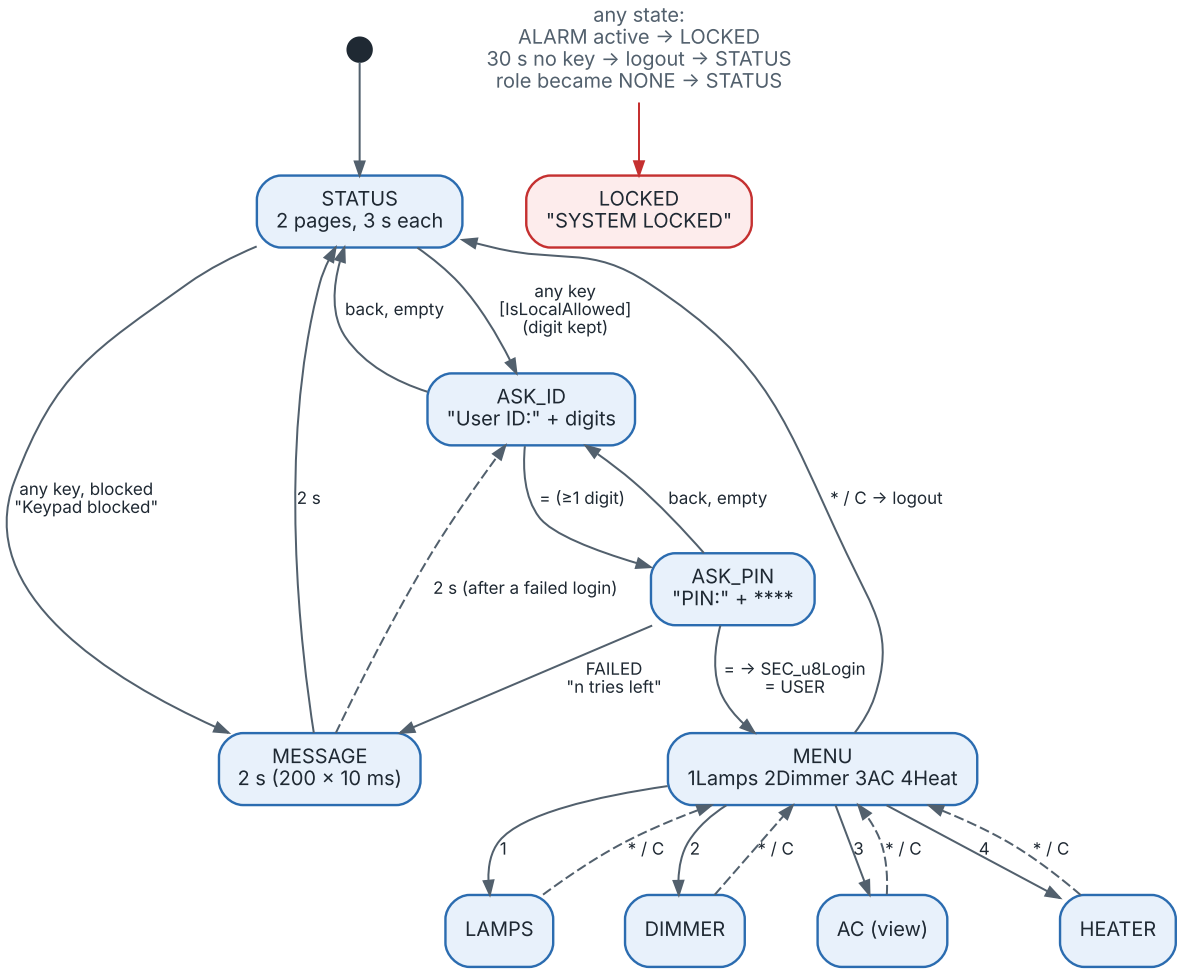


Figure 5.10 UILOC state machine. Keys: digits, `=` Enter, `*` or `C` back, `+` / `-` step (D-7). drawn from the source code

HOW A PASS WORKS

`UILOC_voidTask10ms` always takes the key from KPAD first, so a key pressed while it means nothing is not kept for later. It then checks, in this order:

1. LOCKED → return.
2. Alarm active → LOCKED screen.
3. MESSAGE → count down 2 s.
4. Logged-in screen but SEC says role NONE (the admin logged in or blocked the keypad) → STATUS.
5. ID/PIN being typed but the keypad is no longer allowed → "Keypad blocked". Without this check, SEC would refuse the login and count a failure.
6. Otherwise the key goes to the state's handler.

Every handler only changes RAM and calls `UILOC_voidDraw()`, which rewrites the whole wanted screen in `LCD_BUF`. LCD_BUF then sends only the cells that changed, so a redraw neither blocks nor flickers. `UILOC_voidTask1s` refreshes the values every second, flips the status page every 3 s, and ends a session (or a half-typed login) after 30 s without a key.

Screens (16 × 2) as built

Screen	Row 1	Row 2
Status page 1	`Lamps:1-3-- D70%`	`AC:ON Room:29C`

Screen	Row 1	Row 2
Status page 2	`Heater:HEATING` / OFF / COOLING / IDLE	`Water:55C Set:60`
Login	`User ID:1111`	`PIN:****` (LUI-04)
Menu	`1Lamps 2Dimmer`	`3AC 4Heat C=Exit` (no door, SEC-06)
Lamps	`Lamp 1 2 3 4 5`	`* - * - -`
Dimmer / AC / Heater	`Dimmer: 70%` · `AC:ON Room:29C` · `Water:55C HEAT`	`+ / - C=Back` · `on>28 off<21` · `Set:60C + / -`
Messages	`Wrong ID or PIN` · `Keypad blocked` · `SYSTEM LOCKED`	`2 tries left` · `by admin` · `Reset required`

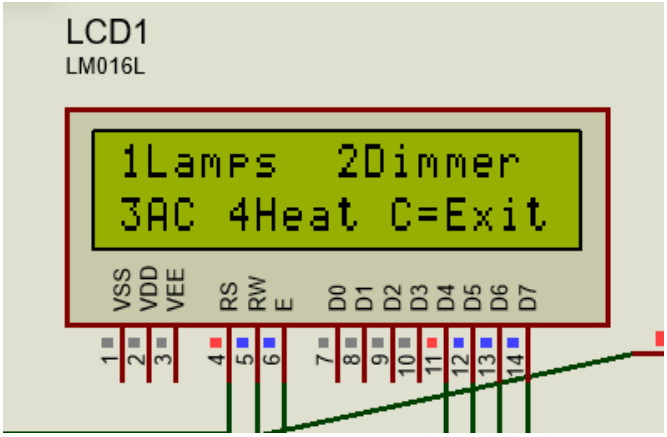


Figure 5.11 LCD menu in Proteus (presentation capture).
measured in Proteus (repository capture)

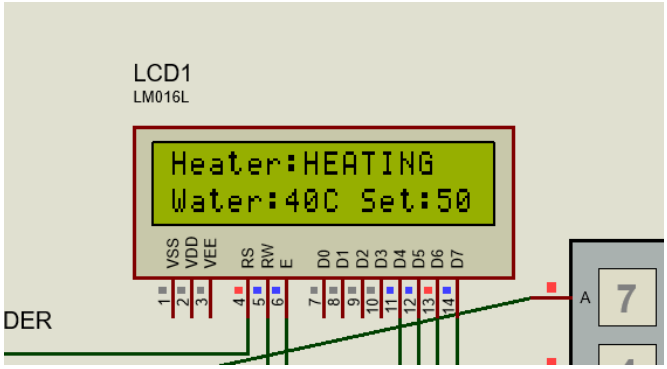


Figure 5.12 Status page 2 in Proteus.
measured in Proteus (repository capture)

INTEGRATION

`UILOC_voidInit()` after LCD_BUF and all features. `UILOC_voidTask10ms()` right after `KPAD_voidUpdate()` in the 10 ms slot; `UILOC_voidTask1s()` in the 1 s slot. `LCD_BUF_voidUpdate()` runs in the 5 ms slot.

5.8 UIREM — the UART terminal APP N

PURPOSE

The remote interface (RUI-01 … 04): banner and login, role-specific menus, 17 commands with an argument either on the same line or prompted, `[OK]` / `[ERR]` replies, and `[INFO]` lines for every change made elsewhere. The full command list is Appendix D.

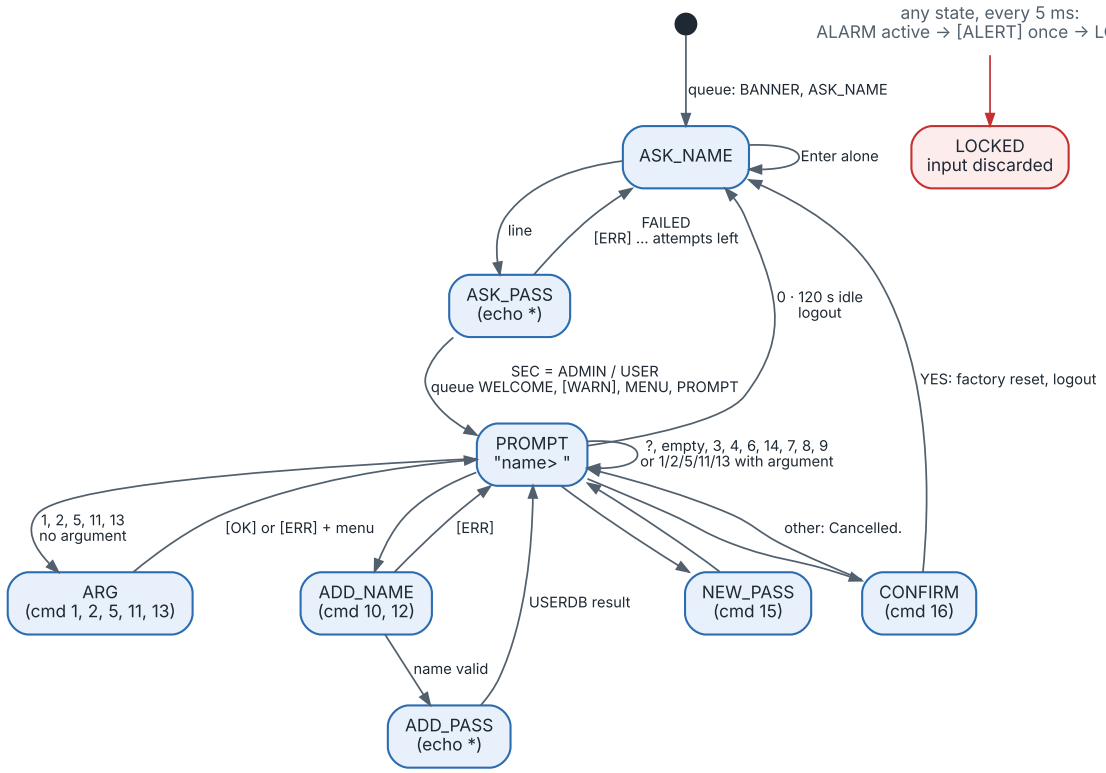


Figure 5.13 UIREM input states. Output is decoupled through a print queue. drawn from the source code

THE OUTPUT RULE: NEVER BLOCK ON A 128-BYTE RING

The admin menu is 408 bytes; the TX ring holds 128. UIREM therefore never prints a reply in one go. Every reply is a chain of **messages** in a 6-entry print queue (for example error line → menu → prompt). Multi-line messages (menu, status block, user list) print **one line per step**.

`src/APP/UIREM/UIREM.c` · lines 887–900 · UIREM_voidTask5ms: at most one print step per call

```
887     if(TERM_u8TxFree() < UIREM_TX_RESERVE){
888         return ;
889     }
890     /*3. at most ONE print step per call */
891     if(Global_u8QueueCount > 0){
892         if(UIREM_u8PrintStep(Global_u8Queue[0],Global_u8Step) == 1){
893             Global_u8QueueCount--;
894             for(i = 0 ; i < Global_u8QueueCount ; i++){
895                 Global_u8Queue[i] = Global_u8Queue[i + 1] ;
896             }
897             Global_u8Step = 0 ;
898         }else{
899             Global_u8Step++;
900         }
```

The task does nothing while fewer than `UIREM_TX_RESERVE` = 64 bytes are free, and it reads typed input only when the queue is empty. The result: no output is ever dropped, the echo of a typed line always fits, and typed bytes simply wait in the RX ring. The cost per 5 ms call is bounded by one line of at most 64 bytes copied into the ring.

EVENTS, MUTE AND IDLE

- An `[INFO]` event is printed only at the prompt and only when nothing has been typed yet (`TERM_u8IsLineEmpty`), so it never cuts into a half-typed line. It is followed by the prompt again.

- While nobody is logged in, events are read and discarded. The lockdown alert is printed from `ALARM_u8IsActive()` whoever caused it, even with nobody logged in.
- EVQ is muted around commands 1, 2, 5 and 16, whose result is answered with `[OK]`. It is not muted around login, logout and command 9, so the admin sees `[INFO] Keypad user logged out` when the admin's own action ended a keypad session.
- 120 s without a received byte ends the session (D-11), so an admin who walks away does not block the keypad for ever.

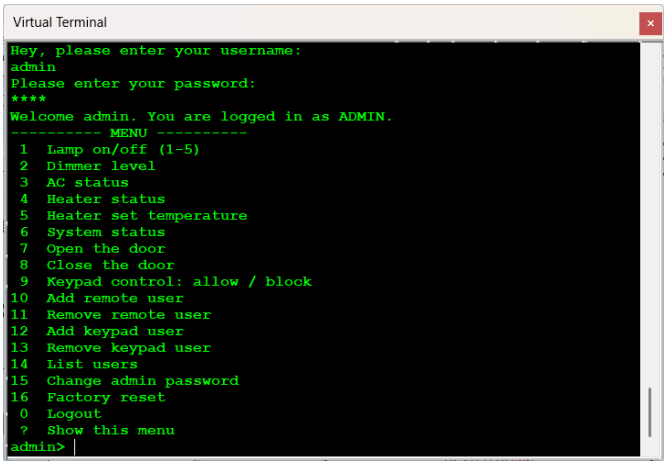


Figure 5.14 Login and admin menu in the Proteus terminal.
measured in Proteus (repository capture)

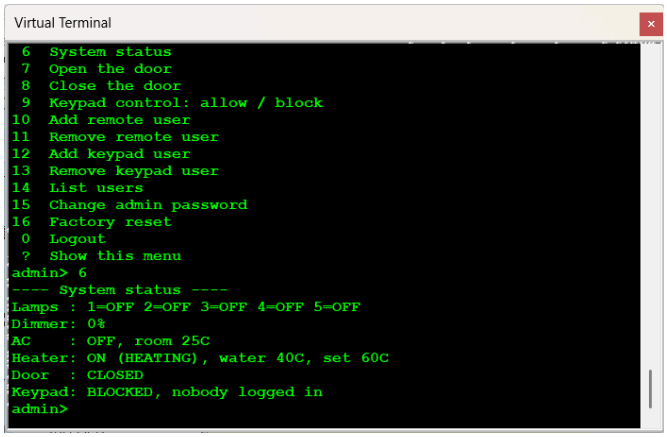


Figure 5.15 Command 6, system status.
measured in Proteus (repository capture)

INTEGRATION

`UIREM_voidInit()` last among the APP inits; it queues the banner. `UIREM_voidTask5ms()` in the 5 ms slot (after `LCD_BUF_voidUpdate`), `UIREM_voidTask1s()` in the 1 s slot. TERM must be initialised first, at boot step 2.

The simavr run reproduced the documented session byte for byte: login, adding users, `1 3`, `2 70`, `5 65`, door, status, events and the lockdown alert (the transcript is kept in `guide_src/data`).

CHAPTER 6

The top-level application

`src/APP/main.c` is about 100 lines. It initialises everything in the order of §1.8, enables interrupts, and runs a super-loop that serves the scheduler flags. This chapter shows how the pieces of chapters 2–5 work together, and follows three real interactions through every layer, with times taken from the simavr run.

6.1 The super-loop

`src/APP/main.c` · lines 80–94 · first three slots of the loop

```
 80         if(SCHED_u8IsTaskDue(SCHED_TASK_5MS)){
 81             LCD_BUF_voidUpdate();
 82             UIREM_voidTask5ms();
 83         }
 84         /* 10 ms : keypad and button debounce , heater buttons , keys , EEPROM write-behind (REQ-HTR-01 ,
REQ-EEP-03) */
 85         if(SCHED_u8IsTaskDue(SCHED_TASK_10MS)){
 86             KPAD_voidUpdate();
 87             BUTTON_voidUpdate();
 88             HEATER_voidTask10ms();
 89             UILOC_voidTask10ms();
 90             ESTORE_voidUpdate();
 91         }
 92         /* 100 ms : water and room temperature , heater control law , AC hysteresis (REQ-HTR-07 , REQ-AC-
02) */
 93         if(SCHED_u8IsTaskDue(SCHED_TASK_100MS)){
 94             HEATER_voidTask100ms();
```

The 500 ms slot calls `HEATER_voidTask500ms()` and `ALARM_voidTask500ms()`; the 1 s slot calls `UILOC_voidTask1s()` and `UIREM_voidTask1s()`. There is no dispatch table and no function-pointer list: one explicit `if` per period keeps the order visible and lets the compiler inline everything (with LTO, all task functions end up inside `main`).

Slot	First due	Calls, in order	Why this order
5 ms	tick 5	LCD_BUF_voidUpdate · UIREM_voidTask5ms	one LCD byte, then terminal input and output
10 ms	tick 12	KPAD_voidUpdate · BUTTON_voidUpdate · HEATER_voidTask10ms · UILOC_voidTask10ms · ESTORE_voidUpdate	inputs first, so HEATER and UILOC see a new event in the same pass; ESTORE last, so a value saved by a UI in this pass may go out at once
100 ms	tick 101	HEATER_voidTask100ms · CLIMATE_voidTask100ms	two ADC samples, control laws, display
500 ms	tick 503	HEATER_voidTask500ms · ALARM_voidTask500ms	blink phase, buzzer beat
1 s	tick 1004	UILOC_voidTask1s · UIREM_voidTask1s	idle timeouts, status page refresh

When no flag is set, the loop simply polls the five flags again. That is the idle time: 93–96 % of the CPU in simavr (§7.4). There is no sleep mode; adding `SLEEP` with the Timer2 interrupt as the wake-up source would be a small change if power ever mattered.

6.2 Data ownership and event flow

Rule: every piece of state has exactly one owner, as `static` variables in the owner's `.c`; everyone else goes through the owner's functions. There are no shared global variables between modules.

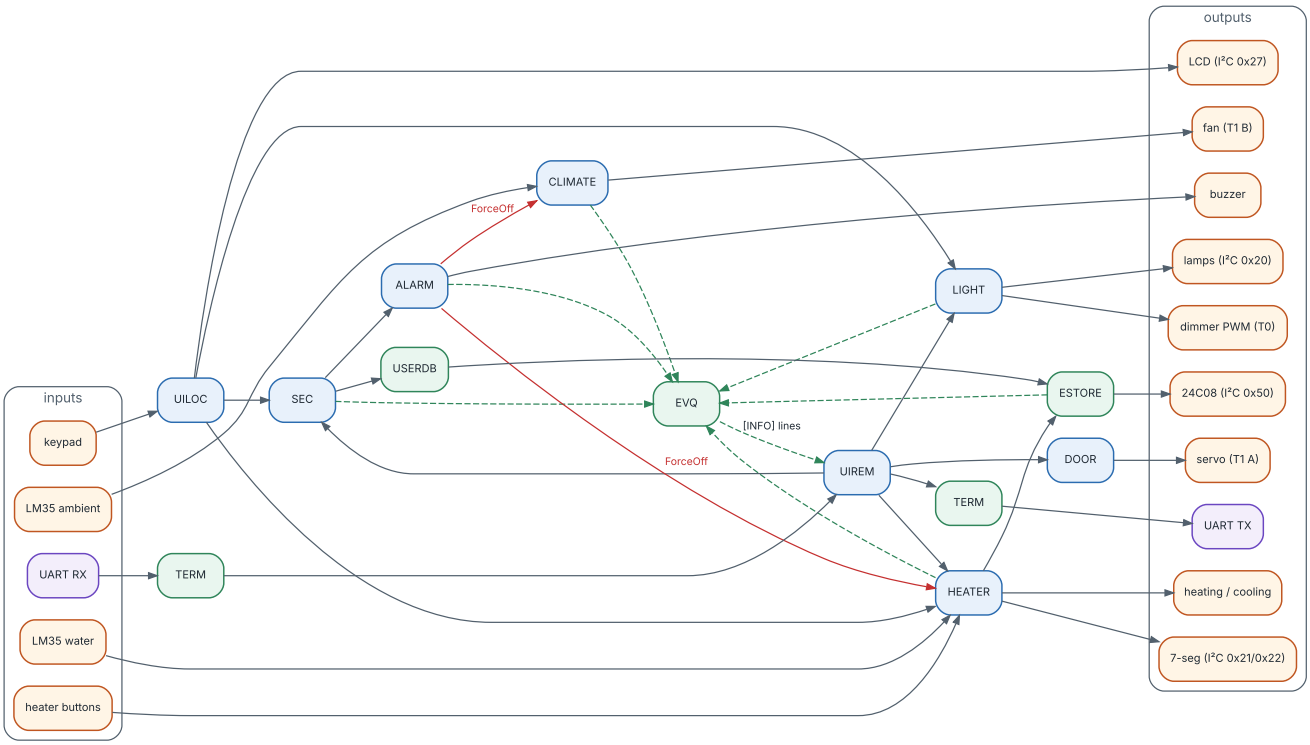


Figure 6.1 Who drives whom. Solid: function calls. Dashed green: events posted to EVQ and printed by UIREM. drawn from the source code

State	Owner	Changed by (owner's API)	Read by
Lamp mask, dimmer %	LIGHT	UIREM, UILOC	UIREM, UILOC
Door state	DOOR	UIREM (admin)	UIREM
Room average, AC on/off	CLIMATE	itself (100 ms), ALARM (force off)	UIREM, UILOC
Water average, panel state, element	HEATER	itself (buttons, control law), ALARM	UIREM, UILOC
Set temperature	HEATER (working) + ESTORE (stored)	panel, UIREM, UILOC	UIREM, UILOC
Sessions, fail counters, keypad-allowed	SEC	UIREM, UILOC	UIREM, UILOC
Lockdown flag	ALARM	SEC	UIREM, UILOC, SEC
Accounts	USERDB (inside the ESTORE image)	UIREM (admin, gate open)	SEC, UIREM
EEPROM image, dirty pages, fault	ESTORE	USERDB, HEATER	USERDB, HEATER, UIREM
Pending events	EVQ	LIGHT, CLIMATE, HEATER, SEC, ALARM, ESTORE	UIREM
Tick, task flags	SCHED	Timer2 ISR	main
LCD text	LCD_BUF	UILOC	— (sent to the LCD)

6.3 Scenario 1 — remote login, set temperature, EEPROM save

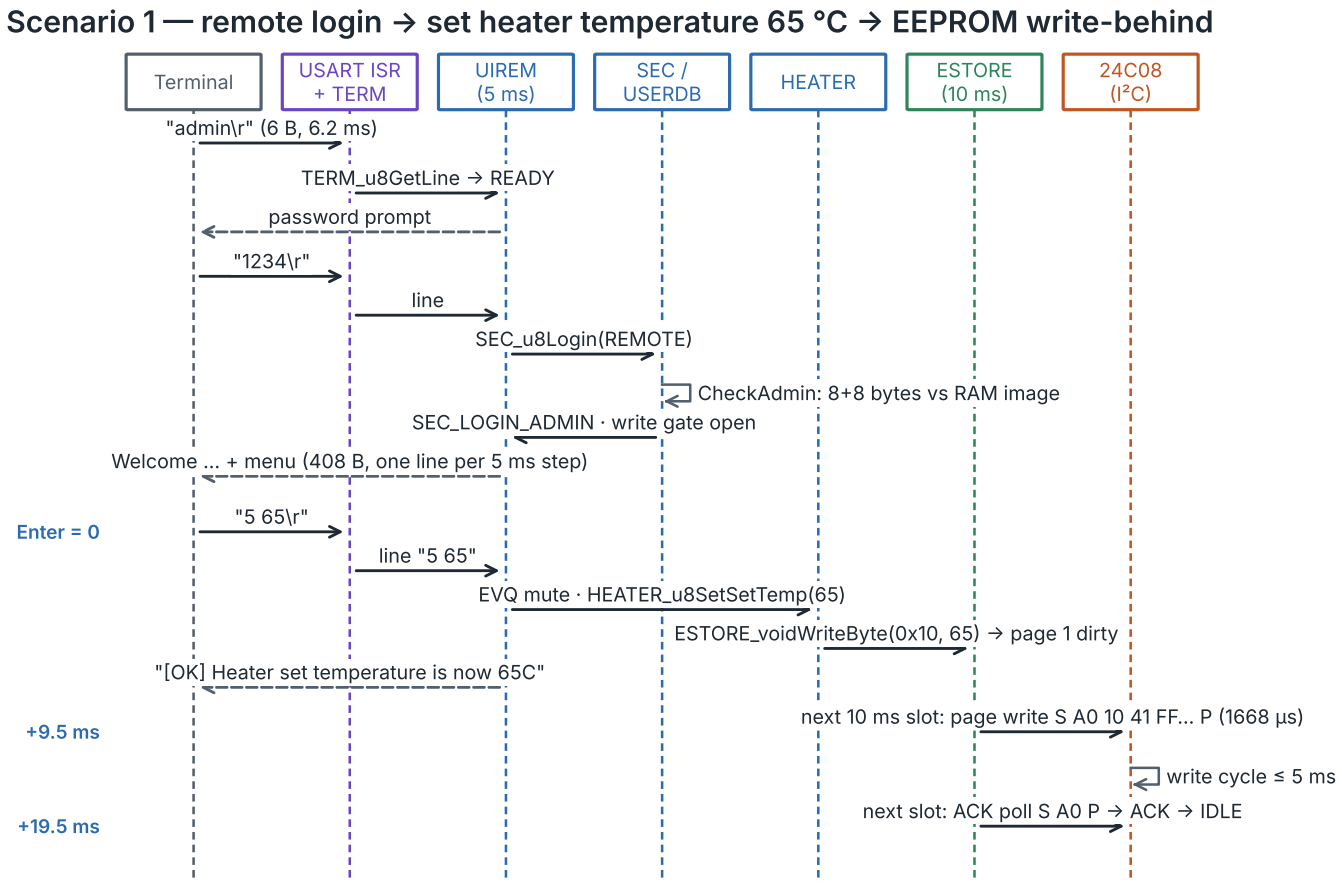


Figure 6.2 Scenario 1 through the layers. Times are from the simavr main run (admin logged in at 1.1 s, `5 65` sent at 7.0 s).
measured in simavr (cycle-accurate simulation of app.elf)

#	What happens	Where
1	Each received byte raises RXC; the ISR puts it in the RX ring (7.6 µs per byte, 122 cycles).	USART ISR → TERM
2	Within 5 ms, `TERM_u8GetLine` takes the bytes, echoes them (`*` for the password) and returns the line on CR.	UIREM 5 ms
3	`SEC_u8Login(REMOTE, "admin", "1234")` compares both 8-byte fields with the RAM image. Admin → write gate open, local allowed = 0, keypad session ended.	SEC → USERDB → ESTORE (RAM)
4	Welcome, menu (one line per 5 ms step) and prompt are queued; the 408 menu bytes leave in ≈ 0.42 s.	UIREM → TERM → UDRE ISR
5	`5 65⏎` → `UIREM_voidCommand` splits command and argument → `EVQ_voidSetMute(1)` → `HEATER_u8SetSetTemp(65)` → `ESTORE_voidWriteByte(0x10, 65)` sets dirty bit 1 → mute off → `[OK] Heater set temperature is now 65C`.	UIREM, HEATER, ESTORE
6	Next 10 ms slot: page 1 written, `S A0 10 41 FF … P`, 1668 µs on the bus, 9.5 ms after the Enter was received (simavr).	ESTORE 10 ms → EXT_EEPROM → TWI
7	Next slot: one ACK poll, chip ready (its 5 ms cycle is over) → IDLE. Saved 19.5 ms after the Enter.	ESTORE

6.4 Scenario 2 — a keypad user toggles a lamp

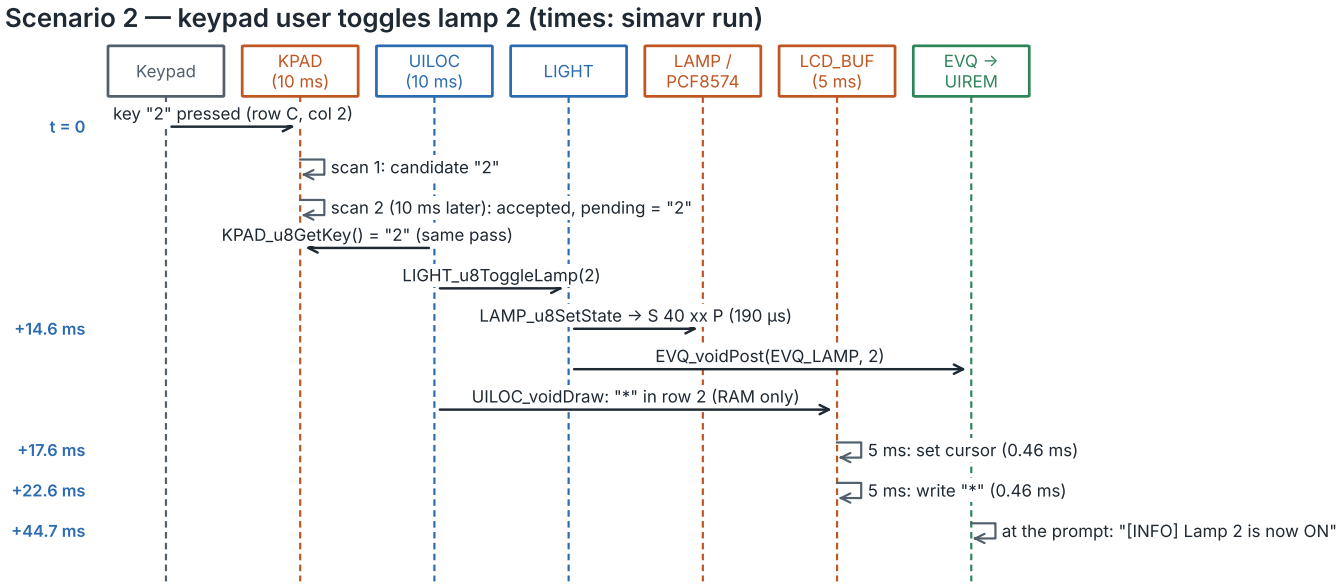


Figure 6.3 Scenario 2 (simavr main run, key "2" pressed at 12.5 s while the lamp screen is open and the admin is at the prompt). measured in simavr (cycle-accurate simulation of app.elf)

t after key-down	Event
0 → 10 ms	first scan sees "2" (candidate)
10 → 20 ms	second equal scan → accepted; UILOC takes it in the same pass
14.6 ms	lamp chip written (`S 40 xx P`, 190 µs); EVQ_LAMP posted; screen redrawn in RAM
17.6 ms	LCD_BUF: set cursor to row 2 under lamp 2
22.6 ms	LCD_BUF: `*` written — the change is visible
44.7 ms	the terminal has received `[INFO] Lamp 2 is now ON` (≈ 24 bytes at 1.04 ms each)

6.5 Scenario 3 — the third wrong login

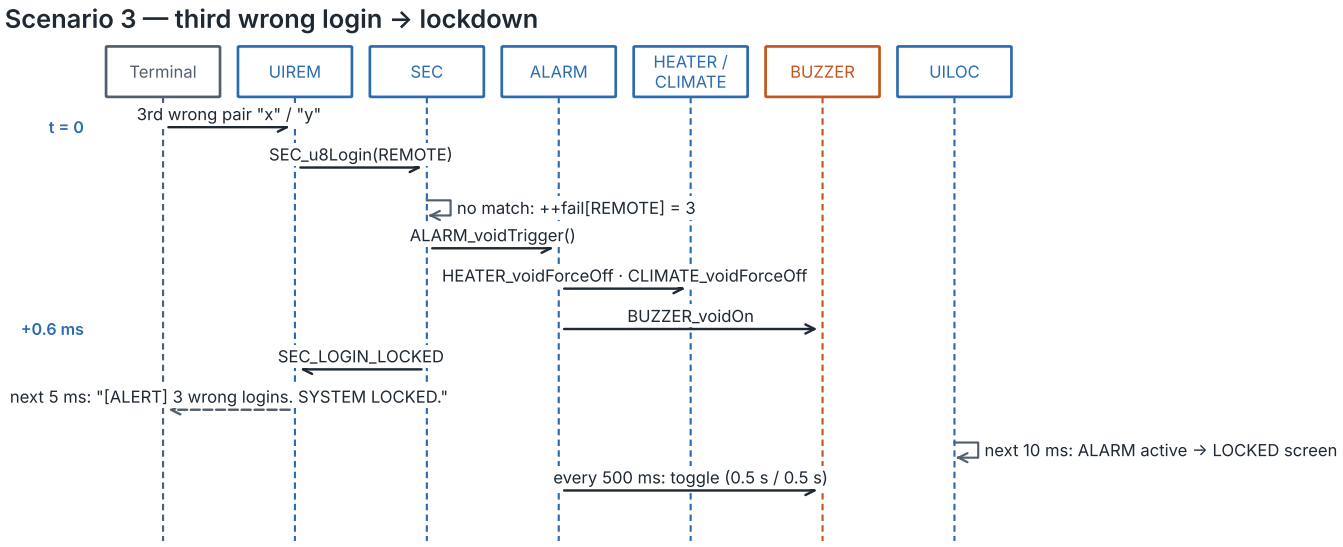


Figure 6.4 Scenario 3 (simavr main run, third wrong pair at 26.9 s). measured in simavr (cycle-accurate simulation of app.elf)

The 3rd failed `SEC_u8Login` calls `ALARM_voidTrigger()` inside the same call: elements and fan off and the buzzer on within 0.6 ms of the Enter (simavr). In the next 5 ms call, UIREM sees `ALARM_u8IsActive()`, empties its print queue, queues the `[ALERT]` line, switches the echo off and enters LOCKED; from then on received bytes are read and

thrown away. In the next 10 ms call, UILOC shows `SYSTEM LOCKED / Reset required` and ignores keys. HEATER (LOCKED) ignores its buttons. The 500 ms task toggles the buzzer (simavr: toggles 500 ms apart). Lamps, dimmer and door are untouched. Only the RESET pin ends this.

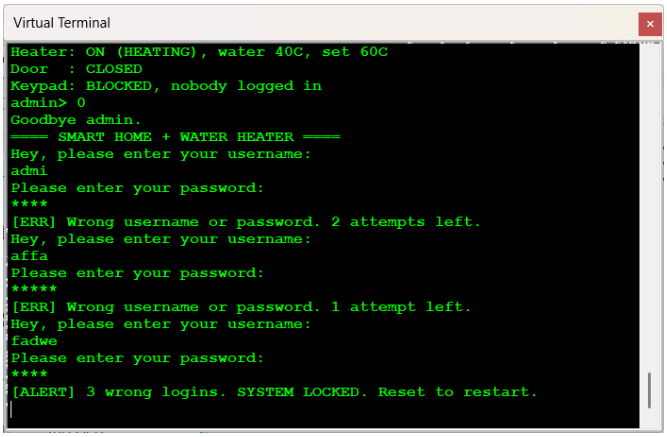


Figure 6.5 The lockdown in the Proteus terminal.
measured in Proteus (repository capture)

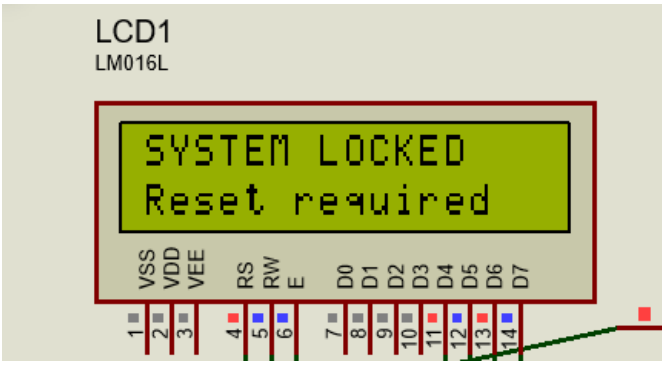


Figure 6.6 The LCD in lockdown.
measured in Proteus (repository capture)

CHAPTER 7

System timing analysis

How long everything takes, how late anything can be, and how much margin is left. Every figure is either **computed** from F_CPU, the configuration constants and the data sheet, or **measured in simavr** on the unchanged application binary. Proteus observations recorded in the repository are quoted where they exist.

7.1 Method

Computed values

Clock-derived values (timer periods, baud rate, ADC and SCL timing) follow from the register settings of chapter 2. They are exact, apart from the crystal tolerance. Bus times are counted in SCL periods: one byte with its ACK = 9 periods = 90 µs; a START is counted as one period. CPU times of ISRs are counted in cycles from the disassembly of an avr-gcc 7.3.0 build made with the PlatformIO flags (`-Os -flto -ffunction-sections -fdata-sections`; this build has the same RAM layout as `pio run`).

simavr measurements

simavr 1.6 is a cycle-accurate AVR simulator. A harness (`guide_src/sim/harness.c`) loads `app.elf` and emulates every external part:

- the four PCF8574 chips and a 24C08 with a 5 ms write cycle (the real chip; the Proteus model has none);
- the keypad matrix, which follows the column the firmware drives low;
- the three buttons, both LM35 inputs, and the terminal (bytes injected one frame time apart).

The harness steps the CPU one instruction at a time and logs, with cycle timestamps:

- every served scheduler slot (from the `SCHED_u8IsTaskDue` call that returns 1 to the next call);
- every ISR (vector entry to the end of `reti`);
- every I²C transaction, the relay/LED/buzzer pins and all UART output.

Three faults in simavr's TWI and ADC models had to be fixed first (the patch is kept in `guide_src/sim/`):

1. writing 1 to TWINT did not clear the flag;
2. SLA+W returned status 0x28 instead of 0x18;
3. writing 1 to ADIF set the flag instead of clearing it, so polled conversions returned stale data.

The TWI bit time was also tied to TWBR: v1.6 counts one SCL period as 1 µs, ten times too fast for 100 kHz.

Two scenarios were run:

- **main** (30 s): first boot on a blank EEPROM, admin login, user management, `1 3`, `2 70`, `5 65`, door, status, keypad login and lamp toggle, heater on, setting mode, water and ambient steps, logout, three wrong logins.
- **stress** (16 s): back-to-back EEPROM writes while the keypad toggles lamps, the heater blinks in setting mode, and the terminal prints status blocks and user lists.

> **LIMITS**
> simavr is not the hardware. Its I²C has ideal edges and no clock stretching; a START is modelled as one SCL period; external parts answer instantly. The cross-check validates the firmware's own timing — code paths, scheduling, bus traffic counted in bytes — against the computed model. The I²C byte durations agree with computation and with the Proteus figure in hal_summary.md (§7.3). A stuck bus was not simulated; its case is computed in §7.7.

7.2 From the clock to the timers

Quantity	Formula (F_CPU = 16 MHz)	Value	Error / note
System tick (Timer2 CTC)	(OCR2 + 1) · 64 / F_CPU, OCR2 = 249	1 000.000 µs	0 ppm (crystal only)

Quantity	Formula (F_CPU = 16 MHz)	Value	Error / note
Dimmer PWM (Timer0)	F_CPU / (8 · 256)	7 812.5 Hz	period 128 µs, duty step 1/256
Servo / fan PWM (Timer1)	F_CPU / (8 · (ICR1 + 1)), ICR1 = 39999	50.000 Hz	0.5 µs resolution
Servo pulse 0° / 90° / 180°	(OCR1A + 1) · 0.5 µs, OCR1A = 2000 / 3000 / 4000	1.0005 / 1.5005 / 2.0005 ms	+0.5 µs from the "+1" of fast PWM
UART baud	F_CPU / (16 · (UBRR + 1)), UBRR = 103	9 615.4	+0.16 % (limit ±2 %)
UART byte time (8N1)	10 bits / baud	1.040 ms	962 bytes/s per direction
ADC clock / conversion	F_CPU / 128; 13 clocks (first: 25)	125 kHz / 104 µs	first conversion 200 µs
I²C SCL	F_CPU / (16 + 2 · TWBR), TWBR = 72	100 kHz	byte + ACK = 90 µs
TWI timeout per wait	2666 passes × 8 cycles	1.33 ms	`TWI_cfg.h` assumes 6 cycles → 1 ms

A Proteus capture of the servo pulse and the dimmer PWM is Figure 2.3. `test_mcal` also counts 1000 Timer2 interrupts against Timer1 over 1 s (test_plan.md §2).

7.3 Task schedule, hyperperiod and I²C budget

Periodic tasks

Period	Due on ticks	Work	Bus / ADC work per call (computed)
1 ms (ISR)	every tick	tick++, 5 down-counters, flags	—
5 ms	5, 10, 15, … (≡ 0 mod 5)	LCD_BUF (≤ 1 LCD byte or cursor command) · UIREM (one input line or one print step)	≤ 460 µs LCD + ≤ 190 µs lamp write from a command
10 ms	12, 22, … (≡ 2 mod 10)	KPAD scan · BUTTON · HEATER buttons · UILOC key · ESTORE step	≤ 190 µs lamp write (keypad) + ≤ 1630 µs page write or 100 µs poll
100 ms	101, 201, … (≡ 1 mod 100)	HEATER sample + control + display · CLIMATE sample + hysteresis	2 × 104 µs ADC + ≤ 2 × 190 µs display
500 ms	503, 1003, … (≡ 3 mod 500)	HEATER blink phase + outputs · ALARM buzzer	≤ 2 × 190 µs display
1 s	1004, 2004, … (≡ 4 mod 1000)	UILOC refresh / idle timer · UIREM idle timer	— (RAM only)

Hyperperiod. The pattern of flags repeats every lcm(5, 10, 100, 500, 1000) = **1000 ms**. In one hyperperiod there are 200 + 100 + 10 + 2 + 1 = **313** slots, and no two of them are released on the same tick (§4.1). The tightest spot is ticks 1001–1005 (mod 1000), where the 100 ms, 10 ms, 500 ms, 1 s and 5 ms flags follow each other one millisecond apart; §7.7 analyses it.

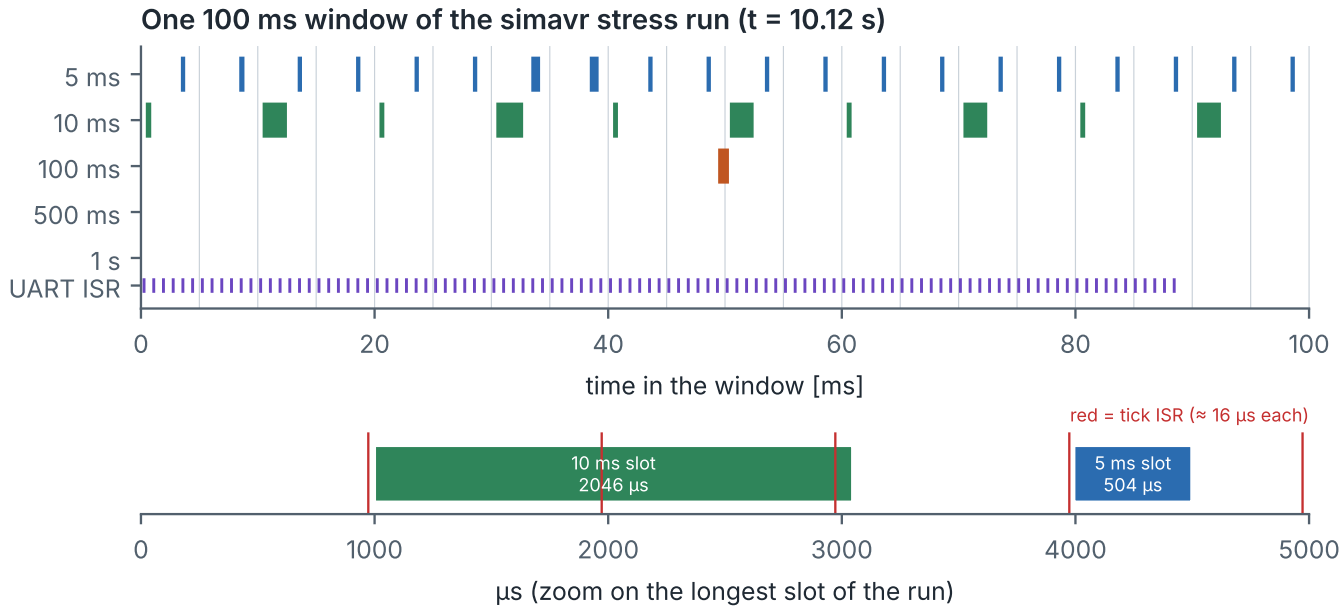


Figure 7.1 One 100 ms window from the stress run. Top: every served slot at its real start and length. Bottom: zoom on the longest slot of the whole run, a 10 ms slot with a keypad lamp toggle and an EEPROM page write, with the tick ISRs (red) that interrupt it.
measured in simavr (cycle-accurate simulation of app.elf)

I²C transactions

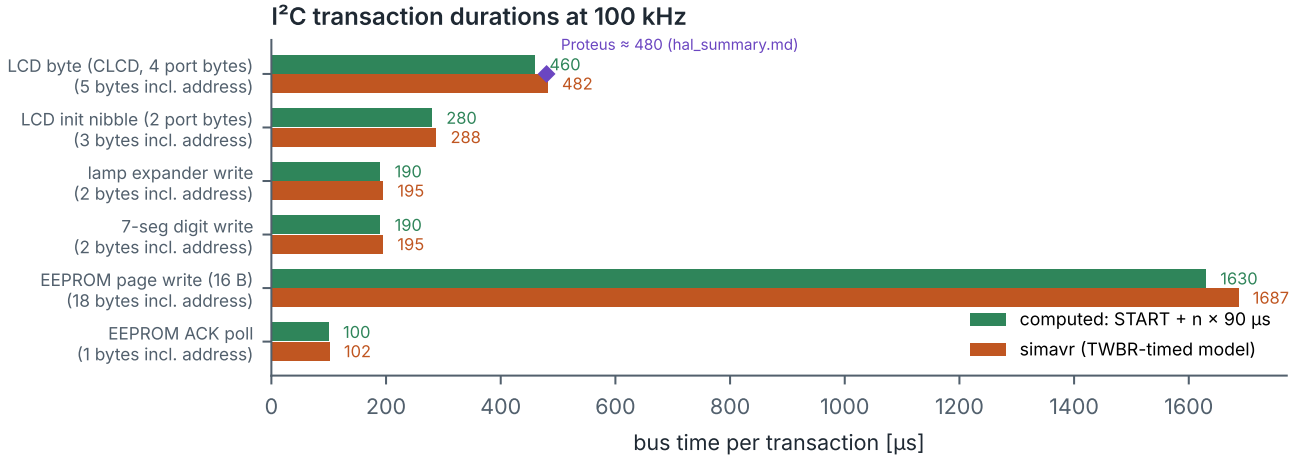


Figure 7.2 Transaction durations: computed (START + n × 90 µs) against simavr. The simavr value includes the CPU work between bytes; the Proteus value for the LCD byte is from hal_summary.md. measured in simavr (cycle-accurate simulation of app.elf)

Transaction	Bytes	Computed µs	simavr µs	Where, how often
LCD byte (4 port values)	5	460	473–482	LCD_BUF, at most once per 5 ms
Lamp write	2	190	195	LIGHT, per toggle
7-seg digit write	2	190	195	HEATER, only when a digit changes (≤ 2 per call)
EEPROM page write	18	1 630	1 678–1 687	ESTORE, one per 20 ms while pages are dirty
EEPROM ACK poll	1	100	102	ESTORE, the slot after a page write
EEPROM image read (boot)	2 + 209	19 010	19 502	once, before the scheduler

Worst-case bus time per slot (computed, one slot per tick): 5 ms slot 460 + 190 = 650 µs; 10 ms slot 190 + 1630 = 1820 µs; 100 ms slot 2 × 190 = 380 µs (+ 208 µs ADC); 500 ms slot 380 µs; 1 s slot 0. The bus is never busy for more than 1.82 ms in a row, and at most 650 µs of every 5 ms slot. Measured in simavr, the bus was busy 0.9 % of the time over the main run and 0.9 % over the stress run.

7.4 Execution times and CPU load

Interrupt service routines

ISR	Work	simavr cycles	+ entry (computed)	µs	Rate	CPU share
Timer2 COMP → SCHED_voidTick	tick++, 5 counters, reloads	235–261	+7	16.8 max	1000/s	1.56 %
USART RXC → RINGBUF_u8Put	read UDR, store 1 byte	122	+7	8.1	≤ 962/s	≤ 0.78 %
USART UDRE → RINGBUF_u8Get	send 1 byte or UDRIE off	124–140	+7	9.2	≤ 962/s	≤ 0.88 %

"Entry" = 4 cycles of interrupt response + 3 for the `jmp` in the vector table. The tick ISR is mostly register saving: avr-gcc saves 15 registers because the ISR calls the callback through a pointer, and the callee could use any call-clobbered register. Inlining the counter loop into the vector would roughly halve it, but the callback style of the course drivers was kept on purpose.

Loop slots

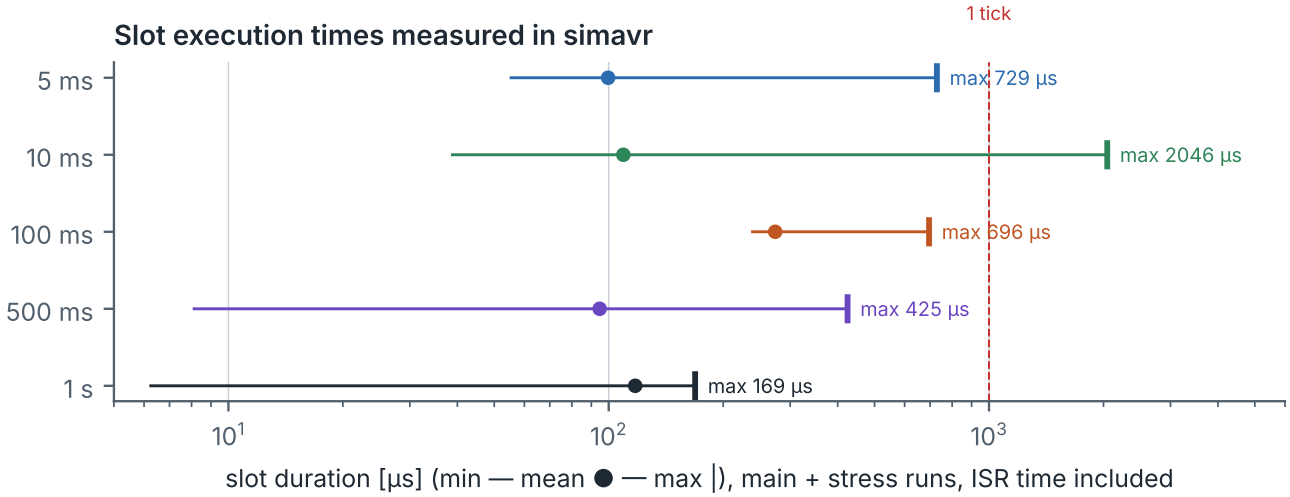


Figure 7.3 Measured slot durations over both runs (≈ 9300 + 4900 slots); ISR time that falls inside a slot is included.
measured in simavr (cycle-accurate simulation of app.elf)

Slot	Worst-case composition (computed)	Computed bound	simavr max	simavr mean
5 ms	LCD byte 460 + lamp write from a terminal command 190 + one print step ≤ 64 B (≈ 0.15 ms) + 1 tick ISR	≈ 0.82 ms	729 µs	100 µs
10 ms	EEPROM page 1630 + keypad lamp write 190 + UILOC redraw (RAM) + scans + 2 tick ISRs	≈ 2.15 ms	2 046 µs	109 µs
100 ms	2 ADC 208 + 2 digit writes 380 + control code	≈ 0.70 ms	696 µs	274 µs
500 ms	2 digit writes 380 + LED, buzzer	≈ 0.43 ms	425 µs	95 µs
1 s	status redraw into RAM, idle counters	≈ 0.2 ms	169 µs	117 µs

RESULT — THE "≈ 2.2 MS WORST LOOP PASS" OF THE README

The longest single slot is the 10 ms slot in which an EEPROM page write (1.63 ms of bus) meets a keypad lamp toggle (0.19 ms). Computed bound ≈ 2.15 ms; measured in simavr 2.05 ms (stress run, t = 10.15 s). **The README's 2.2 ms is a correct, slightly conservative bound for one slot.** Under the worst alignment of §7.7, several slots can follow each other back-to-back, so the loop can be busy for up to ≈ 2.7 ms in a row (computed) without any flag being lost.

CPU utilisation

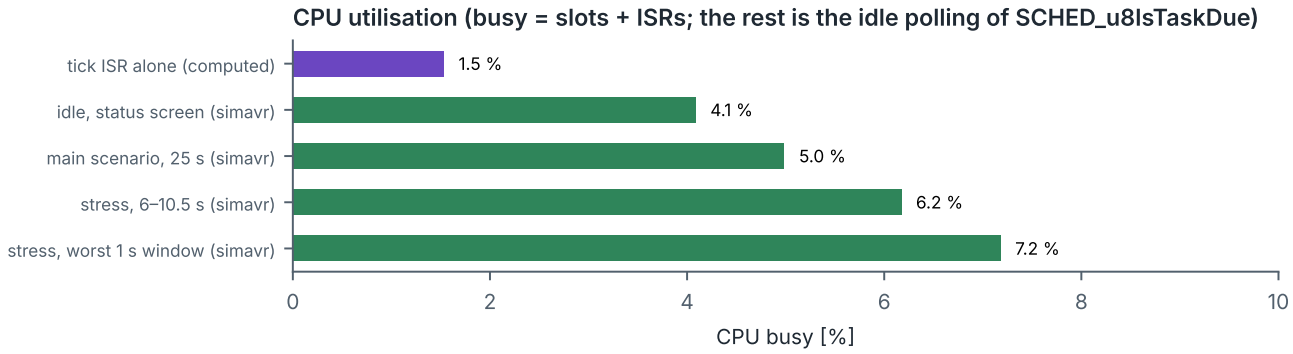


Figure 7.4 CPU busy time = slots + ISRs outside slots. The rest is idle polling. measured in simavr (cycle-accurate simulation of app.elf)

Computed long-term average from the slot means: 200 × 100 µs + 100 × 109 µs + 10 × 274 µs + 2 × 95 µs + 1 × 117 µs per second ≈ 3.4 % (ISRs inside slots included), in line with the 5.0 % measured over the main run. Even the worst 1-second window of the stress run uses 7.2 % of the CPU, leaving more than 90 % idle. The limiting resource is not CPU time but the length of single blocking transfers: an EEPROM page write is the longest thing the loop ever does.

7.5 Latency chains

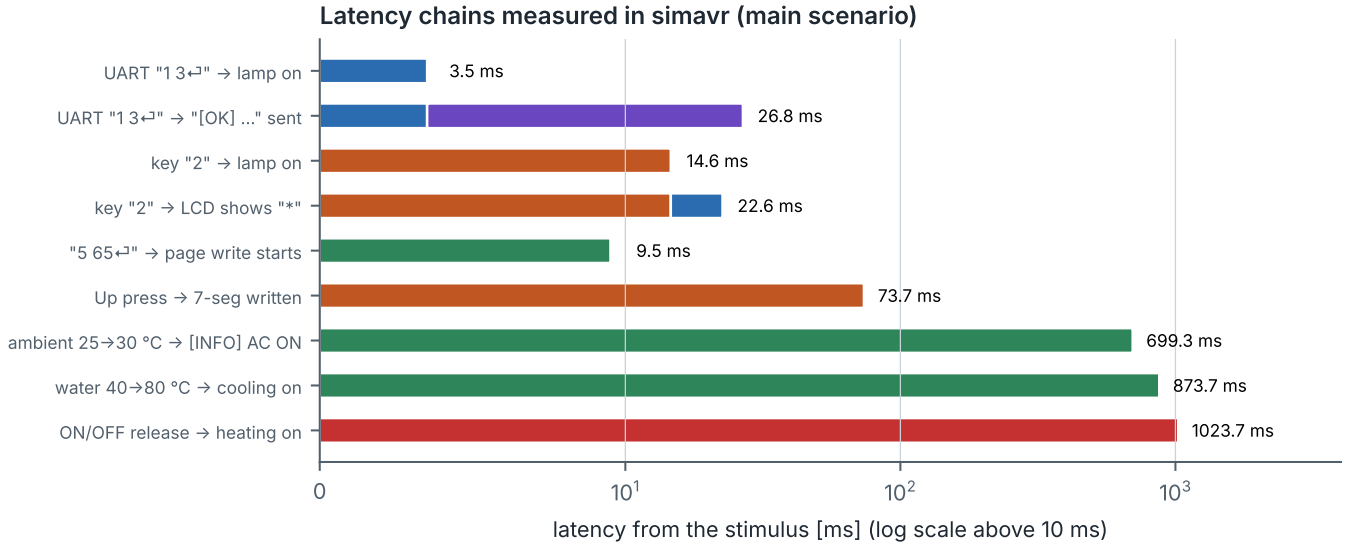


Figure 7.5 Measured latency from stimulus to effect. measured in simavr (cycle-accurate simulation of app.elf)

Chain	Computed expectation	simavr
Keypress → LCD update	debounce: 2 scans → accepted 10–20 ms after the press; UILOC acts in the same pass; LCD_BUF needs a cursor call + a character call → **15–30 ms** for one changed cell (+ 5 ms per extra cell)	lamp 14.6 ms · LCD 22.6 ms
UART command → actuator	Enter received → the next 5 ms slot reads the line (0–5 ms; only if the print queue is empty and ≥ 64 TX bytes are free) → lamp write → **0.2–5.2 ms**; the reply follows at 1.04 ms per byte	3.5 ms · reply done 26.8 ms
Up / Down press → 7-seg change	debounce 20–30 ms → setting mode starts in the same 10 ms slot → the value is shown by the next 100 ms task → **20–130 ms** (see the blink corner case below)	73.7 ms
Heater turn-on → first decision	release debounce 20–30 ms → average reset → 10 samples at 100 ms → **0.92–1.03 s** (HTR-08)	1 024 ms
Water step → element change	a step from 40 to 80 °C with set = 70: the average exceeds set + 5 = 75 °C after 9 new samples ((1·40 + 9·80) / 10 = 76) → **0.8–0.9 s**, plus up to 100 ms until the first new sample	874 ms

Chain	Computed expectation	simavr
Ambient step → AC	25 → 30 °C: average > 28 °C after 7 samples → **0.6–0.7 s**, then the `[INFO]` line ≈ 20 ms	699 ms
EEPROM write-behind	change → next 10 ms slot writes the page (0–10 ms) → the slot after polls (10 ms) → saved after **10–20 ms** per page; never more than 1.7 ms of one slot	page after 9.5 ms, ACK after +9.9 ms
Setting-mode timeout	timer = 50, decremented by each 100 ms task → mode ends 4.9–5.0 s after the last accepted press (HTR-12)	≈ 4.94 s
Blink periods	phase toggled every 500 ms → 1 s period for the display (HTR-11) and the heater LED (HTR-13)	500.0 ms steps
Debounce	buttons 3 samples → 20–30 ms; keypad 2 scans → 10–20 ms	—
3rd wrong login → lockdown	inside the same `SEC_u8Login` call → **< 1 ms**; buzzer beat 500 ms	0.6 ms

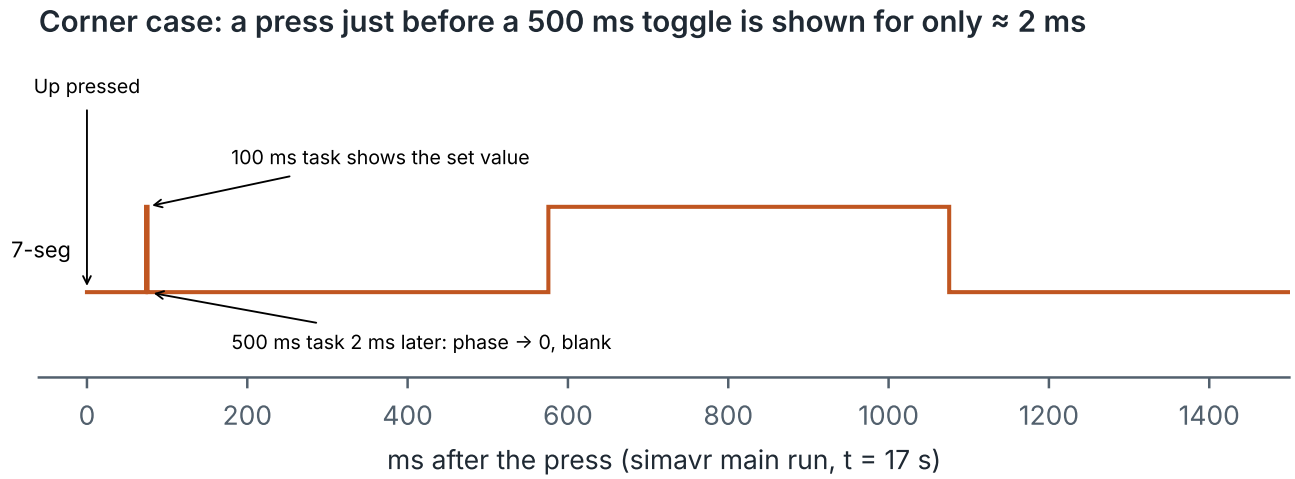


Figure 7.6 The blink corner case seen in simavr: the press is shown by the 100 ms task, and the 500 ms task blanks it 2 ms later.
measured in simavr (cycle-accurate simulation of app.elf)

Why the corner case exists. `HEATER_voidRestartSetting` sets the blink phase to 1 but leaves the 500 ms flag on its own fixed grid. Every 5th 100 ms task (tick ≡ 501 mod 500) runs 2 ms before a 500 ms task. A press that is first shown by that task is blanked almost at once, and the new value is next seen at the following "on" phase, ≤ 500 ms later. HTR-11 still holds (blinking at a 1 s period; the value is never wrong), but "every change is shown immediately" is met only in about 4 of 5 presses. A fix would be to restart the blink timing at each press, for example by counting the phase in the 100 ms task instead of using the 500 ms flag.

7.6 Jitter, overruns and headroom

Release jitter

A flag set by the ISR is served when the loop next reaches that `if`. With an otherwise idle loop, the delay is the remaining tick ISR plus at most one pass through the five checks. simavr measured the delay from the start of the setting tick ISR to the start of the slot:

Slot	min µs	mean µs	max µs (both runs)
5 ms	16.3	21.6	36.1
10 ms	16.3	21.6	35.8
100 ms	16.3	21.9	26.6
500 ms	16.3	20.9	26.2
1 s	16.8	21.9	25.9

Worst alignment (computed)

Because the tasks are released on different ticks, a slot can only be delayed by a slot released before it that has not finished. The tightest sequence is ticks 1001–1005 (mod 1000), using the computed bounds of §7.4:

Tick	Flag	Starts (worst)	Ends (worst)	Release delay	Margin to its next release
1001	100 ms	1001.00	1001.70	0	99.3 ms
1002	10 ms	1002.00	1004.15	0	7.9 ms
1003	500 ms	1004.15	1004.58	1.15 ms	498.4 ms
1004	1 s	1004.58	1004.78	0.58 ms	999.2 ms
1005	5 ms	1005.00	1005.82	0	4.2 ms

The loop is busy from 1002.00 to 1004.78 ms, a 2.8 ms stretch, once per second at worst. No release is delayed by more than 1.15 ms, and no flag comes anywhere near being set a second time before it is served. The smallest margin, 4.2 ms, belongs to the 5 ms task. An **overrun** — a flag set again before it was served — needs a slot chain longer than the period of a waiting task; with normal bus behaviour that cannot happen. Both simavr runs ended with `SCHED_u16GetOverruns() = 0`, as did `test_service` over 10 s.

Fault case: stuck I²C bus (computed)

If a slave holds SDA low, every TWI wait ends after the 1.33 ms timeout, and each driver gives up after its first failed step:

- LCD_BUF then pauses 1 s;
- ESTORE goes to FAULT after its first failed page write;
- LAMP and SEVEN_SEG return an error and retry at their next call.

Worst slots then become about 1.5 ms (5 ms), 2.8 ms (10 ms: lamp + EEPROM attempt), 2.9 ms (100 ms: two digits + ADC) and 2.7 ms (500 ms). In the 1001–1005 chain this can delay the 5 ms task by a few milliseconds and, in the worst case, count an overrun of the 5 ms flag (one LCD byte step lost, no wrong output). The loop never hangs. That is exactly what the timeout was added for (§2.8); `test_mcal` checks a timeout within 2 ms with SDA forced low.

Headroom

- **CPU:** ≥ 92 % idle in every measured second.
- **Single-slot budget:** the longest slot (≈ 2.1 ms) uses 21 % of its own 10 ms period, and leaves ≥ 2.8 ms before the 5 ms task that follows it.
- **RAM:** ≈ 1.3 KB for the stack (§1.6).
- **Flash:** 16.4 KB free.

The design could take a much higher I²C speed, more tasks or a faster terminal before timing became a constraint. The first limit would be the length of single blocking transfers, which is the place to look if features are added.

CHAPTER 8

Verification

In this project "verified" means: it builds with `-Wall` and zero warnings, the logic was traced by hand against the requirement, and a test step for it was run in Proteus (specification §2). This chapter lists the test programs, summarises the recorded Proteus runs, maps every requirement to its code and its tests, and records what failed along the way and what is still open.

8.1 Test environments

Each layer was finished with its own test program before the next layer started (specification §3). A test program prints one `[PASS]` / `[FAIL]` line per check on the terminal, `MANUAL:` lines for things a person must do or look at in Proteus, and a summary line; a status LED on PA3 shows running / all passed / failure.

Program	Check calls in the source	What it proves	Proteus project
`test_base`	27	the original course drivers, before and after the Phase 0/2 fixes (built with LTO)	simulation/design/base_wiring_phase0
`test_mcal`	40	JTAG bit, ADC sync after async, Timer0 0/50/100 %, Timer1 ICR1 = 39999 and pulse widths, 1000 Timer2 interrupts in 1.000 s, UBRR = 103, RX callback, 300-byte TX burst without loss, TWI ACK/NACK and timeout, EXTI with and without callback	layer_tests/1_mcal_test
`test_hal`	65	LCD through LCD_BUF (longest update < 1 ms), keypad once per press, button events, 7-segment (by eye), LM35, outputs, lamp chase with read-back, fan/servo/dimmer ramps, EEPROM page write, ready flag and persistence	layer_tests/2_hal_test
`test_service`	140	RINGBUF, MAVG, FMT edge cases; SCHED 2000/1000/100/20/10 flags in 10 s with 0 overruns; TERM editor; EVQ order/drop/mute; ESTORE defaults, magic page last, one byte = one page write, write during write cycle, fault; USERDB all rules and the write gate	layer_tests/3_service_test
`test_app`	42	automatic: 3-strike counters per source, permissions, dimmer rounding, set-temperature clamp, both hysteresis bands, "no decision before 10 samples", save after 5 s, ALARM latch, keypad arbitration; then the full super-loop for the interactive steps	layer_tests/4_app_test_partA, 5_app_test_partB

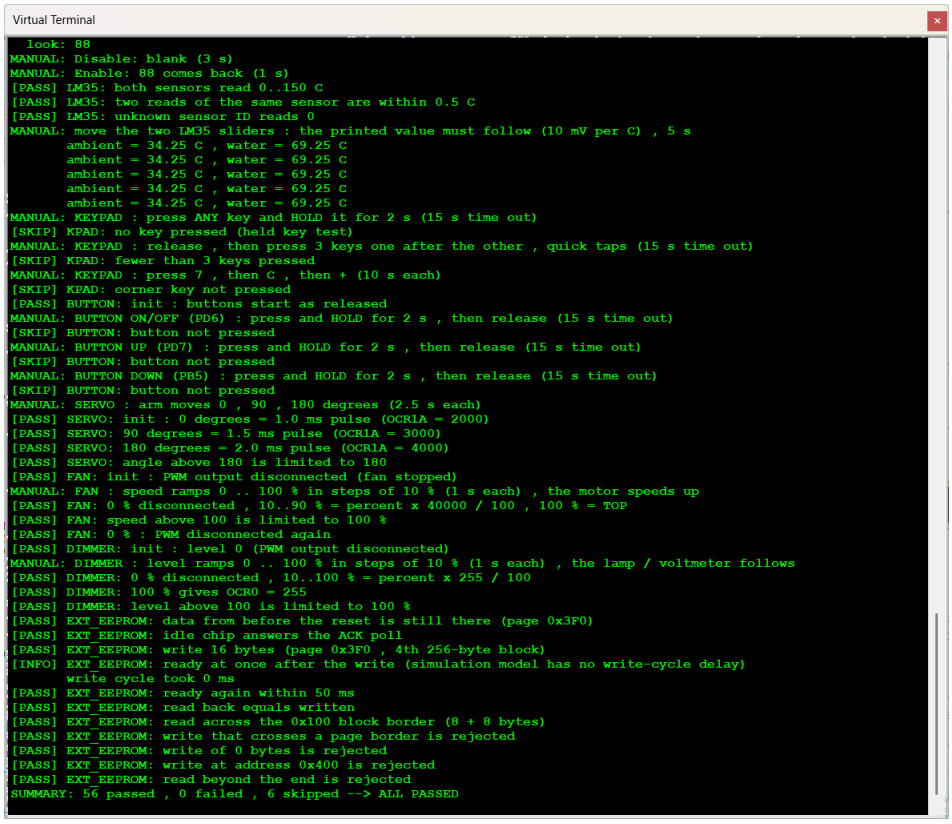


Figure 8.1 HAL layer test output in the Proteus terminal (presentation capture).
measured in Proteus (repository capture)

8.2 The Proteus test plan and the 130 recorded APP steps

`docs/test_plan.md` §3 defines 78 requirement steps (T-01 … T-78) for the full application. The APP layer was then run block by block in Proteus with `test_app` and recorded on 2026-10-04 in two sequence documents: **part A, 48 steps in 9 blocks** (the features without a UI), and **part B, 82 steps in 13 blocks** (both UIs and the full super-loop). **All 130 steps passed.**

Part A — LIGHT, DOOR, CLIMATE, HEATER, ALARM, SEC

Block	Steps	Covers
1 Automatic part	2	all automatic checks, then live loop
2 Start-up state	3	T-40, T-51
3 Lights and door	9	T-41, 42, 44, 45
4 Air conditioning	6	T-46 – T-50
5 Heater power, setting mode	12	T-52 – T-59, 70, 71
6 Element control and LED	9	T-63 – T-69
7 Persistence	2	T-61
8 Lockdown	5	T-73, T-74
Total	**48**	all PASS

Part B — UILOC, UIREM, full super-loop

Block	Steps	Covers
1 Automatic part and boot	4	T-33
2 Factory reset	5	T-01, 21, 76
3 Accounts	11	T-02, 03, 05, 06
4 Terminal commands, errors	13	T-36 – 39, 41 – 46, 70
5 Remote user permissions	5	T-13, 15, 36, 39
6 Keypad and LCD	12	T-22 – T-32
7 Arbitration	6	T-16 – T-20
8 Events	6	T-34, 35, 72
9 Persistence	4	T-07, 61
10 Counters and lockdown	8	T-08 – 12, 73, 74
11 Storage	4	T-77, 78
12 Admin password, final reset	4	T-21
Total	**82**	all PASS

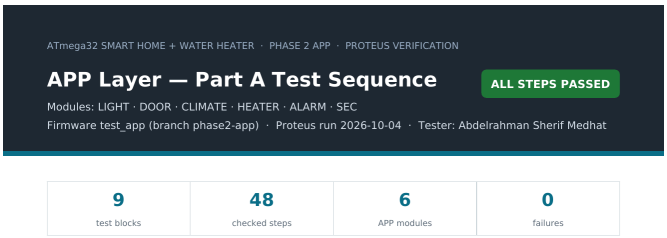


Figure 8.2 Header of the part A record (docs/test_app_partA_sequence.pdf). measured in Proteus (repository capture)

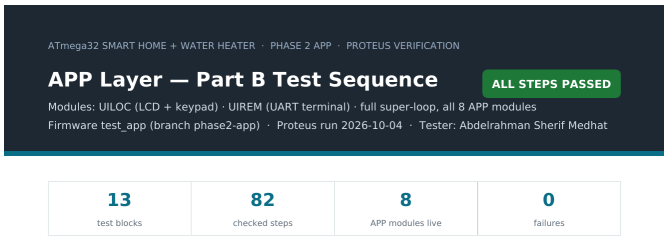


Figure 8.3 Header of the part B record. measured in Proteus (repository capture)

Captures not in the repository

The records above are tables of observed results. Three observations would be easier to check with a picture; they are left as marked placeholders:

PLACEHOLDER — Proteus capture not supplied

Proteus I²C debugger during T-60 / T-77: one 16-byte page write `S A0 A 10 A 32 … P` followed 10 ms later by the poll `S A0 A P`.

PLACEHOLDER — Proteus capture not supplied

Proteus oscilloscope channel B on PD4 (fan) with the AC on (100 %, flat high) and off (flat low) — T-50.

PLACEHOLDER — Proteus capture not supplied

Proteus 24C08 memory window after a first boot on a blank chip: 0x00 = A5, 0x01 = 01, 0x10 = 3C, 0x20 = "admin", 0x28 = "1234" — T-76.

8.3 Requirement traceability

Every requirement ID appears in the code as a `/* REQ-… */` comment where it is implemented. The README counts 126 lines carrying such tags across `src/` and `lib/`; a recount gives the same 126 lines, carrying 150 tags (129 of them in `.c` files), and every implemented ID — 42 of the 43; ALM-02 is not built — appears at least once.

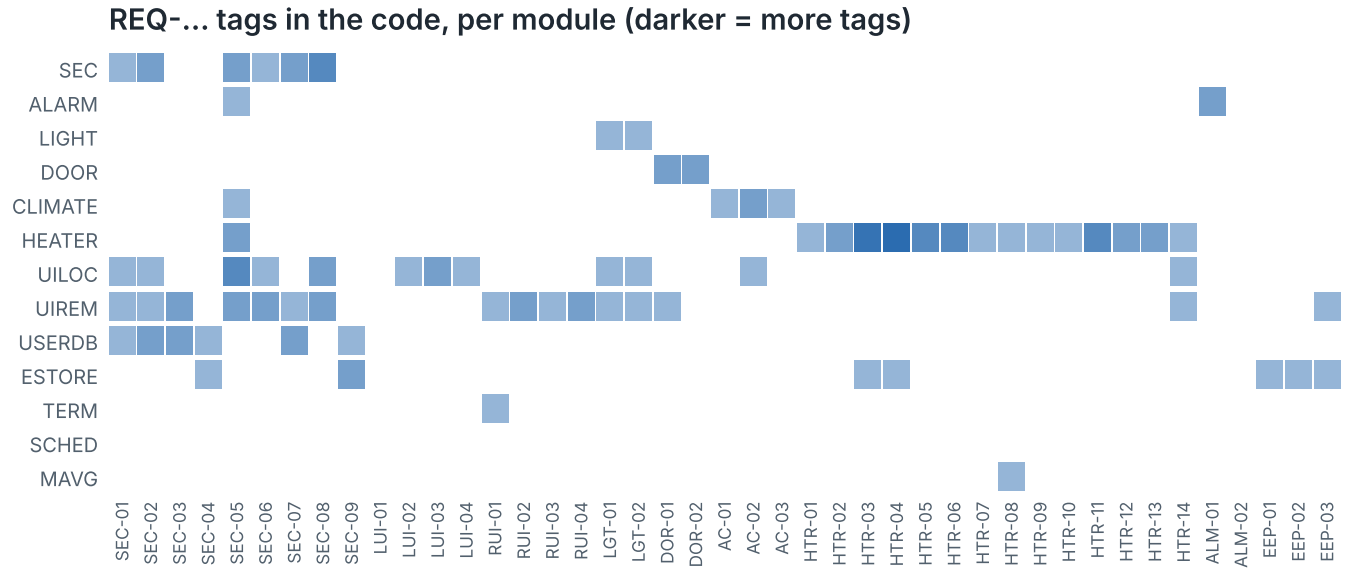


Figure 8.4 Requirement tags per module, counted from the source by the generator. drawn from the source code

Requirement	Code (tags found in)	Test plan steps	Recorded APP blocks
SEC-01 … 09	SEC, USERDB, ESTORE, UIREM, UILOC	T-01 – T-21	B2, B3, B5, B7, B9, B10, B12 · A8
LUI-01 … 04	UILOC	T-22 – T-32	B6
RUI-01 … 04	UIREM, TERM	T-33 – T-39	B1, B4, B5, B8
LGT-01, 02	LIGHT, UILOC, UIREM	T-24, 25, 40 – 43	A3, B4, B6
DOR-01, 02	DOOR, UIREM	T-13, 44, 45	A3, B4, B5
AC-01 … 03	CLIMATE	T-26, 46 – 50	A4, B6
HTR-01 … 14	HEATER, UILOC, UIREM	T-27, 51 – 72	A2, A5, A6, A7, B4, B8, B9
ALM-01	ALARM	T-08, 12, 73, 74	A8, B10
EEP-01 … 03	ESTORE, EXT_EEPROM	T-60, 76 – 78	A5, B2, B11

The full one-line-per-requirement table with step numbers is test_plan.md §4; §1.1 of this guide repeats it with the implementing modules.

8.4 Independent cross-check in simavr

The timing harness of chapter 7 doubles as a functional regression check of the binary, independent of Proteus. In the main scenario the terminal transcript produced by `app.elf` matches the documented protocol (uart_protocol.md) line for line, and the observed behaviour matches the requirements:

- the 13 first-boot pages written highest first;
- `[OK]` replies and `[INFO]` events;
- the keypad session and its events;
- the AC switching at the hysteresis limits;
- heating after 10 samples;
- the lockdown with buzzer and alert.

The scheduler counted 0 overruns in both runs. The harness, the scenarios and the analysis script are kept with the guide sources so the check can be repeated after any firmware change (Appendix H).

8.5 Design decisions and failures

The 7-segment display that failed (D-20)

The first display design used one 7447 BCD-to-7-segment decoder on PC2–PC5, two PNP digit drivers on PC6/PC7, and multiplexing from the 1 ms tick (`SEVEN_SEG_voidRefresh`). In Proteus it was unreliable: 37 showed as 33, and the counter showed the same odd symbol on both digits.

The test bench `firmware/experiments/hal_testing_v1` isolated the cause step by step:

- the port pins followed the code;
- JTAG was off (MCUCSR = 128);
- the generated code was correct (checked in the disassembly);
- a first sampling check gave a false error because it read the digit pins and the BCD pins separately while the tick switched digits — fixed with a single port snapshot.

The real symptom was both digits conducting at once through the slow PNP drivers. A 1 ms blanking interval (`SEVEN_SEG_BLANK_TICKS`) did not make it reliable.

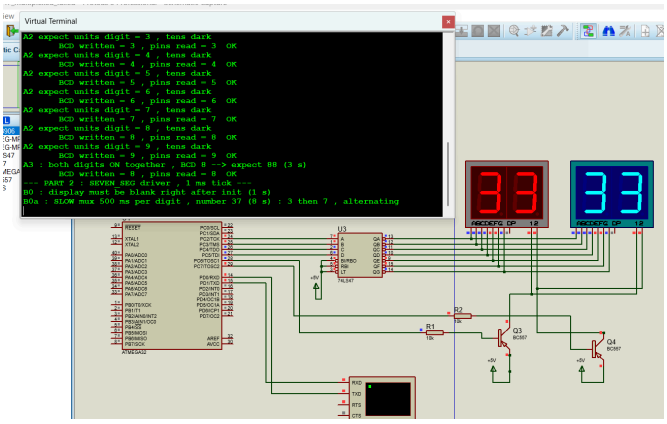


Figure 8.5 Version 1: 7447 decoder + multiplexed PNP digits (failed in Proteus). reused from the repository

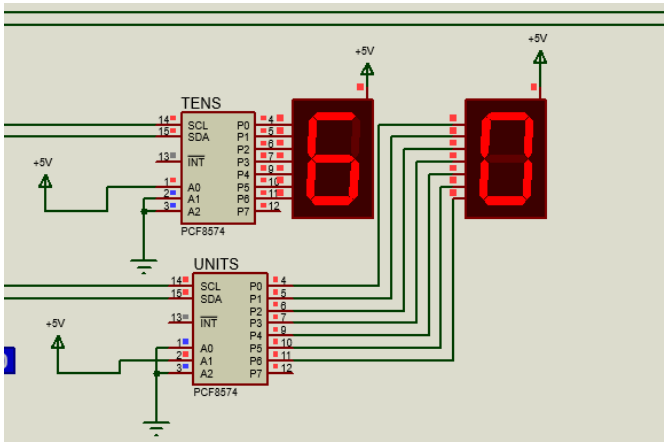


Figure 8.6 Version 2: one PCF8574 per digit (D-20, as built). reused from the repository

Decision D-20 (2026-10-03): two PCF8574 expanders, one per common-anode digit (tens 0x21, units 0x22). A second bench, `hal_testing_v2` (bus scan, 22 raw patterns per chip, all 100 numbers, bus sharing with the LCD and lamps, stress), showed the right digits. Gains: no multiplexing, no tick hook (the Timer2 ISR does no real work at all), no JTAG handling, six port pins freed (PC2–PC7). Cost: two more I²C devices and 190 µs of bus time per changed digit.

Other things that went wrong and were fixed

Problem	Cause	Fix
MCU reset on the first ADC/Timer0/EXTI interrupt	LTO removed ISRs declared only with `signal`	`__attribute__((signal, used, externally_visible))` on every ISR (§2.0)
Dimmer brightest at 0 %, off at 100 %	swapped COM bits; 100 % → OCR0 = 256 → 0	COM01:00 = 10; OCR0 = p × 255 / 100 (§2.4)
Sync ADC read hung after an async one	ADIE left on, the ISR stole ADIF	clear ADIE and ADIF before each polled conversion (§2.3)
Serial format corrupted on real hardware	read-modify-write of UCSRC read UBRRH	one write with URSEL (§2.7)
Loop blocked by a held key, a long text, a stuck bus or an EEPROM write	blocking course drivers	KPAD scan, TERM rings, TWI timeout, ESTORE write-behind
Buzzer silent in Proteus	model defaults 5 V / 12 Ω (0.4 A)	model set to 3 V / 150 Ω; NPN stage needed on real hardware (C-9)
≈ 100 B of RAM lost to a pointer table	compiler turned a text-returning `switch` into a RAM table	cases print directly (§4.9)

8.6 Known limitations

- **Not built in v1** (D-16): fan speed ramp (optional part of AC-03), PIR alarm (ALM-02), LDR street light. Pins PD2 and PA2 stay reserved.
- **Security:** passwords are stored and compared as plain text (D-13), and the UART link is not encrypted. Acceptable for a simulation project; hashing (and a salted record format) is the next step.
- **Timing:** the blink corner case and the up-to-100 ms display/LED lag at turn-off (§7.5, §5.6).
- **Untested paths in simulation:**
  - The Proteus 24C08 has no write delay, so the "chip still busy" branch of the ACK polling (NACK, retry) was never exercised. In simavr with a 5 ms model, the first poll, 10 ms after the write, always succeeded too. Only a write cycle above 10 ms, or a faulty chip, takes that branch.
  - UBRRH cannot be verified in Proteus (§2.7).
- **Robustness:** no watchdog is enabled; a firmware fault that stops the loop would not be recovered. No brown-out handling is configured in firmware (the BOD fuses are a hardware setting).

- **Hardware for a real board** (pin_map.md §4): NPN driver for the buzzer (C-9), series resistors on the ISP pins PB5–PB7 (C-4), a stronger base drive for the dimmer lamp transistor, AREF left open (C-5).
- **Power:** the idle loop polls flags at full speed; there is no sleep mode.

APPENDICES

Reference

Quick-reference tables for re-implementation and review. Every value is the one the firmware writes at run time.

A. Register quick reference

Register	Addr	Value / bits written	Written by	Meaning
TCCR2	0x45	WGM21 = 1, WGM20 = 0, CS22:20 = 100	TIMER2_voidInit	CTC, clk/64
OCR2	0x43	249	TIMER2_voidSetOCR	1 ms period
TIMSK	0x59	OCIE2 (bit 7) = 1	SCHED_voidStart	tick interrupt
TCCR0	0x53	WGM01:00 = 11, CS02:00 = 010, COM01:00 = 10 / 00	DIMMER via TIMER0	fast PWM, clk/8, non-inv. / off
OCR0	0x5C	p × 255 / 100	TIMER0_GeneratePWM	dimmer duty
TCCR1A	0x4F	WGM11 = 1, WGM10 = 0, COM1A1:0 = 10, COM1B1:0 = 10 / 00	TIMER1	mode 14, OC1A/OC1B non-inv.
TCCR1B	0x4E	WGM13:12 = 11, CS12:10 = 010	TIMER1_voidInit	mode 14, clk/8
ICR1H:L	0x47:46	39999 (0x9C3F), high byte first	TIMER1_voidInit	TOP → 50 Hz
OCR1AH:L	0x4B:4A	2000 … 4000	SERVO	servo pulse
OCR1BH:L	0x49:48	39999 (100 %)	FAN	fan duty
UBRRH / UBRRL	0x40 / 0x29	0 / 103	USART_voidInit	9615 baud
UCSRB	0x2A	RXEN, TXEN, RXCIE; UDRIE on demand	USART, TERM	RX/TX + interrupts
UCSRC	0x40	0x86 (URSEL, UCSZ1, UCSZ0)	USART_voidInit	8N1
TWBR / TWSR	0x20 / 0x21	72 / prescaler 00	TWI_voidMasterInit	100 kHz
TWCR	0x56	TWEN; TWINT|TWEN|(TWSTA/TWSTO/TWEA) per step	TWI	bus actions
ADMUX	0x27	REFS1:0 = 11, ADLAR = 0, MUX = 0 / 1	ADC	2.56 V, channel
ADCSRA	0x26	ADEN, ADPS2:0 = 111; ADSC per conversion; ADIF cleared by writing 1; ADIE = 0	ADC	125 kHz, polled
DDRx / PORTx	—	see Appendix B	DIO via HAL inits	pin direction, level, pull-up
SREG.I	0x5F	SEI after all inits; CLI/SEI pairs in SCHED/TERM	GIE	global interrupts

B. Full pin table

Pin	DIP	Dir	Function	Active	Owner (_cfg.h)
PA0	40	analog	LM35 ambient (ADC0)	10 mV/°C	LM35
PA1	39	analog	LM35 water (ADC1)	10 mV/°C	LM35
PA2	38	—	LDR, reserved (not used in v1)	—	—
PA3	37	out	heater LED	high	LED
PA4–PA7	36–33	in, pull-up	keypad rows A–D	low	KPAD
PB0, PB1, PB2, PB4	1, 2, 3, 5	out	keypad columns 1–4	low while scanned	KPAD
PB3	4	out (OC0)	dimmer PWM 7.8 kHz	high	DIMMER
PB5	6	in, pull-up	heater Down button	low	BUTTON

Pin	DIP	Dir	Function	Active	Owner (_cfg.h)
PB6	7	out	heating element (SSR)	high	RELAY
PB7	8	out	cooling element (SSR)	high	RELAY
PC0 / PC1	22 / 23	TWI	I²C SCL / SDA, 4.7 kΩ pull-ups	—	TWI hardware
PC2–PC7	24–29	—	spare (freed by D-20; PC2–PC5 = JTAG on a real chip)	—	—
PD0 / PD1	14 / 15	RXD / TXD	UART terminal	—	USART hardware
PD2	16	—	PIR (INT0), reserved	—	—
PD3	17	out	buzzer	high	BUZZER
PD4	18	out (OC1B)	AC fan PWM 50 Hz	high	FAN
PD5	19	out (OC1A)	door servo, 1–2 ms pulse	—	SERVO
PD6 / PD7	20 / 21	in, pull-up	heater ON/OFF / Up button	low	BUTTON
RESET, VCC, GND, AVCC, AREF, XTAL1/2	9, 10, 11/31, 30, 32, 13/12	—	reset button + 10 kΩ; +5 V; 0 V; +5 V; open; 16 MHz	—	—

C. EEPROM byte map (24C08, block 0)

Address	Size	Page	Macro	Content / default
0x00	1	0	ESTORE_ADDR_MAGIC	0xA5 = valid data
0x01	1	0	ESTORE_ADDR_VERSION	0x01 = layout version
0x02–0x0F	14	0	—	reserved, 0xFF
0x10	1	1	ESTORE_ADDR_HEATER_SET	set temperature °C, default 60 (0x3C)
0x11–0x1F	15	1	—	reserved, 0xFF
0x20–0x27 / 0x28–0x2F	8 + 8	2	ESTORE_ADDR_ADMIN	admin name "admin" / password "1234", 0x00-padded
0x30, 0x40, 0x50, 0x60, 0x70	16 each	3–7	ESTORE_ADDR_REMOTE_USERS	remote users 0–4 (empty = first byte 0x00/0xFF)
0x80, 0x90, 0xA0, 0xB0, 0xC0	16 each	8–12	ESTORE_ADDR_KEYPAD_USERS	keypad users 0–4 (digits only)
0xD0–0x3FF	816	13–63	—	not used (test_service uses 0x3E0 as a marker)

D. UART command list

9600 8N1, line end `\r\n` sent, `\r`, `\n` or `\r\n` accepted; maximum 16 characters per line; one-argument commands accept the argument on the same line (`1 3`).

#	Role	Command	Argument	Reply
1	all	Lamp on/off (toggle)	1–5	`[OK] Lamp 3 is now ON`
2	all	Dimmer level	0–100 step 10	`[OK] Dimmer is now 70%`
3	all	AC status	—	`AC is ON. Room temperature 29C.`
4	all	Heater status	—	`Heater is ON (HEATING). Water 52C, set 60C.`
5	all	Heater set temperature	35–75 step 5	`[OK] Heater set temperature is now 65C`
6	all	System status	—	6–7 line block
7 / 8	admin	Open / close the door	—	`[OK] Door is now OPEN` or `Door is already open.`

#	Role	Command	Argument	Reply
9	admin	Keypad control allow / block (toggle)	—	`[OK] Keypad control is now ALLOWED`
10 / 12	admin	Add remote / keypad user	name, then password (echo *)	`[OK] Remote user bob added.`
11 / 13	admin	Remove remote / keypad user	name	`[OK] Remote user bob removed.`
14	admin	List users	—	names, never passwords
15	admin	Change admin password	4–8 chars (echo *)	`[OK] Admin password changed.`
16	admin	Factory reset	YES	`[OK] Factory reset done. Admin is admin / 1234.` + logout
0 / ?	all	Logout / show menu	—	`Goodbye admin.` / the menu

Errors: `[ERR] Unknown command.`, `[ERR] Not allowed for your role.`, `[ERR] Input too long.`, range errors per command, `[ERR] Device not responding.` (lamp chip); each is followed by the menu. Unsolicited lines: `[INFO]` (changes made elsewhere), `[WARN]` (EEPROM), `[ALERT]` (lockdown).

E. Glossary

ACK polling	checking whether an I²C EEPROM has finished its write cycle by sending its address and seeing whether it acknowledges
CTC	Clear Timer on Compare match: the timer counts to OCR and restarts at 0
Debounce	accepting a mechanical contact's new level only after it has been stable for several samples
Dirty page	an EEPROM page whose RAM copy differs from the chip and still has to be written
Fast PWM	timer mode in which the output is set at BOTTOM and cleared at the compare match; frequency = clock / (TOP + 1)
Hyperperiod	the least common multiple of all task periods; the schedule repeats after it (here 1000 ms)
Hysteresis	two different thresholds for switching on and off, so that a signal near one threshold does not make the output chatter
Jitter	variation in the start time of a periodic task
LTO	link-time optimisation: the compiler optimises across files at link time and removes unused functions
MCAL / HAL	Microcontroller / Hardware Abstraction Layer: register drivers / part drivers
Overrun	a task flag set again before the previous one was served (one period lost)
Quasi-bidirectional	a pin with a strong pull-down and a weak pull-up (PCF8574): it can sink current but hardly source it
Ring buffer	a fixed array used as a FIFO with wrap-around head and tail indexes
Super-loop	the endless `while(1)` in `main()` that calls tasks when their flags are set
Write-behind	a write that updates a fast copy at once and the slow medium later, in the background
WCET	worst-case execution time

F. Interview questions about this project

Question	Short answer
Why a layered architecture on an 8-bit MCU?	Each layer can be tested on its own (one test program per layer), pins live in one place, and a part can be replaced without touching the application — the 7-segment redesign changed one HAL driver and nothing above it.
How do you get an exact 1 ms tick at 16 MHz?	Timer2 CTC, prescaler 64 → 250 kHz, OCR2 = 249 → 250 counts = 1.000 ms; a compile-time `#error` rejects an F_CPU that cannot give an exact millisecond.
What does your ISR do, and why so little?	Counts the tick and sets task flags (≈ 16 µs). Short ISRs keep interrupt latency low for the UART and avoid sharing complex state with the main loop.
How are the ISR and the main loop kept consistent?	`volatile` on shared data; multi-byte values and read-modify-write of flag bytes with interrupts off; single-producer/single-consumer ring buffers with 8-bit indexes need no lock.

Question	Short answer
Why can two tasks never start in the same millisecond?	Their first-due ticks are 5, 12, 101, 503 and 1004: different modulo 5, and every period is a multiple of 5 ms.
What is the worst-case loop pass and how did you check it?	The 10 ms slot with an EEPROM page write and a lamp toggle: ≈ 2.15 ms computed, 2.05 ms measured in a cycle-accurate simulation; no overruns.
Why does the LCD not block the loop?	LCD_BUF keeps a wanted and a shown screen in RAM and sends at most one byte (≈ 0.46 ms) per 5 ms call.
How do you write an EEPROM without a 5 ms delay?	RAM image + dirty bits; every 10 ms one step: write one page, or one ACK poll. Highest page first so the magic byte is written last.
How do you send 400 bytes over a 9600-baud UART without blocking?	TX ring buffer + UDRE interrupt; the UI prints one line per 5 ms call and only when 64 bytes are free.
Why is the I²C driver polled and not interrupt-driven?	Every transfer is short and bounded (≤ 1.7 ms) and only the main loop uses the bus; an interrupt-driven master with a queue would be a much bigger module. Each wait has a timeout so a stuck bus cannot hang the loop.
How did you fit about 41 I/O lines on 32 pins?	LCD, lamps and both display digits on PCF8574 expanders and the EEPROM on the same I²C bus: 2 pins for 5 chips.
Why are the lamps active low?	A PCF8574 can sink 25 mA but sources only ≈ 0.1 mA, and it powers up with every pin high — which then means all lamps off.
How is the LM35 read without floating point?	With the 2.56 V reference one ADC step is 2.5 mV = 0.25 °C, so the raw value is the temperature × 4; thresholds are compared × 4 as well.
Explain the heater control law.	A 10-sample moving average at 100 ms; below set − 5 heat, above set + 5 cool, in between keep the state; no decision until 10 samples exist.
Why hysteresis for the AC?	A single threshold makes the fan chatter on noise; on above 28 °C and off below 21 °C gives a 7 °C dead band.
What happens on the 3rd wrong login?	SEC calls ALARM in the same call: heater and fan forced off, buzzer beeping, UIs locked; only a reset clears it, because the state is not stored.
How are keypad and remote users kept apart?	Separate lists in EEPROM; the admin record is checked only for the UART; the keypad is blocked while the admin is logged in unless allowed.
What did LTO break and how did you find it?	ISRs declared only with `signal` were removed as unused; the MCU reset on the first interrupt. Fix: `used, externally_visible`. Found by the Phase 0 smoke test of the course drivers.
Tell me about a design that failed.	The multiplexed 7447 display: digits ghosted in Proteus; a test bench ruled out the code and the pins, and the display moved to one I²C expander per digit, which also freed 6 pins and emptied the tick ISR.
How would you take this to real hardware?	Buzzer driver transistor, series resistors on the ISP pins, check the I²C rise times, enable a watchdog, keep AREF open, verify the EEPROM busy path with a real chip.

G. Documentation errata (code is the reference)

Places where a document in `docs/` or a code comment disagrees with the code or with its measured behaviour. The guide follows the code throughout.

#	Where	Says	Code / measurement
1	`TWI_cfg.h` comment, architecture.md §2 and §4.3	timeout ≈ 1 ms (6 cycles per wait pass)	the compiled wait loop takes 8 cycles → 2666 × 8 / 16 MHz = 1.33 ms
2	architecture.md §4.3	LCD power-up 46 ms	`_delay_ms` 40 + 5 + 1 + 2 = 48 ms + bus ≈ 50.5 ms computed; 50.6 ms simavr
3	architecture.md §4.2 table, "First due at"	0 / 2 / 1 / 3 / 4	first due on ticks 5 / 12 / 101 / 503 / 1004 (the offsets are added to one full period; the doc's own "as built" note agrees)
4	architecture.md §4.1	tick ISR "under 10 µs = 1 % CPU"	235–261 cycles (+7 entry) ≈ 15–17 µs ≈ 1.6 % CPU
5	architecture.md §5.6	ON/OFF release: elements, LED and display off at once	elements off at once; display and LED follow in the next 100/500 ms task (≤ 100 ms)

#	Where	Says	Code / measurement
6	`HEATER_voidRestartSetting` comment; test_plan T-55 wording	new set value "visible at once"	shown by the next 100 ms task; blanked again after 2 ms when that task precedes a 500 ms toggle (about 1 press in 5)
7	eeprom_map.md §4	invalid stored set temperature: "the heater uses 60 and stores it"	60 is used; it is written only at the next save (set-mode exit, turn-off, remote/keypad change)
8	eeprom_map.md §6	first boot: 13 pages ≈ 0.4 s in the background	20 ms per page (write slot + poll slot) → 260 ms (§7 of the same file says 260 ms); simavr 84.7–324.7 ms
9	pin_map.md §2	Proteus 24C08 model has TD_WRITE = 10 ms	specification.md, eeprom_map.md and hal_summary.md: the model has no write delay (observed in test_hal)
10	uart_protocol.md §6	`[ERR] Device not responding.` for a lamp or EEPROM bus error	used for the lamp chip only; EEPROM problems give `[WARN] EEPROM not responding…`
11	`test_app.c` header	heater LED with 330 Ω	pin_map.md: 220 Ω (wiring note only)

H. Regenerating this guide

The guide is generated, so a firmware change can be followed by a rebuild. Sources: `firmware/project_v1/docs/guide_src/`.

```
pip install matplotlib schemdraw playwright pypdf     # + graphviz (dot) and a Chromium for Playwright
cd firmware/project_v1/docs/guide_src
python build.py            # figures + PDF  ->  out/Technical_Guide.pdf
python build.py --no-figs  # PDF only

# timing cross-check (Linux): build simavr 1.6 with sim/simavr_v1.6_twi_adc.patch, then
#   avr-gcc ... -flto -o app.elf (flags in sim/README.md) ; ./harness app.elf sim/main.txt main
#   python sim/analyze.py      ->  data/results.json  (read by gnum.py)
```

- `gnum.py` reads `F_CPU` from `platformio.ini` and the constants from the `_cfg.h` files.
- Code excerpts are pulled from the sources by function name or line range; the build stops if one exceeds 15 lines.
- Figures are regenerated from the code tables and the simavr data.